Linear Systems on Graphs

Controllability of Structured Networks

Graduate Lecture Notes

Nam-Jin Park
Major of Control and Instrumentation Engineering, Pukyong National University

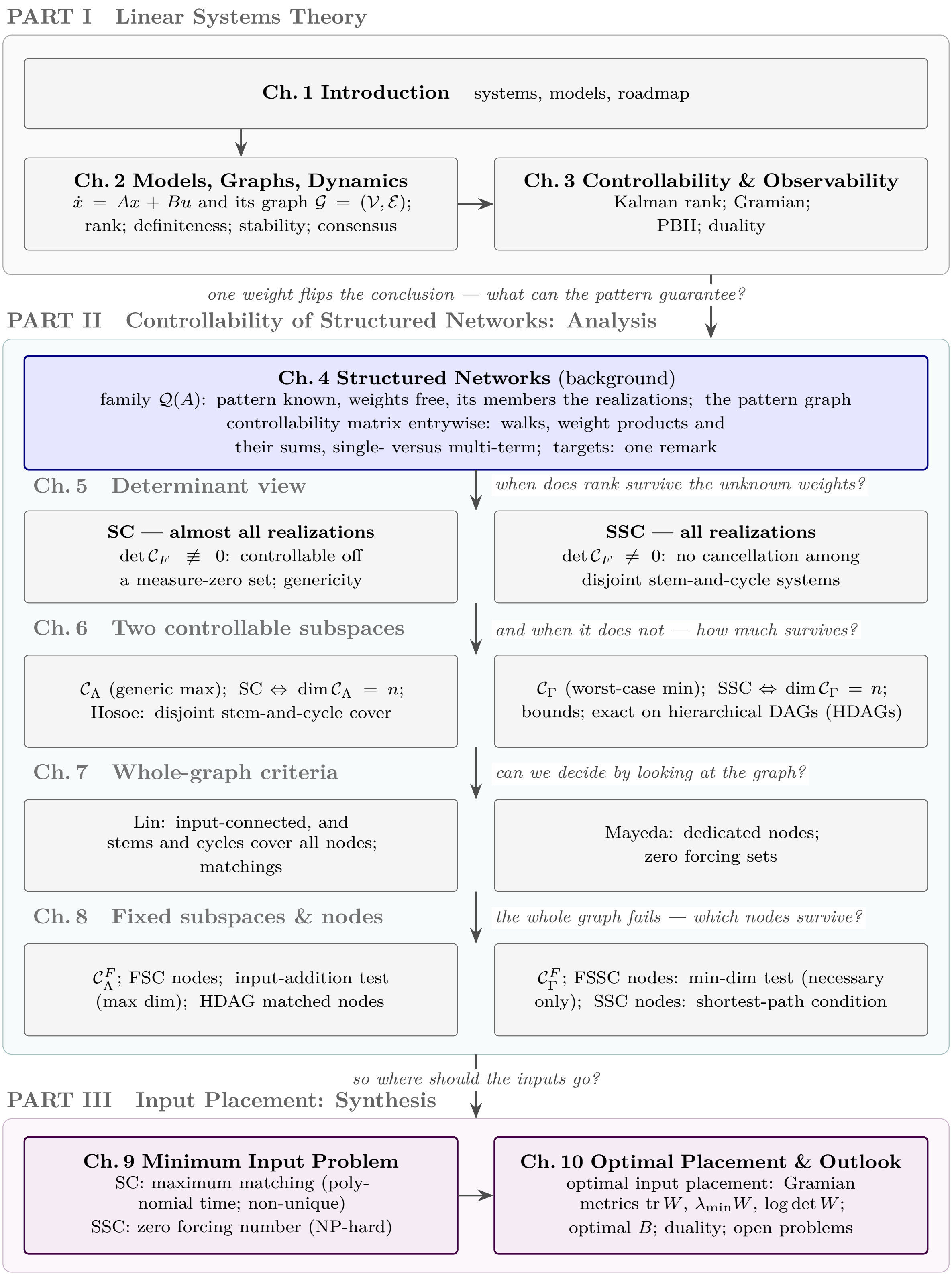
Fall 2026 • Version 1.3 (draft), 2026-08-31

Preface

These notes serve a one-semester graduate course on linear systems, with a second half on networks. Part I is the classical theory: the state-space model $\dot{x} = Ax + Bu$, $y = Cx$, its solutions and its stability, and the two questions that organize everything after it — can the input steer the state anywhere, and does the output reveal the state? The scope follows Chen [1].

Alongside the matrices we carry a picture. Draw one node for each state variable and one for each input and each output, and an arrow from j to i whenever j influences i — that is, whenever the corresponding entry of A, B or C is non-zero. A linear system is then a weighted directed graph, and the two questions become questions about that graph: which nodes does an input reach, and how does its effect spread?

Part II asks what survives when the numbers on the arrows are unknown. In a power grid, a robot team or a gene network, who influences whom is reliable knowledge; how strongly is usually not. Much of the theory survives anyway — controllability can often be decided, or shown to fail, from the arrows alone — and the arguments become combinatorial, organized around one polynomial, the determinant of the controllability matrix. Part III turns analysis into synthesis and asks where the inputs should be attached: how few suffice, and which placement steers the network with least effort. The figure on the following page shows how the book is organized.



Roadmap of the book. Part I develops classical linear system theory up to the Kalman rank condition; one perturbed entry flipping the controllability conclusion motivates Part II, where only the zero/nonzero pattern is reliable. Four analysis levels follow, each split into a cell for structural controllability (SC) and one for strong structural controllability (SSC) — almost all versus all realizations, every condition in both algebraic and graph-theoretical form — linked by the questions each level leaves open. Part III turns analysis into synthesis: the minimum input problem, then Gramian-optimal placement beyond the minimum.

Contents

Part I

Linear Systems Theory

Chapter 1

Introduction: Systems, Models, and the Plan of the Book

Why we study models rather than physical systems, what an analytical study of a system consists of, and where this book is headed: from a single plant with known parameters to networks whose interconnection pattern is the only reliable knowledge.

1.1 Physical Systems and Models

Engineering deals with physical systems — motors, vehicles, circuits, fleets of robots — but engineering *analysis* deals with models. The distinction is worth keeping sharp. A resistor with a fixed resistance is a model: the physical component varies with temperature and burns out beyond its rating. A rigid link of a manipulator is a model: the physical link flexes. Whenever we compute, we compute on the model, and the value of the computation is bounded by the quality of the model.

Which model is appropriate depends on the question being asked. A quadrotor is a point mass to a path planner, a rigid body to an attitude controller, and a set of coupled motor–propeller dynamics to a vibration analyst — three models, one aircraft, no contradiction. Throughout this book the word *system* means a model of a physical system, and we assume the modeling step has been done: our starting point is the model itself.

An analytical study proceeds in four stages: modeling, mathematical description, analysis, and design. Analysis splits further into *quantitative* questions — what is the response to this input? — and *qualitative* ones: is the system stable? can the input steer every state? does the output reveal the state? The qualitative questions are where design insight comes from, and they run through this book: from Chapter 3 onward, almost every page is about one qualitative property, controllability, asked under progressively weaker knowledge of the model.

1.2 From One Plant to Networks

Classical control theory grew around a single plant whose parameters are known — measured on a test bench, identified from data, printed on a datasheet. The state-space theory of Part I assumes that setting: a matrix A full of reliable numbers.

The systems that dominate modern applications are different. A power grid, a formation of mobile robots, a gene regulatory network, a social network: each is a collection of simple units coupled by an interconnection structure. For such systems the roles of knowledge are reversed. The *structure* — who is coupled to whom — is usually reliable: it is the wiring diagram, the communication topology, the regulatory map. The *edge weights* are usually not: they vary, they depend on operating point, they are expensive or impossible to measure at scale, and in a robot team they may be redesigned tomorrow.

This inversion is the premise of Part II. If the numbers in A cannot be relied on but the zero/non-zero pattern can, what remains of controllability? Remarkably much, and the answers are combinatorial: they are read off a graph. For that reason this book puts the graph at the very beginning of the theory — Chapter 2 introduces every linear system together with its graph — so that when the numbers are finally dropped in Chapter 4, the picture that remains is already familiar.

1.3 Book Outline

Roadmap of the three parts. Part II opens with vocabulary-building background (Chapter 4) and then proceeds through four *analysis* levels — definitions and the determinant view (Chapter 5), controllable subspaces (Chapter 6), whole-graph criteria (Chapter 7), node-level criteria (Chapter 8) — and Part III turns analysis into *synthesis*: the minimum input problem (Chapter 9) and optimal placement beyond the minimum (Chapter 10). Two axes run through every level: structural versus strong structural (almost all versus all realizations of the family, Chapter 4), and algebra versus graph (every condition stated in both forms). The roadmap figure in the Preface shows this organization on one page.

Prerequisites are an undergraduate course in control or linear systems and working linear algebra; Chapter 2 reviews, at graduate pace, exactly the material this book uses. Worked examples are small by design — most fit on four nodes and can be verified by hand — and each concept is illustrated on the smallest graph that exhibits it.

Sources. Part I follows the scope of [1]; the networks perspective draws on [2, 3].

Chapter 2

Foundations: Models, Graphs, and Dynamics

Everything the book needs from classical theory, in one chapter at review pace: the state-space model and — immediately — its graph, the linear-algebra background (rank, eigenstructure, definiteness), solutions and stability, and a closing application on a network: Laplacian dynamics and consensus. From here on, a linear system is *a weighted directed graph.*

2.1 State-Space Models

Throughout the book a system is linear, time-invariant, and lumped: its memory is a finite vector $x(t) \in \mathbb{R}^n$, the *state*, evolving as

$$\dot{x} = Ax + Bu, \qquad y = Cx + Du, \tag{2.1}$$

with input $u \in \mathbb{R}^m$, output $y \in \mathbb{R}^q$, and constant matrices $A \in \mathbb{R}^{n\times n}$, $B \in \mathbb{R}^{n\times m}$, $C \in \mathbb{R}^{q\times n}$, $D \in \mathbb{R}^{q\times m}$ (almost always $D = 0$ here). The state is the memory of the system: $x(t_0)$ together with $u(t)$ for $t \geq t_0$ determines the entire future.

Example 2.1 (Mass–spring–damper)**.** A mass m on a spring k with damping c and force input u obeys $m\ddot{q} = -kq - c\dot{q} + u$. With $x = (q, \dot{q})$,

$$A = \begin{bmatrix} 0 & 1 \\ -\frac{k}{m} & -\frac{c}{m} \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ \frac{1}{m} \end{bmatrix}, \quad C = \begin{bmatrix} 1 & 0 \end{bmatrix}$$

if the position is measured.

Example 2.2 (DC motor)**.** Armature current i and shaft speed ω with voltage input: $L\dot{i} = -Ri - k_e\omega + v$, $J\dot{\omega} = k_t i - b\omega$. With $x = (i, \omega)$,

$$A = \begin{bmatrix} -\frac{R}{L} & -\frac{k_e}{L} \\ \frac{k_t}{J} & -\frac{b}{J} \end{bmatrix}, \quad B = \begin{bmatrix} \frac{1}{L} \\ 0 \end{bmatrix}, \quad C = \begin{bmatrix} 0 & 1 \end{bmatrix}.$$

Two remarks close the section. Sampling Eq. (2.1) with period T gives a discrete-time model $x[k+1] = A_d x[k] + B_d u[k]$ with $A_d = e^{AT}$ (Section 2.7). And eliminating the state by the Laplace transform gives the *transfer matrix* $\hat{G}(s) = C(sI - A)^{-1}B + D$ — the external description; we use it sparingly.

2.2 Graph of Linear Systems

Here is the definition that sets the viewpoint of this book.

Definition 2.3 (Graph of a linear system)**.** The system Eq. (2.1) defines a weighted directed graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ with node set $\mathcal{V} = \mathcal{V}^S \cup \mathcal{V}^I \cup \mathcal{V}^O$ — one node per state, input, and output — and edges

$$(j, i) \in \mathcal{E} \iff [A]_{ij} \neq 0, \qquad (u_k, i) \in \mathcal{E} \iff [B]_{ik} \neq 0, \qquad (i, y_k) \in \mathcal{E} \iff [C]_{ki} \neq 0,$$

each carrying the corresponding entry as its weight. State nodes that receive an input edge are *driver* (or *leader*) nodes.

Note the direction: $[A]_{ij} \neq 0$ means state j *influences* state i, an edge from j to i; thus A acts as a weighted adjacency matrix (in the transpose convention, which we fix once and use everywhere). Every linear system can now be *drawn.*

Example 2.4 (A four-node network)**.** The system

$$A = \begin{bmatrix} 0 & 0 & -1 & 0 \\ 2 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 3 & 1 & 0 \end{bmatrix}, \qquad b = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \qquad c = \begin{bmatrix} 0 & 0 & 0 & 1 \end{bmatrix}$$

has the graph of Fig. 2.1: a cycle $1 \to 2 \to 3 \to 1$ plus the edges $2 \to 4$ and $3 \to 4$, the input driving node 1 and the output reading node 4. Matrix and picture carry identical information; the picture will often be the faster one to reason on. This system returns in Chapter 3.

$$A = \begin{bmatrix} 0 & 0 & -1 & 0 \\ 2 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 3 & 1 & 0 \end{bmatrix}, \quad b = e_1, \; c = e_4^\top$$

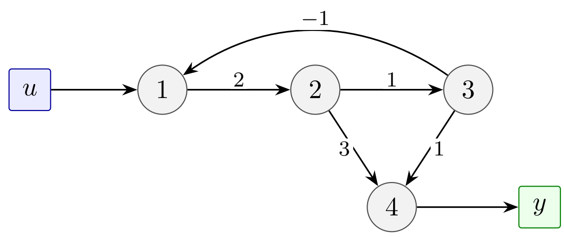


Figure 2.1: The four-node network of Example 2.4: the matrices (left) and their graph (right) are the same object. Input and output nodes are drawn as squares.

Part II will do exactly one thing to this picture: forget the numbers on the edges and keep only which edges exist.

2.3 Worked Models as Graphs

The graph view is not reserved for networks; even a textbook plant is a graph. For the mass–spring–damper of Example 2.1, $[A]_{12} = 1$ gives an edge $\dot{q} \to q$, the entries $-k/m$ and $-c/m$ give an edge $q \to \dot{q}$ and a self-loop at $\dot{q}$, and the input drives $\dot{q}$ (Fig. 2.2, left).

For a *network* the graph is the natural starting point rather than something derived from the matrix: four robots exchanging relative information over communication links form the graph directly, and the matrix is read off the picture (Fig. 2.2, right). The dynamics that such information exchange generates — consensus — closes this chapter in Section 2.9.

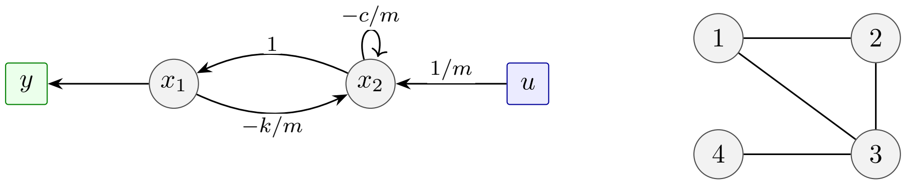


Figure 2.2: Left: the mass–spring–damper of Example 2.1 drawn as a graph. Right: a four-robot communication topology (undirected edges = bidirectional links); its dynamics are built in Section 2.9.

2.4 Rank and Linear Equations

The tool this book uses most often is the rank of a matrix: the dimension of its column space $\operatorname{Im} M = \{Mz\}$. The system $Mz = w$ is solvable iff $w \in \operatorname{Im} M$, and uniquely solvable iff in addition $\ker M = \{0\}$; $\operatorname{rank} M + \dim\ker M$ equals the number of columns. Two points to retain from this review:

Rank is a spanning question. For

$$M = \begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & 1 \\ 2 & 1 & 5 \end{bmatrix}$$

the third row is twice the first plus the second, so $\operatorname{rank} M = 2$: the columns span a plane. We will constantly ask "do these columns span $\mathbb{R}^n$?" — controllability is precisely such a question.

Rank never grows under multiplication: $\operatorname{rank}(MN) \le \min\{\operatorname{rank} M, \operatorname{rank} N\}$, and selecting rows or columns can only lower it. Both facts are used in every chapter.

2.5 Eigenstructure and Matrix Functions

If $Av_i = \lambda_i v_i$ with n independent eigenvectors, then $A = V\Lambda V^{-1}$ with $V = [v_1 \ \cdots \ v_n]$ and $\Lambda = \operatorname{diag}(\lambda_1, \dots, \lambda_n)$, and every polynomial or power series in A is computed on the eigenvalues; when eigenvectors are deficient the Jordan form plays the same role (we use it only at statement level).

Example 2.5. $A = \begin{bmatrix} 0 & 1 \\ -2 & -3 \end{bmatrix}$ has $\lambda_1 = -1$, $\lambda_2 = -2$ with eigenvectors $v_1 = (1, -1)$, $v_2 = (1, -2)$. This A serves again in Sections 2.7 and 2.8.

Part II works mostly with the plain *powers* A^k; the key identity of the book — entries of A^k enumerate the walks of length k (edge sequences that may revisit nodes, Definition 4.7) on the graph of Definition 2.3 — makes those powers combinatorial objects (Chapter 4).

2.6 Quadratic Forms and Definiteness

A symmetric $M = M^\top$ has real eigenvalues and an orthonormal eigenbasis, so the quadratic form $x^\top Mx$ is a weighted sum of squares along that eigenbasis. $M \succ 0$ (positive definite) iff all eigenvalues are positive, $M \succeq 0$ (positive semidefinite) iff none is negative; leading principal minors give the classical test.

Example 2.6. $\begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix} \succ 0$ (eigenvalues $1, 3$); $\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \succeq 0$ (eigenvalues $0, 2$ — the zero eigenvector $(1, -1)$ is a direction along which the form vanishes); $\begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}$ is indefinite (eigenvalues $3, -1$).

Quadratic forms are used three times later: for the graph Laplacian (Section 2.9), for the Lyapunov stability condition (Section 2.8), and for the Gramian-based placement metrics of Chapter 10.

2.7 Solutions of State Equations

The unique solution of Eq. (2.1) is

$$x(t) = e^{At}x(0) + \int_0^t e^{A(t-\tau)}Bu(\tau)\,d\tau, \tag{2.2}$$

a zero-input term carrying the initial state and a zero-state term accumulating the input along the flow. The matrix exponential is computed by series, by eigenstructure, or by taking the inverse Laplace transform of $(sI - A)^{-1}$.

Example 2.7. For A of Example 2.5, expanding the initial condition in the eigenbasis gives

$$e^{At} = \begin{bmatrix} 2e^{-t} - e^{-2t} & e^{-t} - e^{-2t} \\ -2e^{-t} + 2e^{-2t} & -e^{-t} + 2e^{-2t} \end{bmatrix};$$

both modes decay, as Section 2.8 confirms. Sampling with period T gives $A_d = e^{AT}$ exactly — discretization is evaluation of the flow, not approximation.

2.8 Stability

The zero-input response $e^{At}x(0)$ decays for every $x(0)$ iff every eigenvalue of A has negative real part (A is *Hurwitz*); marginal cases sit on the imaginary axis. The eigenvalue test has an algebraic counterpart that avoids computing the spectrum:

Theorem 2.8 (Lyapunov)**.** *A is Hurwitz iff for some (equivalently every) $N \succ 0$ the Lyapunov equation $A^\top M + MA = -N$ has a solution $M \succ 0$.*

Example 2.9. For A of Example 2.5 and $N = I$, solving the three scalar equations of $A^\top M + MA = -I$ gives

$$M = \begin{bmatrix} \frac{5}{4} & \frac{1}{4} \\ \frac{1}{4} & \frac{1}{4} \end{bmatrix} \succ 0 \qquad \left(\text{minors } \tfrac{5}{4} > 0,\ \det M = \tfrac{1}{4} > 0\right),$$

establishing stability without ever finding an eigenvalue — the quadratic form $x^\top Mx$ decreases along every trajectory.

BIBO (input–output) stability is implied by, but strictly weaker than, the internal stability above; the gap is explained by controllability and observability in Chapter 3.

2.9 Laplacian Dynamics and Consensus

The chapter closes with all of its tools applied at once to the network of Fig. 2.2 (right). Let W be the symmetric adjacency matrix of an undirected graph ($w_{ij} = w_{ji} > 0$ on edges) and $D = \operatorname{diag}(W\mathbf{1})$ (the degree matrix, unrelated to the feedthrough D of Eq. (2.1)); the *Laplacian* is $\mathcal{L} = D - W$. Each agent moves toward its neighbours:

$$\dot{x}_i = \sum_{j \in N(i)} w_{ij}\,(x_j - x_i) \qquad \Longleftrightarrow \qquad \dot{x} = -\mathcal{L}x. \tag{2.3}$$

In the sense of Definition 2.3, the graph of $\dot{x} = -\mathcal{L}x$ is that undirected graph with each edge read as two opposite directed edges of weight w_{ij}, and with a self-loop of weight $-[D]_{ii}$ — minus the degree — at every node i.

Example 2.10 (Four agents)**.** For the topology of Fig. 2.2 (right) with unit weights (edges $12, 23, 34, 13$),

$$\mathcal{L} = \begin{bmatrix} 2 & -1 & -1 & 0 \\ -1 & 2 & -1 & 0 \\ -1 & -1 & 3 & -1 \\ 0 & 0 & -1 & 1 \end{bmatrix},$$

with row sums zero by construction.

Three spectral facts, each an application of an earlier section:

1. $\mathcal{L}\mathbf{1} = 0$: the all-ones vector is an eigenvector with eigenvalue 0 — agreement is an equilibrium.
2. $x^\top \mathcal{L}x = \frac{1}{2}\sum_{i,j} w_{ij}(x_i - x_j)^2 \geq 0$ (the factor $\frac{1}{2}$ because each undirected edge is counted in both orders), so $\mathcal{L} \succeq 0$ by Section 2.6: all remaining eigenvalues are positive or zero.
3. The graph is connected iff $\operatorname{rank} \mathcal{L} = n-1$, iff the second smallest eigenvalue λ_2 — the *algebraic connectivity* — is positive.

Consequently, on a connected graph every solution of Eq. (2.3) converges to average agreement, $x(t) \to (\frac{1}{n}\mathbf{1}^\top x(0))\,\mathbf{1}$ (*average consensus* in the cited literature), and the disagreement decays at rate λ_2; the quadratic form $x^\top \mathcal{L}x$ itself is the Lyapunov function establishing it. For a directed graph with a rooted spanning tree consensus is still reached, but to a *weighted* agreement value (the left-Perron-vector combination of the initial states); the simple average is recovered only for balanced directed graphs (statement level; see the references).

Remark 2.11 (Dependent entries in Laplacian families)*.* The entries of a Laplacian are *dependent*: each diagonal entry is the negative sum of its row. When Part II treats matrix entries as free parameters, Laplacian families will need this dependency respected; we flag it now and return to it with the diffusively-coupled networks of Chapter 9.

Sources. [1, 4, 5, 6, 7].

Chapter 3

Controllability and Observability

The central topic of Part I. Controllability asks whether the input can steer the state anywhere; observability asks whether the output determines the state. Both reduce to rank conditions — and the chapter ends on a deliberate open question: the conclusion of the rank test can flip when a single entry of A changes value, which is a serious problem in a network whose weights are uncertain.

3.1 Controllability

Definition 3.1 (Controllability)**.** The pair (A, B) is *controllable* if for every initial state $x(0) = x_0$ and every target x_f there exist a finite time t_f and an input $u(\cdot)$ with $x(t_f) = x_f$.

Substituting the solution formula Eq. (2.2), everything hinges on which states the zero-state term can reach — a spanning question, as promised in Section 2.4. The fundamental answer is algebraic:

Theorem 3.2 (Kalman rank condition)**.** *(A, B) is controllable iff*

$$\operatorname{rank} \mathcal{C} = \operatorname{rank} \begin{bmatrix} B & AB & \cdots & A^{n-1}B \end{bmatrix} = n. \tag{3.1}$$

Powers beyond A^{n-1} add nothing, by Cayley–Hamilton; hence the truncation. When the rank falls short of n, the input still steers the state throughout a part of the state space:

Definition 3.3 (Controllable subspace)**.** The set of states reachable from the origin is the column space $\operatorname{Im} \mathcal{C}$, called the *controllable subspace* of (A, B); the pair is controllable iff $\operatorname{Im} \mathcal{C} = \mathbb{R}^n$.

Everything in Part II is a re-reading of Eq. (3.1) — and of this subspace, whose dimension and position under parameter uncertainty occupy Chapters 6 and 8.

A second characterization refines the yes/no of Theorem 3.2 into *how hard*:

Theorem 3.4 (Gramian test)**.** *(A, B) is controllable iff the* controllability Gramian

$$W(t) = \int_0^t e^{A\tau} B B^\top e^{A^\top \tau} \, d\tau$$

is nonsingular for some (equivalently every) $t > 0$. In that case the minimum-energy input reaching x_f from the origin has energy $x_f^\top W(t_f)^{-1} x_f$.

The Gramian is symmetric positive semidefinite (Section 2.6); the letter W denotes it here and in Chapter 10, and is unrelated to the weight matrix of Section 2.9. Its energy reading — *how hard* each direction is to reach, not just *whether* it is reachable — returns as the metric for input placement in Chapter 10. For Hurwitz A the infinite-horizon Gramian solves the Lyapunov equation $AW + WA^\top = -BB^\top$, tying it to Theorem 2.8.

Example 3.5 (Mass–spring–damper)**.** For Example 2.1, $\mathcal{C} = \begin{bmatrix} 0 & \frac{1}{m} \\ \frac{1}{m} & -\frac{c}{m^2} \end{bmatrix}$, $\det \mathcal{C} = -\frac{1}{m^2} \neq 0$: a force input controls both position and velocity, for every m, c, k.

Example 3.6 (The four-node network)**.** For Example 2.4 the columns of $\mathcal{C}$ are computed by walking the graph of Fig. 2.1:

$$b = e_1, \quad Ab = \begin{bmatrix} 0 \\ 2 \\ 0 \\ 0 \end{bmatrix}, \quad A^2 b = \begin{bmatrix} 0 \\ 0 \\ 2 \\ 6 \end{bmatrix}, \quad A^3 b = \begin{bmatrix} -2 \\ 0 \\ 0 \\ 2 \end{bmatrix},$$

— the input's influence spreads along walks: node 1 at step 0, node 2 at step 1 (via $1 \to 2$, weight 2), nodes 3 and 4 at step 2 (via $1 \to 2 \to 3$ and $1 \to 2 \to 4$, weights $2 \cdot 1$ and $2 \cdot 3$), and so on. Here $\det \mathcal{C} = 8 \neq 0$: controllable. The observation that each column entry is a sum of weight products is no accident; it becomes the key identity of Chapter 4.

3.2 PBH Tests

Theorem 3.7 (PBH)**.** *(A, B) is controllable iff* $\operatorname{rank}[\, A - \lambda I \;\; B \,] = n$ *for every eigenvalue λ of A; equivalently, iff no left eigenvector of A is orthogonal to every column of B.*

The modal reading: an uncontrollable system has a mode — a direction w with $w^\top A = \lambda w^\top$ — that the input cannot excite ($w^\top B = 0$). PBH localizes the failure that the Kalman test only detects.

Example 3.8 (Symmetry destroys controllability)**.** Drive the center of the twin star of Fig. 3.1:

$$A = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 0 \\ 1 & 0 & 0 \end{bmatrix}, \qquad b = e_1.$$

The vector $w = (0, 1, -1)$ satisfies $w^\top A = 0$ and $w^\top b = 0$: the *antisymmetric* mode of the two end nodes $2, 3$ (Definition 4.5) is uncontrollable from the input, which pushes both of them identically. Indeed $\mathcal{C} = [\, e_1, \; (0, 1, 1)^\top, \; 2e_1 \,]$ has rank 2. No choice of weights fixes this — the failure is caused by the twin *structure*, a theme Part II makes precise.

3.3 Observability and Duality

Definition 3.9 (Observability)**.** The pair (A, C) is *observable* if the initial state $x(0)$ is determined by the input and output histories on some finite interval.

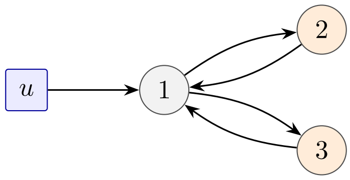


Figure 3.1: Example 3.8: the twin end nodes (shaded) form a symmetric pair; the input at the center cannot tell them apart, and the difference $x_2 - x_3$ is uncontrollable.

Theorem 3.10 (Rank test and duality)**.** (A, C) *is observable iff*

$$\operatorname{rank} \mathcal{O} \;=\; \operatorname{rank} \begin{bmatrix} C \\ CA \\ \vdots \\ CA^{n-1} \end{bmatrix} \;=\; n;$$

equivalently, iff $(A^\top, C^\top)$ *is controllable.*

Duality is exact and complete: every controllability result of this book has an observability dual obtained by transposing — on the graph, by reversing every edge and exchanging the roles of inputs and outputs. We state it once here and use it at the end of the book (Chapter 10).

Example 3.11 (The four-node network, observed)**.** For Example 2.4 with $y = x_4$,

$$\mathcal{O} = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 3 & 1 & 0 \\ 6 & 1 & 0 & 0 \\ 2 & 0 & -6 & 0 \end{bmatrix}, \qquad \det \mathcal{O} = -106 \neq 0 :$$

observable. On the graph: every state node has a directed path *to* the output node (4 directly, $2 \to 4$, $3 \to 4$, $1 \to 2 \to 4$) — the mirror image of the walk picture of Example 3.6, with all arrows reversed.

A closing remark, stated without development: the Kalman decomposition sorts the state space by controllability and observability, and the transfer matrix depends only on the part that is both — which is exactly why internal stability is stronger than BIBO (Section 2.8).

3.4 Fragility of Numerical Tests

Every test in this chapter used the *numbers* in (A, B). The closing example shows how fragile that reliance is.

Example 3.12 (Same graph, opposite conclusions)**.** Two agents, coupled both ways, each with a self-loop, one input feeding both (Fig. 3.2):

$$A_\circ = \begin{bmatrix} 1 & 2 \\ 1 & 2 \end{bmatrix}, \quad A_\bullet = \begin{bmatrix} 1 & 2 \\ 3 & 2 \end{bmatrix}, \qquad b = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

For $b = (1,1)^\top$ one computes $\det[\,b \;\; Ab\,] = a_{21} + a_{22} - a_{11} - a_{12}$, so

$$\begin{aligned} A_\circ &: \ \det \mathcal{C} = 1 + 2 - 1 - 2 = 0 && \text{(uncontrollable)},\\ A_\bullet &: \ \det \mathcal{C} = 3 + 2 - 1 - 2 = 2 && \text{(controllable)}. \end{aligned}$$

The two systems have the *same* graph — same nodes, same edges — and differ in a single edge weight, 1 versus 3.

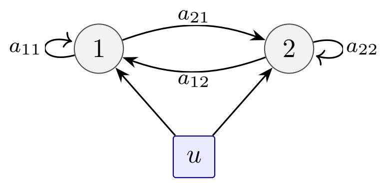


Figure 3.2: Example 3.12: one graph, two realizations (Definition 4.1), opposite controllability conclusions — the graph alone cannot decide, but it can say *when* the numbers matter, which is the subject of Part II.

In a laboratory plant this is a minor issue: the parameters are measured, and non-generic coincidences like $a_{21} + a_{22} = a_{11} + a_{12}$ do not survive a datasheet. In a network the situation is reversed: the weights are exactly what we do not know, so a single rank test computed from nominal values guarantees nothing. What we do know is the pattern — the graph of Definition 2.3 with its numbers erased (Definition 4.3). The question that opens Part II is therefore: *what can the pattern alone guarantee?* Example 3.8 already hints that the answer is not “nothing”: some failures are determined by the graph alone.

Sources. [1]; the closing discussion follows [8, 2].

Part II

Controllability of Structured Networks: Analysis

Chapter 4

Structured Networks and Pattern Graphs

Part II opens by changing what we assume: the zero/non-zero pattern of the system matrix is reliable, its numbers are not. This chapter builds the vocabulary for that setting — the family of a pattern, the pattern graph, and the identity showing that the controllability matrix was a graph object all along: powers of the adjacency matrix enumerate walks, and multiplying by B keeps exactly the walks starting at the inputs.

4.1 Patterns, Not Numbers

Definition 4.1 (Family set)**.** A *pattern matrix* A is a matrix each of whose entries is either structurally zero or a free non-zero parameter. Its *family set* $\mathcal{Q}(A)$ is the set of all matrices A_F with $[A_F]_{ij} \neq 0$ exactly where $[A]_{ij} \neq 0$; a member $A_F \in \mathcal{Q}(A)$ is called a *realization* of the pattern. The free non-zero entries are the *network parameters.*

Example 4.2 (The flip system, as a family)**.** The two matrices of Example 3.12 are two members of one family,

$$A_F = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}, \qquad a_{ij} \neq 0 \text{ arbitrary,}$$

and the question left open there — what survives when only the pattern is known? — becomes: which properties hold *across* the family $\mathcal{Q}(A)$? Likewise the four-node network of Example 2.4 generates the family with free parameters $a_{21}, a_{32}, a_{42}, a_{13}, a_{43}$; the numbers used in Chapter 3 were one member among infinitely many.

At a single realization, everything of Part I applies verbatim — the structured setting strictly contains the classical one. B is held fixed and binary, and — the standing convention of Part II — unless said otherwise each input attaches to a *single* driver node; the flip family, whose one input feeds both nodes, is the flagged exception. For single-attachment columns the non-zero magnitudes never affect controllability (rescaling a column of B merely rescales columns of $\mathcal{C}$), so only *where* inputs attach matters; a multi-attachment column, by contrast, mixes weights whose ratios can decide the outcome.

4.2 Pattern Graph

Every realization $A_F \in \mathcal{Q}(A)$ has the graph of Definition 2.3 with its own weights — but all realizations share the same *unweighted* picture.

Definition 4.3 (Pattern graph)**.** The *pattern graph* of a family is the graph of Definition 2.3 with the edge weights erased: one graph, all realizations.

4.3 Graph-Theoretic Background

Part II reasons on the pattern graph, so we collect in one place the graph vocabulary the analysis chapters draw on. The definitions are elementary and stated once here; the running four-node pattern (Fig. 4.1) illustrates them, and we spend an example only where a notion is genuinely easy to misread — in/out orientation, disjointness, and the layered structure of acyclic patterns. The specialized *criteria* built from this vocabulary stay in the chapters that need them.

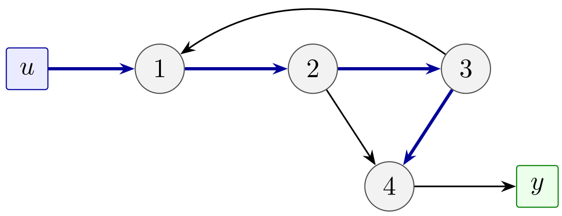


Figure 4.1: The pattern graph of Example 2.4: same picture as Fig. 2.1, numbers erased. It is the running example for this section. The stem $u \to 1 \to 2 \to 3 \to 4$ (bold) covers all state nodes.

4.3.1 Directed Graphs, Undirected Graphs, and Neighbours

The directed graph of a linear system was fixed once in Definition 2.3: a node for every state, input, and output, and a directed edge (j, i) — drawn $j \to i$ — whenever $[A]_{ij} \neq 0$, so that an edge records that j influences i. Two structural specializations recur.

Definition 4.4 (Undirected graph)**.** A pattern is *undirected* (or *symmetric*) if its edge relation is symmetric: $(j, i) \in \mathcal{E}$ iff $(i, j) \in \mathcal{E}$. An *undirected edge* between i and j, drawn $i - j$, is then shorthand for the pair of opposite directed edges $i \to j$ and $j \to i$; algebraically it is the symmetric pair $[A]_{ij}, [A]_{ji} \neq 0$. A *self-loop* at i — an edge $i \to i$ — corresponds to a non-zero diagonal entry $[A]_{ii}$.

Undirected patterns model bidirectional coupling: the four-robot network and the Laplacian consensus dynamics of Section 2.9 are the book's standing example, and self-loops (free diagonal entries) return as the hypothesis of the zero-forcing test (Theorem 7.14).

Definition 4.5 (In- and out-neighbours)**.** For a node v, its *out-neighbours* $N^{\text{out}}(v) = \{w : (v, w) \in \mathcal{E}\}$ are the nodes v points to, and its *in-neighbours* $N^{\text{in}}(v) = \{w : (w, v) \in \mathcal{E}\}$ the nodes that point to v. For a set $S \subseteq \mathcal{V}$ the operators act setwise, $N^{\text{out}}(S) = \bigcup_{v \in S} N^{\text{out}}(v)$ and $N^{\text{in}}(S) = \bigcup_{v \in S} N^{\text{in}}(v)$. On an undirected pattern the two coincide, written $N(\cdot)$, and a node with exactly one neighbour, $|N(v)| = 1$, is an *end node.*

Reading edges the wrong way round is a recurring source of error — under the transpose convention (Definition 2.3) an out-neighbour of v is a node v *feeds*, carried by v's *column* of A, not its row — so one explicit count is worked out here.

Example 4.6 (In- and out-neighbours on the four-node network)**.** The state edges of Fig. 4.1 are $1 \to 2$, $2 \to 3$, $2 \to 4$, $3 \to 1$, $3 \to 4$. Take the set $S = \{3, 4\}$. Its in-neighbours are the nodes with an edge *into* S: node 2 (via $2 \to 3$ and $2 \to 4$) and node 3 (via $3 \to 4$), so

$$N^{\text{in}}(\{3, 4\}) = \{2, 3\}.$$

Its out-neighbours are the nodes S *feeds*: node 3 reaches 1 and 4, while node 4 reaches only the output y; among state nodes,

$$N^{\text{out}}(\{3, 4\}) = \{1, 4\}.$$

The two sets differ — $\{2, 3\}$ against $\{1, 4\}$ — and node 3 belongs to *both*, receiving from 2 and sending to 4. The whole-graph criteria of Chapter 7 are phrased in exactly this in/out vocabulary ("an edge into S", "an out-neighbour inside S"), so the orientation must be read carefully.

4.3.2 Walks, Paths, Cycles, and Distance

Definition 4.7 (Walk, path, cycle)**.** A *walk* of length k is a sequence of k consecutive directed edges; its nodes may repeat. A *path* is a walk that visits no node twice, and a *cycle* is a path whose final edge returns to its start node — the degenerate case being a self-loop $i \to i$, a cycle of length one.

The distinction matters: the identity of Section 4.4 counts *walks*, which may loop around a cycle, whereas the covering objects of every controllability criterion are *paths*. The walk $u \to 1 \to 2 \to 3 \to 1$ of Example 4.15 revisits node 1 and so is no path; walks and paths differ exactly at cycles.

Definition 4.8 (Distance and shortest path)**.** The *distance* from v to w is the length of a shortest directed walk from v to w (infinite if none exists); a walk attaining it repeats no node and is therefore a *shortest path*. The distance from the input nodes to a state node is its *input-distance*.

Distance underwrites two later bounds — the shortest-stem lower bound on $\dim \mathcal{C}_\Gamma$ (Definition 6.1 and Section 6.4) and the shortest-path criterion for SSC nodes (Theorem 8.15).

4.3.3 Stems, Branches, and Reachability

The input and output nodes single out two dual families of paths.

Definition 4.9 (Stem)**.** A *stem* is a path originating at an input node.

Definition 4.10 (Branch)**.** A *branch* is a path terminating at an output node — the observability dual of a stem, obtained by reversing every edge and exchanging the roles of inputs and outputs.

Stems carry the controllability results of Parts II–III; branches carry their sensing counterpart and surface when everything is dualized in Section 10.3. On the four-node network the stem $u \to 1 \to 2 \to 3 \to 4$ runs from the input across every state node, and the path $1 \to 2 \to 3 \to 4 \to y$ is the matching branch into the single output.

Definition 4.11 (Induced subgraph)**.** The subgraph *induced* by a node set $U \subseteq \mathcal{V}$ keeps the nodes of U and every edge of the pattern with both endpoints in U.

Before specializing to the inputs, the general notion: which nodes can be reached from where.

Definition 4.12 (Reachable set)**.** For any subset $S \subseteq \mathcal{V}$ of nodes, the *reachable set* of S is

$$R(S) = \{\, w \in \mathcal{V} : \text{there exist } v \in S \text{ and a directed path } v \to \cdots \to w \,\} \;\supseteq\; S,$$

the inclusion holding because the empty path leaves each $v \in S$ where it is. A node w is *reachable from* S if $w \in R(S)$; walks may be used in place of paths without changing $R(S)$, since every walk from v to w contains a path from v to w.

Reachability here is a property of the *graph* — which nodes can be reached along the edges — not of the state space; the reachable *states* of Definition 3.3 are the vector-space notion for which graph reachability is necessary but not sufficient.

Definition 4.13 (Input-connectedness)**.** A state node is *reachable* if some stem reaches it — equivalently, if it lies in $R(\mathcal{V}^I)$, the reachable set of the input nodes. A pattern in which every state node is reachable, i.e.

$$\mathcal{V}^S \subseteq R(\mathcal{V}^I),$$

is called *input-connected*, and its *reachable part* is the subgraph induced by $R(\mathcal{V}^I)$ — the reachable state nodes together with the input nodes.

Remark 4.14 (Terminology). Some references, and earlier drafts of these notes, call an input-connected pattern *accessible* and a reachable node an *accessible* node; the two vocabularies ask the same picture-level question — is there a path from an input to this node? — and *input-connected* is the term used throughout this book.

A state node outside $R(\mathcal{V}^I)$ contributes nothing to controllability — its row of $\mathcal{C}_F$ vanishes identically (Proposition 4.24) — so several criteria are stated on the reachable part; dropping the restriction breaks them, as Example 7.4 shows in full.

Example 4.15 (Stems of the four-node pattern)**.** The pattern graph of Example 2.4 (Fig. 4.1) has the stems

$$u \to 1, \quad u \to 1 \to 2, \quad u \to 1 \to 2 \to 3, \quad u \to 1 \to 2 \to 4, \quad u \to 1 \to 2 \to 3 \to 4,$$

the last one covering every state node — a fact that Chapter 6 will turn into a sufficient condition for *structural* controllability. All four state nodes are reachable, $R(\{u\}) = \{u, 1, 2, 3, 4\}$, so the pattern is input-connected. Note the walk $u \to 1 \to 2 \to 3 \to 1$ is not a stem (node 1 repeats); walks and paths differ exactly at cycles.

4.3.4 Disjoint Paths and Stems

Definition 4.16 (Disjointness)**.** Write $V(p)$ and $E(p)$ for the node set and the edge set of a path p. Two paths — in particular two stems — are *node-disjoint*, or simply *disjoint*, if they share no node, $V(p) \cap V(q) = \emptyset$. A set of paths is disjoint if its members are pairwise disjoint.

Remark 4.17 (Node- versus edge-disjoint). A weaker notion exists: two paths are *edge-disjoint* if they share no edge, $E(p) \cap E(q) = \emptyset$, nodes allowed. Node-disjoint implies edge-disjoint, never the converse: the stems $u \to 1 \to 2$ and $u \to 3 \to 4$ use entirely different edges yet share the input u — edge-disjoint, not node-disjoint. Throughout this book, and in most of the structured-systems literature, an unqualified *disjoint* always means *node*-disjoint; it is the node version that carries the control meaning developed below: stems that share no state node.

Disjointness is where single-input patterns meet a hard limit. Two stems originating at the *same* input already share that input node, so they are never disjoint; a family of m pairwise disjoint stems therefore requires m distinct input nodes.

Example 4.18 (Two stems that are not disjoint)**.** Consider a single input driving node 1, from which two paths run along $1 \to 2 \to 4$ and $1 \to 3 \to 4$ and rejoin at node 4 — the diamond of Fig. 4.2. Read as stems, the two paths $u \to 1 \to 2 \to 4$ and $u \to 1 \to 3 \to 4$ share the input, node 1, *and* node 4: disjoint in neither. With one input a disjoint set can therefore hold at most one stem, and any further nodes must be picked up by disjoint *cycles*. This is exactly why the generic-dimension count of Theorem 6.4 is stated for disjoint stems *and cycles*, and why the diamond's lone input caps its cover at three of four nodes (Example 6.5).

4.3.5 Connectivity

Definition 4.19 (Connectivity notions)**.** A directed pattern is *strongly connected* if it has a directed walk from every node to every other node. Its *underlying undirected graph* — edges with direction forgotten — is *connected* if it has an undirected path between every pair of nodes, and is a *tree* if it is connected and acyclic. A *rooted spanning tree* of a directed pattern is a subgraph that touches every node and in which one node, the *root*, reaches all others by directed paths (equivalently, an acyclic spanning subgraph in which the root has no in-edge and every other node exactly one).

These notions govern the network dynamics of Section 2.9: an undirected consensus network reaches average agreement iff it is connected, and a directed one reaches weighted agreement iff it has a rooted spanning tree.

4.3.6 Acyclic and Hierarchical Structure

Patterns without cycles support the sharpest results in the book: on them every walk is a path, and no length can be inflated by looping.

Definition 4.20 (Directed acyclic graph)**.** A pattern is a *directed acyclic graph* (DAG) if it has no directed cycle of any length $k \geq 1$. In particular a self-loop (Definition 4.4) is a cycle of length one, so a DAG carries no self-loop and its diagonal is structurally zero: $[A]_{ii} = 0$ for every i. A *source* is a node with no in-edges; a *sink* is a node with no out-edges. (The papers this book draws on call a sink a *terminal node*; one of them reserves that name instead for a state node with a single neighbour, an end node in the sense of Definition 4.5, so *sink* is the name used here.)

On a DAG the state matrix is nilpotent (Remark 10.1) and the nodes fall into layers by input-distance; when those layers are clean the pattern is given a stronger name.

Definition 4.21 (Distance layers and hierarchical DAG)**.** The *distance layers* of a pattern are the sets ℓ_k of state nodes at input-distance exactly k (Definition 4.8); ℓ_k is the notation used for them in Chapter 8 and the appendices as well. A DAG is a *hierarchical DAG* (HDAG) if the sets of nodes reachable from the inputs in exactly k steps are disjoint across k — so that every walk into a given node has one common length and the state nodes are partitioned into their distance layers $\ell_1, \ell_2, \ldots$.

Example 4.22 (The diamond is hierarchical; the four-node network is not)**.** The diamond of Fig. 4.2 — input at node 1, then $1 \to 2$, $1 \to 3$, $2 \to 4$, $3 \to 4$ — lays its four nodes into clean distance layers

$$\ell_1 = \{1\}, \qquad \ell_2 = \{2, 3\}, \qquad \ell_3 = \{4\},$$

because *every* walk from the input to node 4 has length three: it is an HDAG. The four-node network of Fig. 4.1 is not, even setting its cycle aside — node 4 is reached from the input both in two steps ($u{\to}1{\to}2{\to}4$) and in three ($u{\to}1{\to}2{\to}3{\to}4$), so no single distance layer can hold it. This clean layering is what makes the worst-case dimension $\dim \mathcal{C}_\Gamma$ exactly computable on HDAGs (Section 6.4), while no closed formula is available in general.

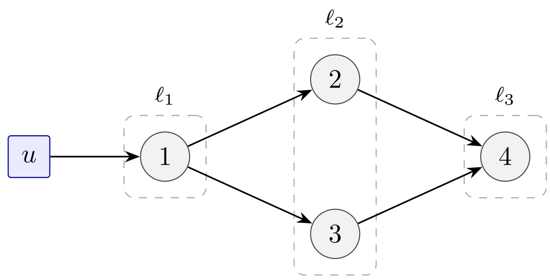


Figure 4.2: The diamond of Example 4.22 arranged into distance layers $\ell_1 = \{1\}$, $\ell_2 = \{2, 3\}$, $\ell_3 = \{4\}$: every walk from the input to a node has a single length, so the pattern is a hierarchical DAG. The four-node network of Fig. 4.1, whose node 4 sits at two different distances, is not.

The vocabulary above is the shared background; further terms are introduced with the criterion that needs them — *dilation* and *matchings* for structural controllability (Definitions 7.1 and 7.5), *dedicated* and *sharing* nodes and *zero forcing* for strong structural controllability (Definitions 7.7 and 7.13), and the *matched nodes* of a hierarchical DAG and the *control paths* that localize those tests to individual nodes (Theorems 8.9 and 8.15). Table 4.1 collects the recurring terms with pointers.

4.4 Graph-Theoretic Meaning of the Controllability Matrix

The identity underlying all of Part II is assembled in two steps — first a fact of pure graph theory, then one matrix multiplication — after which its entries fall into two kinds.

4.4.1 Powers of Adjacency Matrix Count Walks

Lemma 4.23 (Walk enumeration)**.** *Interpreting $A_F \in \mathcal{Q}(A)$ as a weighted adjacency matrix, $[A_F^k]_{lw}$ equals the sum, over all k-step walks from node w to node l on the pattern graph, of the products of the edge weights along the walk; for a 0/1 matrix it counts the walks.*

Proof. Induction on k. For $k = 1$ the statement is the definition of the graph. For the step, $[A_F^{k+1}]_{lw} = \sum_j [A_F]_{lj}[A_F^k]_{jw}$: every $(k{+}1)$-step walk from w to l is a k-step walk from w to some j followed by an edge $j \to l$, and the weight products multiply accordingly. $\square$

Table 4.1: Graph vocabulary of Part II: the shared background of this section (top) and the criterion-specific terms introduced later (bottom).

Term	Meaning (in brief)	Defined in
directed graph	edge $j \to i$ when $[A]_{ij} \neq 0$	Definition 2.3
undirected graph	symmetric pair of edges	Definition 4.4
in/out-neighbour	nodes into / out of node	Definition 4.5
end node	exactly one neighbour	Definition 4.5
walk	edge sequence, nodes repeatable	Definition 4.7
path	walk without repeated node	Definition 4.7
cycle	path returning to start	Definition 4.7
distance	length of shortest walk	Definition 4.8
stem	path starting at input	Definition 4.9
branch	path ending at output	Definition 4.10
induced subgraph	nodes kept with internal edges	Definition 4.11
reachable set $R(S)$	nodes reached by paths from S	Definition 4.12
input-connected	$\mathcal{V}^S \subseteq R(\mathcal{V}^I)$	Definition 4.13
disjoint paths	sharing no node	Definition 4.16
strongly connected	every node reaches every	Definition 4.19
rooted spanning tree	one root reaches all	Definition 4.19
DAG	no directed cycle	Definition 4.20
hierarchical DAG	clean distance layers	Definition 4.21
dilation	set with too few in-neighbours	Definition 7.1
matching	edges sharing no endpoint	Definition 7.5
dedicated node	one edge into set	Definition 7.7
zero forcing set	drivers force whole graph	Definition 7.13
matched node (layer)	on an HDAG, in every maximum disjoint-stem set	Theorem 8.9
SSC node	every subset containing it is controllable	Definition 8.14
control path	shortest-path graph, input to sink	Theorem 8.15

4.4.2 Multiplying by B: Walks from Inputs

Proposition 4.24 (Input-walk enumeration)**.** *Since the columns of B select the driver nodes, the entry $(l, k{+}1)$ of each per-input block of $\mathcal{C}_F = [B, A_F B, \ldots, A_F^{n-1} B]$ equals the sum of the weight products of all k-step walks from the driver node selected by that input column to node l (the leading input edge carries weight 1, so the count starts at the driver node): the controllability matrix lists, length by length, every walk by which the inputs reach the states.*

The enumerated walks may revisit nodes — Example 4.25 below contains a walk that goes once around a cycle — and the repeat-free ones among them are exactly the stems of Definition 4.9.

Example 4.25 (The four-node family, symbolically)**.** For the four-node family generated by Example 2.4 (introduced at the end of Example 4.2),

$$b = e_1, \quad A_F b = a_{21} e_2, \quad A_F^2 b = a_{32}a_{21}\, e_3 + a_{42}a_{21}\, e_4, \quad A_F^3 b = a_{13}a_{32}a_{21}\, e_1 + a_{43}a_{32}a_{21}\, e_4.$$

Each term is a walk read off Fig. 4.1, counted from the driver node 1 (the weight-1 attachment $u \to 1$ is suppressed): a_{21} is the one-step walk $1 \to 2$; the two terms of $A_F^2 b$ are $1 \to 2 \to 3$ and $1 \to 2 \to 4$; the e_1-term of $A_F^3 b$ is the walk once around the cycle $1 \to 2 \to 3 \to 1$, and its e_4-term the stem $1 \to 2 \to 3 \to 4$. The numerical columns of Example 3.6 are these expressions evaluated at one realization. Every entry here is a *single* product of weights — the case named in Section 4.4.3 below, which Chapter 5 makes decisive. One computation we record for later:

expanding $\det \mathcal{C}_F$ along these columns, the only non-vanishing choice picks e_1 from the first, $a_{21}e_2$ from the second, the e_3-term from the third, and the e_4-term from the fourth, so

$$\det \mathcal{C}_F \;=\; a_{21}^3\, a_{32}^2\, a_{43}$$

— a single monomial in the network parameters.

4.4.3 Single-Terms and Multi-Terms

By Proposition 4.24 every entry of $\mathcal{C}_F$ is a polynomial in the network parameters, one term per enumerated walk. Two names separate its two levels [9, 10]: the *weight product* (WP) of a walk is the product of the weights of its edges, and the entry $[\mathcal{C}_F]_{i,k+1}$ is the *sum of weight products* (SWP) of all k-step walks from the driver node to node i, steps counted from the driver as in Proposition 4.24. What distinguishes one entry from another is how many walks it sums.

Definition 4.26 (Single-term and multi-term)**.** A non-zero entry of $\mathcal{C}_F$ is a *single-term* if it consists of a single weight product — one monomial, possibly a single edge weight, or the empty product 1 that the driver node carries in the first column — and a *multi-term* if it is a sum of two or more distinct weight products. A weight product carried by more than one walk counts as one term, whatever integer coefficient it acquires. An entry that enumerates no walk is zero, and is neither.

On an acyclic pattern the classification is readable on the graph, with no polynomial to compute; the statement is adapted from [10].

Proposition 4.27 (Composition of an entry)**.** *Let the pattern graph be a DAG (Definition 4.20) with a single driver node. Then the entry $[\mathcal{C}_F]_{i,k+1}$ is zero if no stem reaches node i in k steps from the driver, a single-term if exactly one stem does, and a multi-term if two or more do.*

Proof. On a DAG every walk is a path — a repeated node would close a cycle — so the walks enumerated by Proposition 4.24 are exactly the stems of Definition 4.9 reaching node i, with their input edge dropped. Two distinct paths with the same endpoints use distinct edge sets and so contribute distinct monomials, each entering with coefficient $+1$; none can merge or cancel, and the number of terms is the number of stems. □

Remark 4.28 (Walks against terms)*.* With cycles present the correspondence is walk-based and looser. The $(1,4)$ entry $a_{13}a_{21}a_{32}$ of Example 4.25 is a single-term carried by one walk, once around the cycle $1\to2\to3\to1$, though no stem returns to node 1 at all; and two distinct walks of equal length may carry the same WP, combining into one monomial with an integer coefficient. What holds in general is that a single-term never vanishes: an entry becomes zero only when distinct monomials cancel at a particular realization, and that takes a multi-term.

The patterns already in hand carry the count from one term to two, and one further pattern takes it to three. For the chain $u\to1\to2\to3$ of Example 5.7,

$$\mathcal{C}_F = \begin{bmatrix} 1 & 0 & 0 \\ 0 & a_{21} & 0 \\ 0 & 0 & a_{21}a_{32} \end{bmatrix},$$

every non-zero entry is a single-term: each node is reached by exactly one stem, at one length. Assembling the columns of Example 4.25 for the four-node pattern of Fig. 4.1,

$$\mathcal{C}_F = \begin{bmatrix} 1 & 0 & 0 & a_{13}a_{21}a_{32} \\ 0 & a_{21} & 0 & 0 \\ 0 & 0 & a_{21}a_{32} & 0 \\ 0 & 0 & a_{21}a_{42} & a_{21}a_{32}a_{43} \end{bmatrix},$$

and every entry is a single-term again, cycle notwithstanding: node 4 is reached in two steps along 1→2→4 and in three along 1→2→3→4, but stems of different lengths land in different columns and never share an entry. The diamond of Example 6.2 is the first pattern here in which two stems do share one entry:

$$\mathcal{C}_F = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & a_{21} & 0 & 0 \\ 0 & a_{31} & 0 & 0 \\ 0 & 0 & a_{21}a_{42} + a_{31}a_{43} & 0 \end{bmatrix}.$$

Its $(4, 3)$ entry is a multi-term of two WPs, one from u→1→2→4 and one from u→1→3→4, the two stems reaching node 4 in the same number of steps. It is the entry whose cancellation leaves the diamond with the worst-case dimension $\dim \mathcal{C}_\Gamma = 2$ computed in Example 6.2.

Example 4.29 (Three stems into one node)**.** The pattern of Fig. 4.3 — input at node 1, then 1→2, 1→3, 1→4, and 2→5, 3→5, 4→5 — sends three stems of equal length into node 5:

$$\mathcal{C}_F = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & a_{21} & 0 & 0 & 0 \\ 0 & a_{31} & 0 & 0 & 0 \\ 0 & a_{41} & 0 & 0 & 0 \\ 0 & 0 & a_{21}a_{52} + a_{31}a_{53} + a_{41}a_{54} & 0 & 0 \end{bmatrix}.$$

The $(5, 3)$ entry is a multi-term of three WPs, one for each of u→1→2→5, u→1→3→5 and u→1→4→5; every other non-zero entry is a single-term. Generically the rank is three, and that entry is the only place where it can drop: at $a_{21}a_{52} = a_{31}a_{53} = 1$ and $a_{41}a_{54} = -2$ the sum is zero, the third column vanishes, and the rank is two.

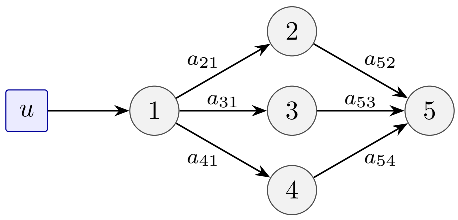


Figure 4.3: The fan of Example 4.29: three stems of equal length into node 5 make the $(5, 3)$ entry of $\mathcal{C}_F$ a multi-term of three WPs, while every other non-zero entry is a single-term.

Where the multi-terms sit is itself a property of the picture. Two node classes name it, and the proposition after them locates every multi-term on a hierarchical DAG; both are adapted from [10].

Definition 4.30 (Integrator and intermediator)**.** A state node with two or more in-neighbours among the state nodes is an *integrator* (a graph classification, unrelated to the dynamic element $1/s$). A state node with exactly one such in-neighbour is an *intermediator* if every stem from the input node to it passes through at least one integrator.

Proposition 4.31 (Location of the multi-terms)**.** *Let the pattern graph be an HDAG (Definition 4.21) that is input-connected (Definition 4.13) and has a single driver node; the driver is then its only source among the state nodes. Row i of $\mathcal{C}_F$ contains a multi-term if and only if node i is an integrator or an intermediator.*

Proof. On an HDAG every stem reaches i in the same number of steps, so row i carries at most one non-zero entry and Proposition 4.27 reads it as a count of stems. Two or more in-neighbours give two or more stems. With exactly one in-neighbour j, every stem to i extends a stem to j, so the two counts agree; iterating back to the driver, whose count is one, the count exceeds one exactly when an integrator lies on the way. □

The diamond and the fan are both hierarchical DAGs driven at their only source, so the proposition applies to both: node 4 of the diamond and node 5 of the fan have two and three in-neighbours, are integrators, and carry the multi-terms computed above. An intermediator, by contrast, is fed by a single edge and lies downstream of an integrator, whose multi-term it inherits. Append one edge 4→5 of weight a_{54} to the diamond and node 5 becomes one: its only non-zero entry is

$$[\mathcal{C}_F]_{5,4} = a_{54}(a_{21}a_{42} + a_{31}a_{43}),$$

the multi-term of node 4 carried one step further. A node need not collect several edges of its own for its entry to become zero.

The two classes divide the work of the chapters that follow. A single-term is a product of non-zero network parameters and cannot vanish; only a multi-term can. Chapter 5 reads $\det \mathcal{C}_F$ through this division, and Chapter 6 measures how much rank a vanishing multi-term can cost. Which multi-terms can never vanish — and with it the exact worst-case dimension on hierarchical DAGs — is settled in [10] and stays outside this book.

4.5 Remark on Outputs and Targets

If only some states matter, a selection matrix C picks them out, and the natural condition becomes $\operatorname{rank}(C\,\mathcal{C}) = q$, the number of selected states — *target controllability*, the output-level counterpart of everything that follows. We note it here and leave its theory to the literature [11, 12].

4.6 Questions of Part II

With the vocabulary in place, the questions of Part II: when does the rank of $\mathcal{C}_F$ survive the unknown weights (Chapter 5); how much survives when full rank fails (Chapter 6); the same questions asked on the graph (Chapter 7); which individual nodes survive (Chapter 8); and finally, where inputs should be placed (Chapters 9 and 10).

Sources. [13, 3, 14, 9, 10]; walk enumeration is classical algebraic graph theory [5].

Chapter 5

Structural and Strong Structural Controllability: Determinant View

The central definitions of the book, and one object that separates them: the determinant of the controllability matrix, read as a polynomial in the network parameters. It vanishes identically (never controllable), vanishes only on a measure-zero set (structurally controllable), or never vanishes (strongly structurally controllable) — and each of the three cases fits on a graph with two or three nodes.

5.1 Almost All versus All

Definition 5.1 (Structural controllability). The family $\mathcal{Q}(A)$ with input matrix B is *structurally controllable* (SC) if the pair (A_F, B) is controllable for almost all realizations $A_F \in \mathcal{Q}(A)$.

Definition 5.2 (Strong structural controllability). The family $\mathcal{Q}(A)$ with input matrix B is *strongly structurally controllable* (SSC) if the pair (A_F, B) is controllable for *all* realizations $A_F \in \mathcal{Q}(A)$.

A single realization recovers the classical notion, and SSC $\Rightarrow$ SC. The open question of Section 3.4 can now be answered in one line.

Example 5.3 (The flip family, resolved). For the family of Example 4.2 with $b = (1, 1)^\top$,

$$\det \mathcal{C}_F = a_{21} + a_{22} - a_{11} - a_{12},$$

a polynomial that is not identically zero but does vanish — on the hyperplane $a_{21} + a_{22} = a_{11} + a_{12}$ in parameter space. The family is SC (almost every member controllable) but not SSC (the member $A_\circ$ of Example 3.12 sits exactly on the hyperplane). What looked like a numerical special case in Chapter 3 is a *structural* fact about where, in parameter space, the pattern can fail.

5.2 Determinant as Parameter Polynomial

Proposition 5.4 (Determinant trichotomy). *Let $m = 1$, so that $\mathcal{C}_F$ is square and $\det \mathcal{C}_F$ is a polynomial in the network parameters. Exactly one of the following holds:*

(i) $\det \mathcal{C}_F \equiv 0$*: no realization is controllable;*

(ii) $\det \mathcal{C}_F \not\equiv 0$ *but vanishing for some realization: the family is SC, and the uncontrollable realizations form a non-empty measure-zero set;*

(iii) $\det \mathcal{C}_F \neq 0$ *for every realization: the family is SSC.*

Case (ii) contains the genericity principle — a non-trivial polynomial vanishes only on a measure-zero set, so *one* controllable realization establishes SC. The three cases are exhibited by three minimal patterns (Fig. 5.1):

Example 5.5 (Never: the twin star)**.** For the star of Example 3.8 as a family (free weights $a_{12}, a_{13}, a_{21}, a_{31}$, input at the center),

$$\mathcal{C}_F = [\, e_1,\ \ a_{21}e_2 + a_{31}e_3,\ \ (a_{12}a_{21} + a_{13}a_{31})\, e_1 \,] :$$

the first and third columns are parallel for *every* realization, so $\det \mathcal{C}_F \equiv 0$. The symmetry failure of Example 3.8 was never about the numbers: no realization can separate two end nodes fed through a single driver node.

Example 5.6 (Almost all: the flip family)**.** Example 5.3: $\det \mathcal{C}_F \not\equiv 0$ with a non-empty zero set — SC, not SSC.

Example 5.7 (Always: the chain)**.** For the chain $u \to 1 \to 2 \to 3$,

$$\mathcal{C}_F = [\, e_1,\ \ a_{21}e_2,\ \ a_{32}a_{21}e_3 \,], \qquad \det \mathcal{C}_F = a_{21}^2\, a_{32},$$

a single product of non-zero parameters: it cannot vanish. The chain is SSC — controllable at every realization.

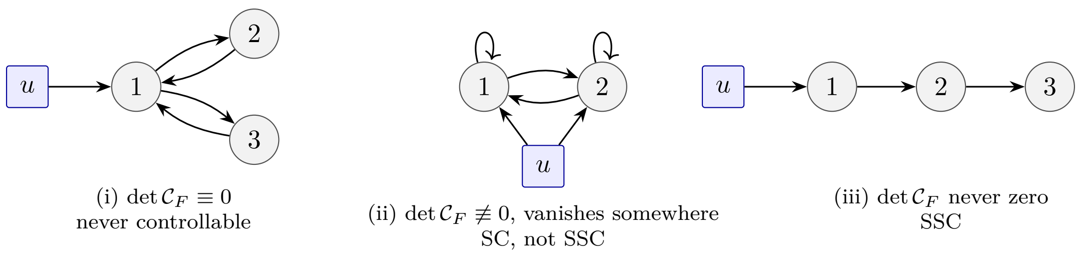


Figure 5.1: The determinant trichotomy on three minimal patterns: Examples 5.5 to 5.7. Each of the three outcomes is a property of the picture, established by one polynomial.

5.3 Reading Determinants on Graphs

Why did the three examples behave so differently? By Proposition 4.24 every entry of $\mathcal{C}_F$ is the SWP of the walks it enumerates, so every non-zero entry is a single-term or a multi-term (Definition 4.26): the former can never vanish, so an entry that becomes zero for some realization is necessarily a multi-term.

The determinant inherits this structure: its expansion terms are signed products of walk weights, one walk per node, and the surviving monomials correspond to *disjoint sets of stems and cycles* on the pattern graph. The three examples again: the chain's determinant is a single monomial — there is exactly one way to assign walks, the full stem, and nothing can cancel it (SSC); the flip family — the exception flagged in Section 4.1, whose one input feeds both nodes, so that the disjoint-stem reading does not apply — instead splits into two expansion terms of equal length that cancel for some realization (SC only); the star admits no such assignment of walks at all — one stem cannot cover two twin end nodes — so every expansion term vanishes identically (never controllable). Making this reading precise, and quantitative, is the subject of the next two chapters: the count of nodes that can be covered measures rank itself (Chapter 6), and the whole-graph criteria of Chapter 7 are this observation turned into necessary and sufficient conditions.

5.4 Multiple Inputs

For $m > 1$ the controllability matrix is $n \times nm$ and the determinant becomes a family of maximal minors; the trichotomy survives with one subtlety worth a remark: strong structural controllability requires that *for every* realization *some* minor is non-zero — the surviving minor may change with the parameters. The single-input case remains the teaching case.

5.5 Grid: Algebra Row

The book's running summary — SC/SSC against algebra/graph — begins with its algebra row:

	SC (almost all)	**SSC** (all)
algebra	$\det \mathcal{C}_F \not\equiv 0$	$\det \mathcal{C}_F$ never zero
graph	Chapters 6 and 7	Chapters 6 and 7

Sources. [13, 15, 8, 16, 3, 14].

Chapter 6

Two Controllable Subspaces

When full controllability fails, the right question is not yes or no but how much. As the parameters vary, the rank of the controllability matrix moves between a generic maximum and a worst-case minimum, defining two subspaces — and full dimension of each is exactly equivalent to SC and SSC. One pattern, the diamond, exhibits all of this: a generic dimension of three, a worst case of two, and a twin pair of nodes never independently controllable.

6.1 Rank Between Two Bounds

Definition 6.1 (SCS and SSCS)**.** $\dim \mathcal{C}_\Lambda$ is the maximum (generic) rank of $\mathcal{C}_F$ over the family, and the *structurally controllable subspace* $\mathcal{C}_\Lambda$ denotes the controllable subspace (Definition 3.3) of any realization attaining it; $\dim \mathcal{C}_\Gamma$ is the minimum (worst-case) rank, and the *strongly structurally controllable subspace* $\mathcal{C}_\Gamma$ the controllable subspace of a realization attaining that. Only the two *dimensions* are invariants of the pattern — the subspace itself varies with the realization, a point Chapter 8 takes up. For every realization,

$$\dim \mathcal{C}_\Gamma \;\le\; \operatorname{rank} \mathcal{C}_F \;\le\; \dim \mathcal{C}_\Lambda \;\le\; n. \tag{6.1}$$

Example 6.2 (The diamond)**.** Take the pattern of Fig. 6.1: input at node 1, two parallel paths 1→2→4 and 1→3→4. Symbolically,

$$\mathcal{C}_F = \big[\, e_1,\;\; a_{21}e_2 + a_{31}e_3,\;\; (a_{42}a_{21} + a_{43}a_{31})\, e_4,\;\; 0 \,\big].$$

Generically the three non-zero columns are independent: $\operatorname{rank} \mathcal{C}_F = 3$, so $\dim \mathcal{C}_\Lambda = 3$. But the only non-zero entry of the third column is a *multi-term* in the sense of Definition 4.26 — the two paths to node 4 have equal length — and the realization $a_{21} = a_{31} = a_{42} = 1$, $a_{43} = -1$ cancels it, leaving rank 2; no realization does worse, so $\dim \mathcal{C}_\Gamma = 2$. Each inequality of Eq. (6.1) is strict for some realization:

$$2 \;\le\; \operatorname{rank} \mathcal{C}_F \;\le\; 3 \;<\; 4.$$

The pattern is never controllable ($\dim \mathcal{C}_\Lambda < 4$), yet *how much* of it is controllable genuinely depends on the weights — exactly the information the two subspaces record. Note also what never varies: nodes 2 and 3 are controllable only in the combination $a_{21}e_2 + a_{31}e_3$, one direction for two nodes — the twin-star failure of Example 5.5 appearing inside a larger graph.

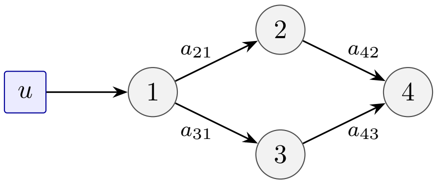


Figure 6.1: The diamond of Example 6.2: two equal-length paths to node 4 make its one non-zero entry of $\mathcal{C}_F$ a multi-term, which the realization of that example makes zero, and one input for the twins 2, 3 caps the generic dimension at three.

6.2 Algebraic Necessary and Sufficient Conditions

Theorem 6.3 (Full dimension)**.** *The family $\mathcal{Q}(A)$ with input matrix B is SC iff* $\dim \mathcal{C}_\Lambda = n$*, and SSC iff* $\dim \mathcal{C}_\Gamma = n$*.*

Every graph-theoretic criterion in this book is a combinatorial condition equivalent to one of these two equalities. Revisiting Chapter 5's examples: the chain has $\dim \mathcal{C}_\Gamma = n$ (SSC), the flip family has $\dim \mathcal{C}_\Lambda = n$ but $\dim \mathcal{C}_\Gamma = n-1$ (SC only), the twin star has $\dim \mathcal{C}_\Lambda = 2 < 3$ (never), and the diamond sits strictly inside both gaps.

6.3 Generic Dimension on Graphs

Theorem 6.4 (Generic dimension)**.** $\dim \mathcal{C}_\Lambda$ *equals the maximum number of state nodes that can be covered by a disjoint set of stems and cycles on the* reachable part *(Definition 4.13) of the pattern graph.*

This is Section 5.3 made quantitative: the largest surviving minor of $\mathcal{C}_F$ is built from the largest disjoint stem-and-cycle cover, and the count is computable in polynomial time. The restriction to the reachable part is necessary: a covered but *unreachable* cycle establishes nothing, and dropping the restriction makes the count wrong — Example 7.4 in Chapter 7 works out that case in full. (All three patterns below are input-connected, so there the restriction has no effect.)

Example 6.5 (Covers yield dimensions)**.** *Four-node network* (Fig. 4.1): the single stem $u \to 1 \to 2 \to 3 \to 4$ covers all four nodes, so $\dim \mathcal{C}_\Lambda = 4$ — the graph-side counterpart of the determinant $a_{21}^3 a_{32}^2 a_{43}$ computed in Example 4.25. *Diamond* (Fig. 6.1): a stem can take only one of the two paths — $u \to 1 \to 2 \to 4$ covers $\{1, 2, 4\}$ — and no disjoint cycle exists to pick up node 3, so $\dim \mathcal{C}_\Lambda = 3$, matching the symbolic computation. *Twin star*: the longest stem covers $\{1, 2\}$ or $\{1, 3\}$, so $\dim \mathcal{C}_\Lambda = 2$. In all three cases the algebra of Chapter 5 and a cover found from the graph alone agree exactly.

6.4 Bounding Worst-Case Dimension

For the worst-case dimension no such clean formula is available. What is available, at statement level: lower bounds from zero-forcing derived sets (a propagation process defined in Section 7.4) and from disjoint *shortest* stems — on the diamond, the shortest stem $u \to 1 \to 2$ has single-term entries that no realization can cancel, ensuring $\dim \mathcal{C}_\Gamma \geq 2$, which the cancellation of Example 6.2 shows is tight; with several drivers the count over disjoint shortest stems needs two further hypotheses — that no two stems reach the same node in the same number of steps, and that each stem is shortest against *all* the drivers (the input-distance of Definition 4.8), not merely against its own — since otherwise the single-terms of two drivers can cancel against each other — and an exact algorithm on the hierarchical DAGs of Definition 4.21, whose clean distance layers let the worst-case dimension be read off layer by layer. The diamond is such a hierarchical DAG and the four-node network of Fig. 4.1 is not (Example 4.22), so the exact algorithm applies to the first but not the second. Chapter 8 uses these results; the shortest-stem bound is proved as Theorem B.1, and the remaining details are in the references.

6.5 The Four Controllable Subspaces

Two more subspaces complete the picture. Beyond its *dimension*, the controllable subspace also changes *orientation* as the parameters move; intersecting over all realizations yields the *fixed* subspaces — the fixed structurally controllable subspace (FSCS) inside $\mathcal{C}_\Lambda$ and the fixed strongly structurally controllable subspace (FSSCS) inside $\mathcal{C}_\Gamma$ — collecting the directions controllable in every realization. Figure 6.2 shows the containment relations among the four; the fixed pair, and its node-level meaning, is the subject of Chapter 8.

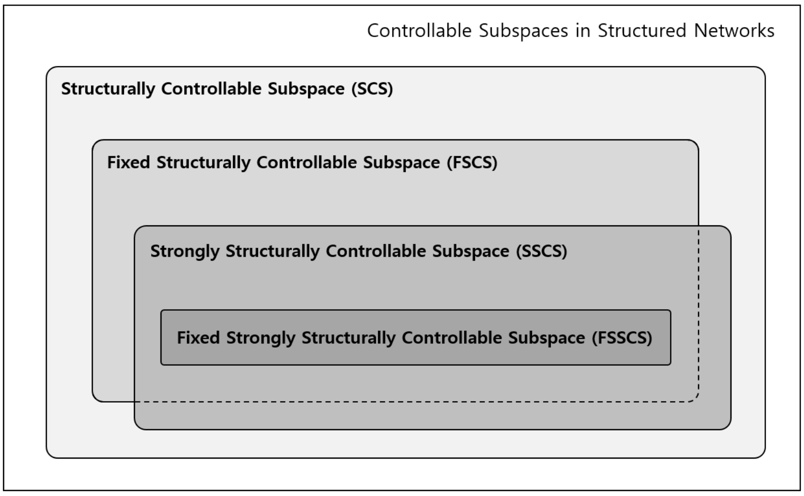


Figure 6.2: The four controllable subspaces of a structured network and their inclusions (figure from the author's dissertation [14]; the four-subspace framework was introduced in [10]). This chapter concerned the outer pair SCS/SSCS; Chapter 8 enters the fixed pair.

6.6 Grid: Subspace Row

	SC (almost all)	**SSC** (all)
algebra	$\dim \mathcal{C}_\Lambda = n$	$\dim \mathcal{C}_\Gamma = n$
graph	stems+cycles cover count (Theorem 6.4)	bounds; exact on HDAGs

Both columns of this row still rely on algebra — a dimension to compute, bounds to establish. The next chapter asks whether SC and SSC of the whole network can instead be decided from the graph alone (Chapter 7).

Sources. [17, 18, 10, 3, 14].

Chapter 7

Whole-Graph Criteria

The graph-theoretic necessary and sufficient conditions for SC and SSC of the entire network: Lin's stems-and-cycles criterion as the combinatorial condition equivalent to $\dim \mathcal{C}_\Lambda = n$, and Mayeda's two subset conditions as the condition equivalent to $\dim \mathcal{C}_\Gamma = n$. The needed vocabulary — matchings for the SC side, dedicated nodes and zero forcing for the SSC side — is introduced on the spot, and every criterion is stated back-to-back with its algebraic counterpart. The most-quoted versions of these theorems carry hypotheses that are often dropped — input-connectedness for Lin, free self-loops for the zero-forcing test — and each of the two comes with a hand-checkable counterexample showing what goes wrong when it is dropped, one of them our running four-node companion.

7.1 Structural Controllability of Graphs

Theorem 6.3 left one task: establish $\dim \mathcal{C}_\Lambda = n$ from the graph alone. By Theorem 6.4 the generic dimension is a cover count, so the whole-graph question reduces to whether a disjoint set of stems and cycles can cover *all* state nodes — almost. The count of Theorem 6.4 is computed on the *reachable* part of the pattern graph, and when the criterion is quoted as a bare cover condition that hypothesis is silently lost. We state it with that hypothesis, in the two classical forms.

Definition 7.1 (Dilation)**.** For a set $S \subseteq \mathcal{V}^S$ of state nodes, let $T(S)$ be the set of all nodes — state or input — with at least one edge into S. The pattern graph has a *dilation* if $|T(S)| < |S|$ for some non-empty S: more nodes than in-neighbours to feed them. Such a set S is itself called a *dilation*, and the pattern graph is *dilation-free* if it has none.

Theorem 7.2 (Structural controllability of a graph)**.** *For a pattern graph with at least one input, the following are equivalent:*

(i) the family is SC;

(ii) the pattern is input-connected (Definition 4.13), and some disjoint set of stems and cycles covers all state nodes;

(iii) the pattern is input-connected, and no dilation exists.

Both graph conditions are checkable in polynomial time [16], and each direction of failure is visible in two lines. If some state node is unreachable, no walk from any input reaches it, so its row of $\mathcal{C}_F$ is identically zero (Proposition 4.24). If S is a dilation, the rows of S in $[A_F\ B]$ have their non-zeros only in the columns of $T(S)$, so those $|S|$ rows have rank at most $|T(S)| < |S|$ *for every realization*: the PBH test (Theorem 3.7) fails at $\lambda = 0$ always. Note what this means against the trichotomy of Proposition 5.4: a whole-graph SC failure admits no exception — if the family is not SC, no realization at all is controllable. Sufficiency is Lin's theorem [13, 19]; the classical proof rebuilds the graph as a union of *cacti* — stems carrying cycle *buds*, each bud joined to the stem by a single *bridge edge* — and a spanning family of cacti is precisely condition (ii).

Example 7.3 (Three graphs decided by Lin's theorem)**.** *Four-node network* (Fig. 4.1): all four nodes are reachable (Example 4.15) and the single stem $u{\to}1{\to}2{\to}3{\to}4$ is a cover — SC by Theorem 7.2, the graph-side reading of $\det \mathcal{C}_F = a_{21}^3 a_{32}^2 a_{43} \not\equiv 0$. *Diamond* (Fig. 6.1): input-connected, but $S = \{2, 3\}$ has $T(S) = \{1\}$ — a dilation; equivalently no cover exceeds three nodes (Example 6.5) — not SC. *Twin star* (Example 3.8): the same dilation, $S = \{2, 3\}$, $T(S) = \{1\}$ — the graph reading of $\det \mathcal{C}_F \equiv 0$ in Example 5.5. The diamond and the star show that input-connectedness alone proves nothing; Fig. 7.1 shows the converse failure.

Example 7.4 (Covered but not reachable from the inputs)**.** In Fig. 7.1, the stem $u{\to}1$ and the cycle $2{\to}3{\to}2$ are disjoint and together cover all three state nodes — yet nodes $2, 3$ are unreachable from the input, and $\dim \mathcal{C}_\Lambda = 1$. A covered cycle only recycles what already arrives from a stem; feed it nothing and it yields nothing. This is why input-connectedness must be stated as a hypothesis and why the cover count of Theorem 6.4 is taken on the reachable part.

Figure 7.1: Why input-connectedness is needed (Example 7.4): stem $u{\to}1$ plus cycle $2{\to}3{\to}2$ cover every state node, but the cycle is unreachable and the family is far from SC.

7.2 Matchings

The dilation condition has an equivalent form in terms of matchings, which organizes the entire SC side of Part III; we fix the vocabulary now.

Definition 7.5 (Matching)**.** A *matching* is a set of edges no two of which share a tail or a head. A state node is *matched* (by a given matching) if it is the head of one of the matching's edges, and *unmatched* otherwise. A matching of largest possible size is a *maximum matching*; one that matches every state node is *perfect*.

A matching is a system of node-disjoint paths and cycles — follow matched edges tail-to-head — which is exactly the shape of the covers in Theorem 7.2. The precise statement is Hall's marriage theorem applied to the bipartite graph representation (tails on one side, state nodes on the other; see, *e.g.*, [5]): *some matching that may use input edges matches every state node if and only if the pattern graph is dilation-free.* Counting instead of deciding gives the result of Liu et al. [2]: with a maximum matching taken on the *state subgraph* — the subgraph induced by the state nodes (Definition 4.11), input edges dropped — the minimum number of inputs for SC equals the number of unmatched nodes, and is one when the matching is perfect. That count, its non-uniqueness, the hypotheses it carries, and where the inputs must attach are the subject of Chapter 9; here matchings serve as vocabulary.

Example 7.6 (Matchings of the four-node network)**.** On the state subgraph of Fig. 4.1, the edge set $\{1\to 2,\ 2\to 3,\ 3\to 4\}$ is a matching: tails $1,2,3$ distinct, heads $2,3,4$ distinct. It matches $2,3,4$ and leaves node 1 unmatched — precisely the driver node, which is why one input suffices. No matching of size four exists (a perfect matching would need four distinct heads, but only $3\to 1$ can match node 1, consuming tail 3; heads 3 and 4 must then share the single remaining tail 2, via $2\to 3$ and $2\to 4$, so one stays unmatched), so this matching is maximum. Maximum matchings are not unique: $\{1\to 2,\ 2\to 3,\ 3\to 1\}$ also has size three but leaves node 4 unmatched instead. Different maximum matchings expose different nodes — same count, different placement — a fact Chapter 9 returns to. For the diamond, every matching must choose between the heads 2 and 3 (both have the single tail 1): a maximum matching such as $\{1\to 2,\ 2\to 4\}$ leaves *two* nodes unmatched, the matching form of its dilation.

7.3 Strong Structural Controllability of Graphs

For SSC the condition must hold for all realizations, so no cancellation may be possible at any of them — and the cost is a condition on every *subset* of state nodes, not on one cover. The subset vocabulary comes from the author's reinterpretation of Mayeda's theorem [14, 20]:

Definition 7.7 (Dedicated and sharing nodes)**.** Let $S \subseteq \mathcal{V}^S$ be non-empty. A node v (state or input) with *exactly one* edge into S is a *dedicated node* of S; a node with two or more edges into S is a *sharing node* of S.

A dedicated node touches S through a single weight, and a single non-zero weight cannot be cancelled; one linear-algebra lemma makes this precise. Recall from Definition 4.1 that a *pattern matrix* is a matrix each of whose entries is either structurally zero or a free non-zero parameter.

Lemma 7.8 (Full row rank for all realizations)**.** *A pattern matrix has full row rank at every realization if and only if every non-empty set R of rows admits a column with exactly one non-zero entry in R.*

Proof. ($\Leftarrow$) If some realization had a linear dependency among its rows, let R be the support of one; the column with exactly one non-zero in R contributes a single term (non-zero coefficient times non-zero entry) to the dependency — impossible. ($\Rightarrow$) If some R has no such column, give every row of R the coefficient 1 and choose each column's free entries within R to sum to zero — possible, since each column has none or at least two of them — giving a realization with dependent rows. □

Now run the PBH test (Theorem 3.7) over the whole family, splitting on λ. At $\lambda = 0$ the test matrix is $[A_F\ B]$; its column v has non-zeros exactly in the rows that v feeds, so "a column with exactly one non-zero in the rows S" *is* a dedicated node of S — possibly a member of S itself, since the diagonal of A_F is structurally zero. At $\lambda \neq 0$ the shift $-\lambda$ puts a non-vanishing entry on every diagonal position, and two things change: if a node $p \in S$ has no out-neighbour inside S, its column meets the rows S only in the diagonal entry $-\lambda$, so no dependency can have support all of S — (C2) asks nothing of such subsets; and a member of S with exactly one edge into S is no longer dedicated, since the shift puts a second non-zero, $-\lambda$, in its own row, so its single edge weight can be balanced against that diagonal entry. What survives is exactly Mayeda's pair of conditions [15], in modern algebraic form [21]:

Theorem 7.9 (Strong structural controllability of a graph)**.** *Let each input attach to a single driver node (the standard convention of Part II, declared in Section 4.1). The family is SSC if and only if both hold:*

(C1) every non-empty $S \subseteq \mathcal{V}^S$ has a dedicated node — possibly a node of S itself;

(C2) every non-empty $S \subseteq \mathcal{V}^S$ in which each node has at least one out-neighbour inside S has a dedicated node outside *S.*

Neither condition implies the other, and both are needed: (C1) alone is the $\lambda = 0$ test, (C2) alone the $\lambda \neq 0$ test. Note also the relation to Definition 7.1: a dilation means S has too few in-neighbours; failing (C1) means S has no *dedicated* in-neighbour — the in-neighbours may be many and all sharing. Dedicated-node analysis is dilation analysis sharpened from counting in-neighbours to counting each in-neighbour's edges into S [14]. The cost of the sharpening is that the conditions quantify over all 2^n subsets, so a direct check is exponential in n (see Remark 7.17 for propagation criteria that avoid the enumeration); for patterns with a self-loop at every state node the two conditions collapse into one — (C2) absorbs (C1) [20] — and the next section gives a test of polynomial complexity in exactly that regime (Theorem 7.14).

Example 7.10 (The four-node network passes both conditions)**.** For the pattern of Fig. 4.1, every subset has its dedicated node; the interesting cases:

- $S = \{3, 4\}$: node 2 is *sharing* (edges into both 3 and 4), but node $3 \in S$ is dedicated — its only edge into S is 3→4, since 3→1 leaves S. (C1) holds through a member of S; and node 4 has no out-neighbour in S, so (C2) asks nothing of this subset.
- $S = \{1, 2, 3\}$: a cycle — every node has an out-neighbour inside, so (C2) demands an outside dedicated node, and the input provides it: u has exactly one edge into S, at node 1.
- $S = \mathcal{V}^S$: again u is dedicated. Since node 4 has no out-edges, (C2) asks nothing of any subset containing it.

The remaining subsets are routine, and both conditions hold: the family is SSC — the subset-level reading of the single-monomial determinant $a_{21}^3 a_{32}^2 a_{43}$ of Example 4.25.

Example 7.11 (Diamond and star fail (C1))**.** For the diamond (Fig. 6.1), take $S = \{2, 3\}$: node 1 is a sharing node (edges into 2 and 3), the input reaches only 1, and nodes $2, 3$ send nothing into S — no dedicated node exists and (C1) fails, the same subset that already carried the dilation. The twin star fails identically at its two end nodes. Both families are not SSC for the strongest reason: they are not even SC.

Remark 7.12 (A sharing input: the flip family). The flip family (Example 5.3) is SC but not SSC, and Definition 7.7 explains why even though its input $b = (1, 1)^\top$ feeds two nodes, outside the single-attachment convention of Theorem 7.9: for $S = \{1, 2\}$ every in-neighbour — node 1, node 2, and the input itself — feeds both nodes of S. All sharing, no dedicated node, and indeed the balanced realizations $a_{21} + a_{22} = a_{11} + a_{12}$ are uncontrollable.

7.4 Zero Forcing

The subset conditions of Theorem 7.9 quantify over all 2^n subsets. *Zero forcing* replaces that enumeration with a propagation process that reads the graph alone [22, 23].

Definition 7.13 (Zero forcing)**.** Color the driver nodes black and all other state nodes white, then apply the *color-change rule* until no more white nodes can be colored black: a black node with exactly one white out-neighbour forces that neighbour black. The final set of black nodes is the *derived set*; the driver set is a *zero forcing set* (ZFS) if the derived set is all of $\mathcal{V}^S$.

Forcing never uses a weight: each step counts the white out-neighbours of one black node, and the process is over as soon as no black node has exactly one. Such a node is, in the vocabulary of Definition 7.7, a dedicated node of the current white set lying outside it — so forcing searches for outside dedicated nodes one step at a time instead of one subset at a time. When every diagonal entry is free that search is all (C1) and (C2) ask for, and the test is exact:

Theorem 7.14 (ZFS criterion)**.** *Suppose every diagonal entry of A_F is a free parameter — a self-loop of arbitrary weight, zero included, permitted at every state node — while off-diagonal entries follow the pattern graph as usual. Then the family is SSC if and only if the driver set is a zero forcing set. The test is polynomial time.*

The hypothesis cannot be dropped. With free diagonals the two ways in which a loop-free pattern can satisfy (C1)/(C2) *without an outside dedicated node* both close. A member of S can no longer serve as its dedicated node: if it has an edge into S, its own loop makes two, and if its only entry in S is that loop, the weight is free *including zero*, so it cannot be the guaranteed single non-zero that Lemma 7.8 asks for. And a node without out-neighbours in S now has one, its own loop, so (C2) applies to its subsets as well. Mayeda's two conditions therefore collapse into the single demand "every S has a dedicated node outside S", which is precisely what forcing checks. Drop the hypothesis and the equivalence breaks, in exactly one direction: forcing remains sufficient for SSC (the free-diagonal family contains the zero-diagonal one), but it is no longer necessary. Our four-node companion is the counterexample.

Example 7.15 (The four-node network: SSC without a ZFS)**.** Run forcing on Fig. 7.2. The driver 1 forces 2 (its only out-neighbour); then node 2 has *two* white out-neighbours, 3 and 4, and the process stops with derived set $\{1, 2\}$. The drivers are not a ZFS — yet the family is SSC (Example 7.10). The remaining white set $W = \{3, 4\}$ carries the two cases that forcing cannot check: its dedicated node is node 3, *inside* W, and the subsets containing node 4 are left unconstrained by (C2) for a diagonal reason — neither case is available to a rule that only lets outside black nodes act. The conclusion of Theorem 7.14 is nonetheless correct *for its own hypothesis*: allow self-loops and controllability really does fail at some realization. With unit edge weights and a unit self-loop at node 3, rows 3 and 4 of the realization

$$A' \;=\; \begin{bmatrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 \end{bmatrix}$$

are identical, so $w = e_3 - e_4$ satisfies $w^\top A' = 0$ and $w^\top b = 0$: PBH fails at $\lambda = 0$. Symbolically, self-loops d_3, d_4 at nodes $3, 4$ give

$$\det \mathcal{C}_F \;=\; a_{21}^3\, a_{32}\big(a_{32}a_{43} + a_{42}(d_4 - d_3)\big)$$

— a sum of monomials that vanishes at some realization, whereas the zero-diagonal family keeps a single monomial, which no realization can cancel. In short: state the zero-forcing criterion *with* its self-loop hypothesis; for structurally loop-free patterns it is a sufficient condition only, and the exact criterion remains Theorem 7.9.

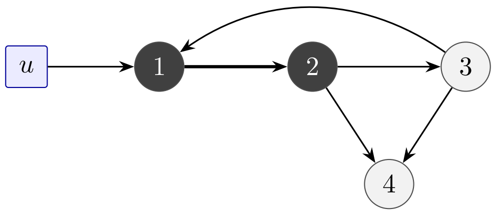


Figure 7.2: Forcing on the four-node pattern (Example 7.15): 1 forces 2, then node 2 has two white out-neighbours (3 and 4) and the process stops with derived set $\{1, 2\}$ — although the family is SSC.

Example 7.16 (Forcing on the chain)**.** For the chain $u{\to}1{\to}2{\to}3$ of Example 5.7, forcing proceeds along the chain: 1 forces 2, 2 forces 3 — a ZFS, hence SSC even with arbitrary self-loops, and *a fortiori* for the loop-free chain, matching $\det \mathcal{C}_F = a_{21}^2 a_{32}$. The order in which the nodes are forced — the *forcing chain* — retraces the stem; that is no accident: a ZFS corresponds to a *constrained matching* — a matching that is the unique one on its node set — whose paths are exactly the forcing chains [23]. Uniqueness is the matching-side form of "nothing can cancel".

Remark 7.17 (Repairing the criterion for loop-free patterns)*.* The gap shown by Example 7.15 can be closed: for patterns with structurally zero diagonals, exact zero-forcing-type criteria have been developed that run *two* propagation rules on the graph with its self-loops *specified* — one rule per Mayeda condition, covering both a dedicated node inside S and the subsets that (C2) does not constrain — while constrained matchings give the equivalent combinatorial form of the plain forcing test [23]; the survey [3] collects these variants. The minimum *size* of a zero forcing set, and its role as the SSC counterpart of Liu's driver count, belongs to Chapter 9.

7.5 Grid: Whole-Graph Row

The whole-graph row of the book's running grid is now complete; each graph cell is the exact combinatorial necessary and sufficient condition for the algebraic cell above it.

	SC (almost all)	**SSC** (all)
algebra	$\dim \mathcal{C}_\Lambda = n$ (Theorem 6.3)	$\dim \mathcal{C}_\Gamma = n$ (Theorem 6.3)
graph	input-connected + stems/cycles cover = no dilation (Theorem 7.2)	dedicated node for every subset (C1)$\wedge$(C2) (Theorem 7.9)
cost	polynomial [16]	direct check exponential; ZFS polynomial under free self-loops (Theorem 7.14)

Remark 7.18 (Symmetry, dilation, and the PBH picture)*.* Every whole-graph failure in this chapter is a PBH failure at some mode, which the graph exhibits. In the twin star, *every* realization admits the left null vector $w = (0, a_{31}, -a_{21})$: $w^\top A_F = 0$ and $w^\top b = 0$ — the left eigenvector of Theorem 3.7 concentrated on the dilation $\{2, 3\}$. At equal weights it becomes the antisymmetric direction $e_2 - e_3$ of the end-node-swapping automorphism that fixes the driver. Equitable partitions systematize this: nodes grouped symmetrically with respect to the drivers generate uncontrollable modes, giving graph-symmetry *necessary* conditions for SC [24, 5]. Dilations, sharing nodes, remaining white sets, and quotient symmetries are four forms of one statement: some direction of state space is fed by too few independent inputs.

So the whole graph can be decided from its structure alone: covers for SC, dedicated nodes for SSC, forcing when self-loops are free. But most real networks simply fail these tests, and a conclusion of "not controllable" for the whole graph is where analysis starts, not ends: which individual nodes remain controllable in every realization? That is the question of Chapter 8.

Sources. Lin's theorem and its formulations: [13, 19, 17, 16]; matchings and the driver count: [2, 5]; Mayeda's two conditions and their modern form: [15, 21]; dedicated/sharing-node reinterpretation and the self-loop collapse: [14, 20]; zero forcing and constrained matchings: [22, 23, 25]; symmetry conditions: [24]; survey treatment of the whole chapter: [3].

Chapter 8

Node-Level Controllability: Fixed Subspaces and Fixed Nodes

Most real networks fail the whole-graph tests of the previous chapter, so the practical question is which individual *nodes remain controllable in every realization. Even at fixed dimension the controllable subspace* rotates *as the parameters move; the directions that survive every rotation form the fixed subspaces FSCS and FSSCS, and the coordinate axes they contain name the fixed nodes. Both notions come, as always, in an SC and an SSC version, each with an algebraic comparison — add an input, watch a dimension — and a graph criterion: matched layers on hierarchical DAGs for the SC side, shortest control paths for the SSC side. The SC side comes out exact; on the SSC side the algebraic comparison decides in one direction only, and the graph criterion is exact for a related class of nodes. A ranking of the nodes by robustness to weight uncertainty completes the book's 2×2 grid.*

8.1 Orientation Problem

Chapter 6 settled the *dimension* of the controllable subspace: it moves between $\dim \mathcal{C}_\Gamma$ and $\dim \mathcal{C}_\Lambda$, both invariants of the pattern. But a subspace is more than its dimension. Two realizations may agree that the controllable subspace is a plane and still disagree about *which* plane — and a state controllable in one of them may be uncontrollable in the other. The following three-node pattern shows this in its simplest form.

Example 8.1 (A rotating plane)**.** Drive node 1 and let it feed two nodes that also feed each other (Fig. 8.1): $u \to 1$, edges $1 \to 2$ (a_{21}), $1 \to 3$ (a_{31}), $2 \to 3$ (a_{32}), $3 \to 2$ (a_{23}). By stem enumeration (Proposition 4.24),

$$\mathcal{C}_F = \begin{bmatrix} 1 & 0 & 0 \\ 0 & a_{21} & a_{23}a_{31} \\ 0 & a_{31} & a_{32}a_{21} \end{bmatrix}, \qquad \det \mathcal{C}_F = a_{21}^2 a_{32} - a_{23} a_{31}^2.$$

Each non-zero entry is a *single-term* (Definition 4.26), one WP with one walk per length and target: $a_{23}a_{31}$ is $u{\to}1{\to}3{\to}2$, $a_{32}a_{21}$ is $u{\to}1{\to}2{\to}3$. The cancellation therefore happens not inside an entry, as in the diamond, but across the 2×2 minor. Generically the determinant is non-zero, so the pattern is SC and $\dim \mathcal{C}_\Lambda = 3$; on the variety $a_{21}^2 a_{32} = a_{23} a_{31}^2$ the rank drops to 2, and never lower (the second column cannot vanish), so $\dim \mathcal{C}_\Gamma = 2$. On that variety the third column is proportional to the second, and the controllable subspace is the plane

$$\operatorname{Im} \mathcal{C}_F = \operatorname{span}\{\, e_1,\ a_{21}e_2 + a_{31}e_3 \,\}.$$

Every such plane contains the x_1-axis, and as the ratio $a_{21} : a_{31}$ sweeps the variety the plane *rotates* about that axis: Fig. 8.2 shows three worst-case realizations, from nearly the x_1x_2-plane to nearly the x_1x_3-plane. The dimension is pinned at two, yet whether x_2 (or x_3) is controllable depends entirely on the realization. Only the axis on which all the planes agree — the x_1-axis — is controllable in every worst case.

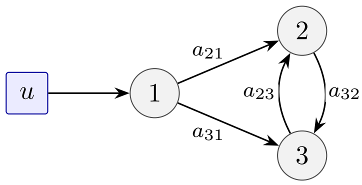
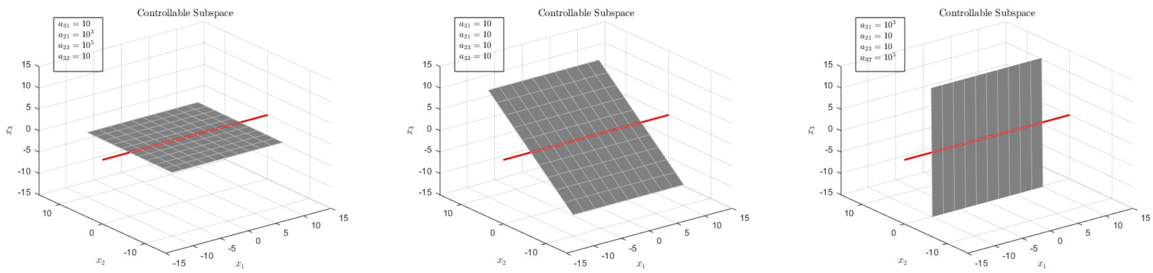


Figure 8.1: The pattern of Example 8.1: one driver, two mutually coupled followers. SC but not SSC; worst-case rank 2 on the variety $a_{21}^2 a_{32} = a_{23} a_{31}^2$.



(a) $a_{21} \gg a_{31}$ (b) $a_{21} = a_{31}$ (c) $a_{21} \ll a_{31}$

Figure 8.2: Three worst-case realizations of Example 8.1, all with $a_{21}^2 a_{32} = a_{23} a_{31}^2$ (figures from the author's dissertation [14]). The controllable plane $\operatorname{span}\{e_1,\, a_{21}e_2 + a_{31}e_3\}$ rotates about the x_1-axis (red line): (a) $a_{21} = 10^3$, $a_{23} = 10^5$, $a_{31} = a_{32} = 10$ — almost the x_1x_2-plane; (b) all weights 10 — the diagonal position; (c) $a_{31} = 10^3$, $a_{32} = 10^5$, $a_{21} = a_{23} = 10$ — almost the x_1x_3-plane. The axis common to every position is the fixed subspace.

8.2 Fixed Subspaces

The cure for a rotating subspace is an intersection: keep only the directions that every realization retains.

Definition 8.2 (FSCS and FSSCS)**.** For a pattern matrix A with family set $\mathcal{Q}(A)$ and fixed B, the *fixed structurally controllable subspace* and the *fixed strongly structurally controllable subspace* are

$$\mathcal{C}_\Lambda^F = \bigcap_{\substack{A_F \in \mathcal{Q}(A) \\ \operatorname{rank} \mathcal{C}_F = \dim \mathcal{C}_\Lambda}} \operatorname{Im} \mathcal{C}_F, \qquad \mathcal{C}_\Gamma^F = \bigcap_{\substack{A_F \in \mathcal{Q}(A) \\ \operatorname{rank} \mathcal{C}_F = \dim \mathcal{C}_\Gamma}} \operatorname{Im} \mathcal{C}_F :$$

the intersection of the generic-dimension realizations of the controllable subspace, and of the worst-case-dimension realizations, respectively [26, 10].

Unlike $\mathcal{C}_\Lambda$ and $\mathcal{C}_\Gamma$, which fix a dimension but admit countless realizations, the fixed subspaces are *unique*: intersections do not depend on the realization. They complete the containment picture of Fig. 6.2: the FSSCS sits inside every worst-case realization *and* inside the FSCS,

$$\mathcal{C}_\Gamma^F \subseteq \mathcal{C}_\Lambda^F,$$

while the FSCS and a given SSCS realization are in general not comparable. The inclusion is not immediate: it compares intersections over *different* sets of realizations; the proof is deferred to the Appendix (adapted from [10]).

Example 8.3 (Fixed subspaces of the rotating plane)**.** In Example 8.1 the pattern is SC, so every generic realization has $\operatorname{Im}\mathcal{C}_F = \mathbb{R}^3$ and $\mathcal{C}_\Lambda^F = \mathbb{R}^3$ — on the SC side the fixed question is vacuous exactly when the whole-graph test passes. The worst-case realizations are the rotating planes, and intersecting two positions already leaves only the common axis: $\operatorname{span}\{e_1, 10e_2 + 10e_3\} \cap \operatorname{span}\{e_1, 10e_2 + 10^3 e_3\} = \operatorname{span}\{e_1\}$, so

$$\mathcal{C}_\Gamma^F = \operatorname{span}\{e_1\},$$

of dimension $1 < 2 = \dim \mathcal{C}_\Gamma$: the worst case always controls a plane, but only one *fixed* direction lies in all of them.

Example 8.4 (Fixed subspaces of the diamond)**.** The diamond (Example 6.2 and Fig. 6.1) has $\mathcal{C}_F = [\,e_1,\ a_{21}e_2 + a_{31}e_3,\ (a_{42}a_{21} + a_{43}a_{31})e_4,\ 0\,]$. The generic realizations ($a_{42}a_{21} + a_{43}a_{31} \neq 0$) span $\operatorname{span}\{e_1,\ a_{21}e_2 + a_{31}e_3,\ e_4\}$; the middle direction rotates with $a_{21} : a_{31}$ while e_1 and e_4 stay put, so

$$\mathcal{C}_\Lambda^F = \operatorname{span}\{e_1, e_4\},$$

of dimension $2 < 3 = \dim \mathcal{C}_\Lambda$: even the generic subspace carries a rotating direction that the intersection discards. The worst-case realizations (cancellation $a_{42}a_{21} = -a_{43}a_{31}$) span the planes $\operatorname{span}\{e_1,\ a_{21}e_2 + a_{31}e_3\}$ — the rotating-plane picture again, appearing inside the diamond — and

$$\mathcal{C}_\Gamma^F = \operatorname{span}\{e_1\} \subsetneq \mathcal{C}_\Lambda^F.$$

8.3 Fixed Nodes

A subspace answers "which directions"; the node-level question is "which nodes". The translation is the standard basis: node k is individually controllable exactly when its own axis e_k lies in the subspace — membership of a skew direction like $a_{21}e_2 + a_{31}e_3$ controls a *combination* of nodes, never each node separately.

Definition 8.5 (FSC and FSSC nodes)**.** Node k is a *fixed structurally controllable* (FSC) node if $e_k \in \mathcal{C}_\Lambda^F$, and a *fixed strongly structurally controllable* (FSSC) node if $e_k \in \mathcal{C}_\Gamma^F$.

The literature also calls these SC and SSC *states* [26, 3]; the name *SSC node* is used differently here (Remark 8.18). Three consequences of the definitions are worth recording at once.

1. *Inclusion.* $\mathcal{C}_\Gamma^F \subseteq \mathcal{C}_\Lambda^F$ gives: every FSSC node is an FSC node. The converse fails — diamond node 4 below.
2. *Counts versus dimensions.* The number of FSSC nodes can be strictly smaller than $\dim \mathcal{C}_\Gamma$: in Example 8.3 the worst case controls a 2-dimensional plane, yet only node 1 is FSSC. Dimensions count directions; fixed nodes count *axes*.

3. *Drivers are fixed.* A driver node already has an input, and duplicating an input changes nothing, so drivers pass every test below trivially [27].

On the running examples: the rotating plane has FSC nodes $\{1,2,3\}$ (SC pattern) but only FSSC node 1; the diamond has FSC nodes $\{1,4\}$ and FSSC node 1, while the twins $2,3$ — controllable only in the combination $a_{21}e_2 + a_{31}e_3$ — are fixed in neither sense. It remains to *compute* these sets without computing the subspaces; that is the subject of the next two sections.

8.4 Input-Addition Tests

The algebraic characterization compares two patterns: if node k were already controllable in every (generic, resp. worst-case) realization, then giving it its own input should add nothing.

Theorem 8.6 (Fixed-node tests)**.** *Let the pattern graph be input-connected (Definition 4.13) and let $B_k = [\,B,\ e_k\,]$ denote the input matrix with a dedicated input added at node k. Then k is an FSC node if and only if the addition does not increase* $\dim\mathcal{C}_\Lambda$ *[26]. On the SSC side the same test is necessary but not sufficient: if k is an FSSC node the addition does not increase* $\dim\mathcal{C}_\Gamma$*, so a strict increase rules k out, while a non-increase decides nothing [10].*

Remark 8.7 (No converse on the SSC side)*.* Attach a new node 5 to the diamond by the edge $2 \to 5$. Its $\mathcal{C}_F$ has third column $(a_{42}a_{21} + a_{43}a_{31})e_4 + a_{52}a_{21}e_5$, whose e_5 entry is a single-term, so $\operatorname{rank}\mathcal{C}_F = 3$ at every realization and an input at 5 leaves $\dim\mathcal{C}_\Gamma$ at 3 — yet $e_5 \notin \mathcal{C}_\Gamma^F$, since the realizations with $a_{42}a_{21} + a_{43}a_{31} \neq 0$ do not contain it. The exact criterion is the subspace equality of Lemma B.3.

Both proofs (deferred to the Appendix) run through the invariance of the controllable subspace under A_F: if e_k already lies in the subspace at each realization the relevant intersection runs over — generic on the SC side, worst-case on the SSC side — then all its propagations $A_F^j e_k$ do too, so the augmented pair spans nothing new. That argument gives "fixed node $\Rightarrow$ no increase" on both sides. Only the SC side has a converse, and it turns on $\dim\mathcal{C}_\Lambda$ being a *maximum*: a single generic realization missing e_k already lifts the augmented maximum by one. Since $\dim\mathcal{C}_\Gamma$ is a *minimum*, a realization missing e_k need not move it, and the implication does not reverse.

Both readings compare dimensions — which is why Chapter 6 had to come first. Each side inherits the computability of its own dimension:

- *FSC side: fully computable.* $\dim\mathcal{C}_\Lambda$ is the maximum disjoint stem-and-cycle cover count (Theorem 6.4), a polynomial matching computation; running it once for the pattern and once per candidate node settles all FSC nodes in $O(n\,(n + |\mathcal{E}|))$ time [26, 27].
- *FSSC side: a one-sided screen, computable where* $\dim\mathcal{C}_\Gamma$ *is.* The worst-case dimension has no general formula (Section 6.4); on the hierarchical DAGs (Definition 4.21) it is exact via the subgraph reduction of [10], so there the comparison can be carried out — but it only rules nodes out (Remark 8.7), and in general it is open, exactly as open as the dimension itself. Deciding FSSC membership needs Lemma B.3.

Example 8.8 (Testing the diamond)**.** The diamond has $\dim\mathcal{C}_\Lambda = 3$ and $\dim\mathcal{C}_\Gamma = 2$ (Examples 6.2 and 6.5), and it is an HDAG (Definition 4.21; distance layers $\{1\},\{2,3\},\{4\}$), so both dimensions are computable. *Node* 4*, FSC test:* with inputs at 1 and 4, disjoint stems can cover at most $\{1,2,4\}$ or $\{1,3,4\}$ — the new stem at 4 is the single node 4, and any stem from u through 2 or 4 collides with it — so the cover count stays 3: node 4 *is* FSC.

Node 4*, FSSC test:* for the augmented pair $B_4 = [\,e_1, e_4\,]$ the columns e_1, e_4, $a_{21}e_2 + a_{31}e_3$ are independent for *every* realization — the cancellation $a_{42}a_{21} = -a_{43}a_{31}$ makes the old third column vanish but not the new input column — so the worst case rises from 2 to 3. A strict increase is the direction Theorem 8.6 does decide: some worst case must have been missing e_4, so node 4 is *not* FSSC. *Node* 2 *(and by symmetry* 3*):* an input at 2 lets stems $u{\to}1{\to}3$ and $2{\to}4$ cover all four nodes, raising $\dim \mathcal{C}_\Lambda$ to 4: not FSC, hence not FSSC. *Node* 1*:* a driver, fixed in both senses. Conclusion: FSC nodes $\{1, 4\}$, FSSC node $\{1\}$ — matching the subspaces of Example 8.4 axis for axis.

8.5 Graph Criteria

The comparisons of Theorem 8.6 still talk about dimensions. The graph criteria read the fixed nodes off the picture directly — one per side.

8.5.1 FSC Nodes on Hierarchical DAGs: Matched in Every Maximum Disjoint-Stem Set

Let the pattern be an HDAG (Definition 4.21) whose drivers are its source nodes. Its *layers* $\ell_1, \ell_2, \ldots$ can be built in two ways, and the HDAG hypothesis is what makes the two agree. Recursively: ℓ_1 collects the state nodes with no incoming state edges (all drivers among them); remove them and repeat, so that every edge runs from a lower-index layer to a higher one [27]. By distance: ℓ_k collects the nodes at input-distance k. The recursive layering works on any DAG, but there an edge may skip layers and the two layerings then differ; on an HDAG every walk into a node has one length, so no edge skips a layer and the layerings coincide — the diamond's $\{1\}, \{2, 3\}, \{4\}$ is both. Everything below rests on that agreement.

For a layer ℓ_k, call a set of disjoint stems — at most one per driver, since two stems from one driver share it — that covers as many nodes of ℓ_k as possible a *maximum disjoint-stem set for* ℓ_k; the nodes of ℓ_k it covers are its *matched nodes* — a per-layer notion, not the matching of Definition 7.5. Several such sets may be feasible, with different matched sets.

Theorem 8.9 (Matched-node criterion)**.** *Let the pattern be an HDAG (Definition 4.21) whose drivers are its source nodes, so that the recursive layers above are the distance layers. Then node $i \in \ell_k$ is an FSC node if and only if i is matched in* every *feasible maximum disjoint-stem set for ℓ_k [27].*

Corollary 8.10 (Single driver)**.** *On an HDAG with one driver, node $i \in \ell_k$ is an FSC node if and only if it is alone in its layer, $|\ell_k| = 1$.*

Remark 8.11 (Why the hierarchy is assumed). On $1 \to 2$, $1 \to 3$, $2 \to 3$, $2 \to 4$ with driver 1, edge $1 \to 3$ skips a layer: recursion gives $\{1\}, \{2\}, \{3, 4\}$, the distance layers are $\{1\}, \{2, 3\}, \{4\}$. Node 2 is alone in its recursive layer, so Corollary 8.10 without the hypothesis would call it FSC; yet a driver at 2 frees the old stem to run $1 \to 3$ while $2 \to 4$ takes the rest, lifting $\dim \mathcal{C}_\Lambda$ to 4: node 2 is not FSC, though [27] states the criterion for general layered DAGs.

The intuition is an HDAG intuition. Because the layers are the distance layers, a stem meets ℓ_k in at most one node and does so at its k-th step, so a node matched in every maximum disjoint-stem set for ℓ_k is already covered: an input added there re-traces an existing stem and covers nothing new. A node missed by *some* maximum disjoint-stem set can instead be covered by a new stem from an added input. Both halves need the two layerings to agree

— once an edge skips a layer, the added input can send the old stem down a different path and cover more (Remark 8.11). Proofs in the Appendix. Layer by layer the criterion is a disjoint-path computation: connecting a super-source to the drivers and a super-sink to ℓ_k turns it into the m-vertex-disjoint-paths problem, solvable in $O(n + |\mathcal{E}|)$ for one driver and $O(|\mathcal{E}|\, n^{m-1})$ for $m \geq 2$ drivers on DAGs — polynomial for any fixed number of drivers [27].

Example 8.12 (The diamond, by layers)**.** The diamond is an HDAG (Definition 4.21 and Example 4.22) driven at its only source, so the criterion applies: single driver, layers $\{1\}, \{2, 3\}, \{4\}$ by recursion and by distance alike. Corollary 8.10: nodes 1 and 4 are alone in their layers — FSC; the twins share ℓ_2 — not FSC. Via Theorem 8.9 directly: the maximum disjoint-stem sets for ℓ_2 are the single stems $u \to 1 \to 2$ and $u \to 1 \to 3$, with matched sets $\{2\}$ and $\{3\}$ — intersection empty; for ℓ_3 both $u \to 1 \to 2 \to 4$ and $u \to 1 \to 3 \to 4$ match $\{4\}$ — intersection $\{4\}$. Same outcome as Example 8.8, no algebra used.

Remark 8.13 (Layers decide target control too)*.* The layer count settles more than full-state questions. Suppose only a *target set* of states must be steered — the setting of Section 4.5, with condition $\operatorname{rank}(C\,\mathcal{C}_F) = p$. On a tree rooted at a single driver, every node is reached by a unique walk, so the distance layers are clean (a tree is the simplest hierarchical DAG of Definition 4.21), and the same one-representative-per-layer counting gives the classical *k-walk criterion*: the target set is generically controllable iff *each layer contains at most one target node* [11]. Two targets in one layer collide in the same column of $C\,\mathcal{C}_F$ for the same reason the twins of Example 8.12 share one direction.

8.5.2 Worst-Case Conditions: Shortest Control Paths

The SSC-side counterpart localizes the dedicated-node conditions of Theorem 7.9 to a single node. Requiring the dedicated node to lie *outside* the subset strengthens what (C1) asks, and where Theorem 7.9 demanded such a node for *every* subset of state nodes, the node-level notion demands one only for the subsets that contain the node in question.

Definition 8.14 (Controllable subset and SSC node)**.** A set $S \subseteq \mathcal{V}^S$ is a *controllable subset* if some node outside S, state or input, sends *exactly one* edge into S — that is, if S has a dedicated node (Definition 7.7) lying outside S. A state node k is an *SSC node* if every $S \subseteq \mathcal{V}^S$ containing k is a controllable subset [20, 28].

On patterns that are trees (Definition 4.19) when edge directions are ignored (*polytrees*), the condition over all subsets collapses to a statement about the paths from the input node to the sinks of Definition 4.20 — the nodes with no out-edges.

Theorem 8.15 (Shortest-path criterion)**.** *Let an input-connected polytree pattern have a single input node u and sinks $t_1, \ldots, t_m$. Then k is an SSC node if and only if k belongs to the intersection $\mathcal{V}^u_{\text{int}}$ of the shortest-path graphs (control paths) from u to $t_1, \ldots, t_m$ [28].*

An SSC node is thus a node lying on every control path of u. With several inputs, the union over inputs of these per-input intersections is still a set of SSC nodes (sufficiency) [28]; computing all the shortest-path graphs costs $O(pmn^2)$ for p inputs and m sinks, at most $O(n^4)$. And the property transfers down the grid:

Proposition 8.16 (SSC nodes are FSC nodes)**.** *Let an input-connected polytree pattern have a single input node. Then every SSC node is an FSC node. With several inputs the same conclusion holds for every node of the union $\bigcup_j \mathcal{V}^{u_j}_{\text{int}}$ of the per-input intersections [28].*

With one input a polytree is an HDAG, so Theorem 8.9 applies: a node on every control path is matched in every maximum disjoint-stem set for its layer, and an added input cannot raise the cover count — the FSC half of Theorem 8.6 restated on the graph (that proof, and the several-input case, in the Appendix).

Example 8.17 (Pruned diamond, and a gap between the two notions)**.** Delete the diamond's edge $3 \to 4$: $u \to 1$, $1 \to 2$, $1 \to 3$, $2 \to 4$ — a polytree with sinks 3 and 4. The control paths are $u{\to}1{\to}3$ (state nodes $\{1,3\}$) and $u{\to}1{\to}2{\to}4$ (state nodes $\{1,2,4\}$); their intersection is $\{1\}$, so node 1 is the only SSC node (Theorem 8.15), and by Proposition 8.16 it is FSC. The other node classes, for contrast: layers are again $\{1\},\{2,3\},\{4\}$, so the FSC nodes are $\{1,4\}$; and since $\mathcal{C}_F = [\,e_1,\ a_{21}e_2 + a_{31}e_3,\ a_{42}a_{21}e_4,\ 0\,]$ has a single-term entry in its third column, every realization has rank exactly 3 and $\mathcal{C}_\Gamma^F = \operatorname{span}\{e_1, e_4\}$: node 4 is FSSC as well. Two lessons. First, the two worst-case node notions do not coincide: node 4 is FSSC, yet $S = \{2,3,4\}$ has node 1 sending two edges into it and no outside dedicated node, so 4 is not an SSC node. The subset notion of Definition 8.14 asks that *every* subset of state nodes containing 4 be a controllable subset; membership of the single axis e_4 in the FSSCS asks something else, and neither reading is claimed here to imply the other (Remark 8.18). Second, the polytree hypothesis matters: the diamond itself is *not* a polytree (undirected cycle 1 2 4 3), so Theorem 8.15 does not apply to it — but the subset definition still does, and a direct check ($\{2,3\}$ and $\{2,3,4\}$ have no outside dedicated node; every subset containing 1 is served by u) gives SSC node $\{1\}$, agreeing with the FSSC computation of Example 8.8.

Remark 8.18 (A terminology caution)*.* The naming here is not universal. This book reserves *SSC node* for the subset/path notion of Definition 8.14 [20, 28] — every subset containing k controllable — and *FSSC node* for the membership $e_k \in \mathcal{C}_\Gamma^F$ (Definition 8.5), which the min-dimension comparison of Theorem 8.6 can rule out but not confirm. The cited survey [3] attaches the name "SSC node" to *that* min-dimension property, so the survey's "SSC node" is this book's *FSSC node*. The two are genuinely different tests: pruned-diamond node 4 is FSSC but not an SSC node (Example 8.17). No implication in the other direction is asserted in this book; the published results relate the subset/path notion to the *FSC* nodes (Proposition 8.16) and leave its relation to the FSSC nodes open [28].

8.6 Node Ranking and Grid Completion

The node classes assemble into a ranking of the nodes by robustness to weight uncertainty — by how reliably each of them can be controlled when only the pattern is known:

1. **FSSC nodes** — their own axis survives the worst-case intersection — and **SSC nodes** — every subset of state nodes containing them is a controllable subset (Definition 8.14), which on a polytree means lying on every control path. Two different tests, not two names for one (Remark 8.18); each puts its nodes above the next class, the SSC nodes by Proposition 8.16 and the FSSC nodes by $\mathcal{C}_\Gamma^F \subseteq \mathcal{C}_\Lambda^F$;
2. **FSC nodes** — controllable in every generic realization, but lost in some worst cases;
3. **reachable non-fixed nodes** — contribute to the dimension only jointly (the diamond's twins: one direction, two nodes);
4. **unreachable nodes** — never controllable.

On the diamond the ranking reads $1 \succ 4 \succ \{2,3\}$. The ranking has a direct practical use: assign the tasks that *must* be executed to the FSSC and SSC nodes, tolerate parameter variations at the FSC nodes, and treat the remaining nodes as candidates for redesign [28, 3].

And Theorem 8.6 gives the ranking a second reading that opens Part III: a *new input that increases a dimension* always sits on a node that is not fixed — exactly the non-FSC nodes on the SC side, and on the SSC side the one direction that holds. The non-fixed nodes are the effective input sites — so the next question is unavoidable: how many inputs does a network need, and where should they go (Chapter 9)?

With the node row the book's summary grid is complete. Every cell above the node row is a necessary and sufficient condition, stated once in algebra and once on the graph. The node row is where the two sides part company: on the SC side both cells are exact (the graph cell under the hierarchy hypothesis of Definition 4.21), while on the SSC side the min-dimension comparison is only necessary (Remark 8.7) and the shortest-path condition, exact on a polytree with one input, decides the related but distinct class of *SSC nodes* (Remark 8.18):

	SC (almost all)	**SSC** (all)
definition	$\det \mathcal{C}_F \not\equiv 0$	$\det \mathcal{C}_F$ never zero
subspace	$\dim \mathcal{C}_\Lambda = n$	$\dim \mathcal{C}_\Gamma = n$
whole graph	stems+cycles cover (Theorem 7.2)	dedicated subsets; ZFS (Theorem 7.14)
node	FSC: max-dim test (Theorem 8.6); on an HDAG, matched in every maximum disjoint-stem set (Theorem 8.9)	FSSC: min-dim test, necessary only (Theorem 8.6); SSC nodes: shortest-path condition (Theorem 8.15)

Sources. The fixed subspace and the FSC input-addition test originate with [26], with a separator-based follow-up in [29]; $\dim \mathcal{C}_\Lambda$ computability is [17]. The matched-node criterion and its algorithms are the author's [27], stated there for layered DAGs and restricted here to hierarchical ones — a hypothesis neither that paper nor the dissertation [14] carries, and one the general form cannot do without (Remark 8.11); the FSSCS, the min-dimension comparison, and the HDAG computation of $\dim \mathcal{C}_\Gamma$ are [10]; the SSC-node notion (dedicated nodes and controllable subsets) originates in [20], and the shortest-path criterion and the ranking application are [28]. The survey [3] organizes the node-level theory, and the orientation figures and the unified treatment follow the dissertation [14].

Part III

Input Placement: Synthesis

Chapter 9

Minimum Input Problem

Part III reverses the direction of every question so far: instead of judging a given input matrix, we choose one. The first design question is cardinality — how few inputs, and attached where, make the network structurally or strongly structurally controllable — and its two answers bracket the field. For SC the answer is a maximum matching: polynomial and exact, with the minimum count equal to the number of unmatched nodes. For SSC the answer is a minimum zero forcing set, whose size is NP-hard to compute in general, the cost of a worst-case guarantee; solvability in polynomial time returns only for structured classes. The same limitation applies on both sides: the minimum fixes how many *inputs but barely* where *— the same count is achieved by many placements, and Chapter 10 must choose among them.*

9.1 From Analysis to Synthesis

Every criterion of Part II took the pair (A_F, B) as given and returned an answer. Design inverts the arrow. The pattern A — the physics of the network, its wires and couplings — is fixed by the application and not ours to change; what we place is B, the set of nodes we agree to actuate. Reread in this light, each theorem of Chapters 7 and 8 becomes a *feasibility constraint* on that choice. Lin's cover condition (Theorem 7.2) says a driver set is SC-feasible iff, together with the pattern, it leaves no dilation and no unreachable node; Mayeda's pair (Theorem 7.9) says it is SSC-feasible iff every subset has a dedicated node (C1), one lying outside the subset in the cycle-like case (C2). A *driver set*, written $\mathcal{D}$, is a set of state nodes $\mathcal{D} \subseteq \mathcal{V}^S$, each carrying its own dedicated input column e_k; the input matrix is then $B = [\, e_k : k \in \mathcal{D}\,]$. The minimum input problem asks for a smallest feasible driver set:

Problem 9.1 (Minimum input problem, MIP)**.** Given a pattern graph, find a driver set $\mathcal{D}$ of minimum cardinality such that the family is SC (respectively SSC), and locate its nodes.

Two questions hide inside one: *how many* inputs, and *where*. The two controllability notions answer them on opposite ends of the tractability scale, and the rest of the chapter takes each in turn.

9.2 Minimum Inputs for Structural Controllability

On the SC side the matching vocabulary of Section 7.2 already did the combinatorial work. Recall (Definition 7.5) that a matching on the state subgraph is a set of edges sharing no tail and no head, and that a state node is *matched* if it is some matched edge's head. Following matched edges tail-to-head decomposes the matched nodes into disjoint paths and cycles; the unmatched nodes are exactly the nodes at which those paths *start* — the nodes with no matched in-edge, at which a stem must be started from outside. Placing one input at each of them turns the matching into a disjoint set of stems and cycles covering all state nodes, which is the cover half of condition (ii) of Theorem 7.2. Counting the unmatched nodes is therefore counting the inputs, and the count is a matching invariant. The other half of that condition, input-connectedness, does not come with it (Remark 9.2).

Theorem 9.1 (Minimum inputs for SC)**.** *For a pattern graph on state nodes $\mathcal{V}^S$, let ν be the size of a maximum matching on the state subgraph. Then the minimum number of inputs rendering the family SC equals the number of unmatched state nodes, $|\mathcal{V}^S| - \nu$, or one if that number is zero (a perfect matching) [2].*

Remark 9.2 (Where the inputs attach). Theorem 9.1 counts inputs, and one input may be linked to several nodes. Attaching one input to each unmatched node is the standard choice, and since it already supplies the cover, Theorem 7.2 makes it succeed exactly when the result is input-connected — but a cycle of the matching decomposition has every one of its nodes matched, so it receives no input and can be left unreachable. On $1 \to 2$, $3 \to 4$, $4 \to 3$ the matching $\{1{\to}2,\ 3{\to}4,\ 4{\to}3\}$ is maximum, so $\nu = 3$ and node 1 is the only unmatched node; an input there gives $\mathcal{C}_F = [\,e_1,\ a_{21}e_2,\ 0,\ 0\,]$, of rank 2. The repair costs no input: link the *same* input to node 3 as well, so that $B = [\,e_1 + e_3\,]$ and $\det \mathcal{C}_F = a_{21}a_{34}^2a_{43}^3$, a single non-zero monomial. So the count of Theorem 9.1 is a count of inputs; a minimum *driver set* in the sense of Problem 9.1, one dedicated column per node, can be strictly larger.

The count is independent of *which* maximum matching is used — all have the same size — so the *number* is canonical even though the *location* is not, a non-uniqueness Section 9.4 returns to. Because a maximum matching of the bipartite graph representation is computable in $O(\sqrt{n}\,|\mathcal{E}|)$ time, the whole minimum problem for SC is polynomial [2]: the tractable end of the field. The theorem is due to Liu, Slotine, and Barabási [2], who read it as the "minimum driver nodes" that render a complex network controllable; it is the design-side form of Lin's analysis theorem, connected through the matching–cover correspondence of Section 7.2.

Example 9.3 (The four-node network already attains the minimum)**.** On the state subgraph of Fig. 4.1 the edge set $\{1{\to}2,\ 2{\to}3,\ 3{\to}4\}$ is a matching of size three (Example 7.6), and no matching of size four exists, so $\nu = 3$ and $|\mathcal{V}^S| - \nu = 1$. The one unmatched node is node 1 — its only in-edge is the cycle edge $3{\to}1$, so leaving it unmatched is the natural choice. Since $\nu = 3$ and $|\mathcal{V}^S| = 4$, any maximum matching leaves *exactly one* node unmatched: the alternative matching of Example 7.6 exposes node 4 instead, an equally valid single-input site. Theorem 9.1 thus places a single input, here at node 1. This is precisely the driver node of the running network, whose one input we took for granted throughout Part II; the minimum problem now *explains* that choice rather than assuming it. And the input gives the strongest possible outcome: the stem $u{\to}1{\to}2{\to}3{\to}4$ covers every node, so the family is SC — indeed SSC, with the single-monomial determinant $a_{21}^3a_{32}^2a_{43}$ of Example 4.25. One input, optimally placed, and nothing to improve.

Example 9.4 (The diamond needs two)**.** The diamond (Fig. 6.1) behaves in the opposite way. Its state subgraph has edges 1→2, 1→3, 2→4, 3→4, and every matching must choose between the heads 2 and 3, which share their only tail 1: a maximum matching such as {1→2, 3→4} has size two and leaves nodes 1 and 3 unmatched (Example 7.6). Hence $\nu = 2$, $|\mathcal{V}^S| - \nu = 2$, and the minimum is *two* inputs — the matching form of the dilation $S = \{2, 3\}$, $T(S) = \{1\}$, that already denied the diamond SC with one input (Example 7.3). Node 1, a source with no in-edges, must be driven directly; the second input goes to whichever twin the chosen matching left unmatched. The two parallel paths to node 4 demand two independent inputs: no single stem can cover both twins, so no single input can. We verify below (Section 9.4) that either $\{1, 2\}$ or $\{1, 3\}$ does the job, and neither is forced.

9.3 Minimum Inputs for Strong Structural Controllability

Strengthening SC to SSC replaces the matching by its worst-case counterpart from Section 7.4. There, forcing was a decision procedure: a driver set is SSC-feasible (under the free-self-loop hypothesis of Theorem 7.14) iff it is a zero forcing set, one whose color-change process blackens all of $\mathcal{V}^S$. Minimizing the number of inputs is therefore minimizing the size of a zero forcing set.

Definition 9.5 (Zero forcing number)**.** The *zero forcing number* $Z(\mathcal{G})$ of a pattern graph is the smallest cardinality of a zero forcing set (Definition 7.13).

Theorem 9.6 (Minimum inputs for SSC)**.** *Under the standing free-self-loop hypothesis of Theorem 7.14, the minimum number of driver nodes rendering the family SSC equals the zero forcing number $Z(\mathcal{G})$, and a minimum driver set is any minimum zero forcing set [23].*

The identification of the minimum driver count with $Z(\mathcal{G})$ is due to Trefois and Delvenne [23], who also cast a zero forcing set as a *constrained matching* — the uniqueness-of-matching condition that Example 7.16 met on the chain. Uniqueness is exactly what worst-case robustness costs: an SC minimum tolerates many maximum matchings, whereas an SSC minimum demands a matching that is the only one on its own node set.

That extra demand is expensive. Deciding whether a driver set forces is polynomial (apply the color-change rule until no more white nodes can be colored black), but searching for the *smallest* forcing set is a combinatorial optimization over 2^n candidate sets, and no polynomial algorithm is known — nor expected.

Remark 9.7 (NP-hardness of the SSC minimum). Computing the zero forcing number of a general graph is NP-hard, and so therefore is the SSC minimum input problem in general [24, 23]. This is not a gap in the present treatment but a feature of the worst-case guarantee: SSC requires controllability for *every* realization, and finding the lowest-cost way to obtain that guarantee is as hard as the underlying set-cover-like problem. The same hardness applies to exact (non-structural) minimal controllability [30]. The practical consequences are the usual ones — exact solution only for small or structured instances, approximation or heuristics otherwise — and they emphasize the contrast with the polynomial SC side of Theorem 9.1.

Tractability returns on structured classes, where the graph's structure determines the forcing process. The following are stated at the level of results; the constructions and proofs are the author's [20] and the survey's [3].

Remark 9.8 (Polynomial special classes). For several families the minimum forcing set — hence the SSC minimum input set — is computable in polynomial time, by decomposing the network into basic controllable components and *merging* them [20]. These component results are stated for the undirected diffusively-coupled networks of Remark 9.9 — every state edge two-way, every node self-looped — so the "end node" and "two inputs" readings below are the undirected-with-self-loops statements; a *directed* cycle, by contrast, is a single forcing chain and needs only one input. A *bridge edge* below is an edge joining two otherwise disjoint components of the decomposition — the edge that attaches a cycle bud to its stem in Lin's cacti (Theorem 7.2) is one instance — and the decompositions below are the special case in which no two components are joined by more than one such edge:

- *Paths.* A state subgraph that is a path is SSC iff a single input is attached to an *end* node (Definition 4.5); the minimum is one. The path is the simplest case — its forcing chain is the stem of Example 7.16.
- *Trees.* A state subgraph that is a tree is SSC iff it decomposes into SSC path components joined by single bridge edges; the minimum input count equals the number of such paths, a path-cover count computable in polynomial time [31, 20].
- *Cycles.* A cycle needs *two* suitably placed inputs (adjacent driven neighbours), never one: a lone input on a cycle leaves a subset with no dedicated node.
- *Pactus.* A *pactus* — a generalization of the cactus in which paths, trees, and cycles are glued along single bridge edges — is SSC iff each component is, and a merging procedure assembles the minimum external inputs by reusing internal *component input nodes*: state nodes that, seen from an adjacent component, act exactly like an external input. The merging itself is $O(|\mathcal{V}^S|)$; only the initial path-cover decomposition inherits the general NP-hardness [20].

Remark 9.9 (Diffusively-coupled networks and the self-loop hypothesis)*.* The free-self-loop hypothesis of Theorems 7.14 and 9.6 is not an idealization but the native regime of *diffusively-coupled* networks — consensus, synchronization, opinion dynamics — whose system matrix is a (modified) Laplacian $\tilde{\mathcal{L}} = \mathcal{L} - \mathcal{S}$ with a tunable self-loop $\mathcal{S}$ at each agent. There the diagonal really is a free parameter, so the zero-forcing count of Theorem 9.6 is the *exact* minimum, and the merging rules of Remark 9.8 apply verbatim [20]. One caveat, promised long ago: a Laplacian family has *dependent* entries, each diagonal being minus its row sum (Remark 2.11), so the self-loop weights are not wholly free of the off-diagonal ones. The merging analysis is carried out for exactly this modified-Laplacian model, which is why its self-loop freedom is legitimate. For a structurally loop-free pattern the case is the one shown in Example 7.15: forcing is only *sufficient*, so $Z(\mathcal{G})$ is an upper bound on the SSC minimum, and the exact count needs the two-rule / constrained-matching machinery of Remark 7.17. The four-node network is precisely such an instance — it is SSC with one input (Example 9.3), yet the color-change process starting from $\{1\}$ stops with derived set $\{1, 2\}$, so its zero forcing number overcounts.

9.4 Non-Uniqueness

The minimum fixes a number, not a placement. Different maximum matchings expose different unmatched nodes, and each is an equally valid minimum driver set; the same freedom holds for minimum zero forcing sets. The count is canonical; the location is a choice — and, as Chapter 10 shows, not a cost-neutral one.

Example 9.10 (Two minima for the diamond)**.** Example 9.4 fixed the diamond's minimum at two inputs, one of them forced at the source node 1. Where does the second go? Two maximum matchings give the two answers: $\{1\to 2,\ 3\to 4\}$ leaves $\{1,3\}$ unmatched, and $\{1\to 3,\ 2\to 4\}$ leaves $\{1,2\}$. Both $\{1,3\}$ and $\{1,2\}$ are minimum driver sets — *verify by direct computation.* With inputs at $\{1,2\}$, $B = [\,e_1,\ e_2\,]$ and the four columns $e_1,\ e_2,\ A_F e_1 = a_{21}e_2 + a_{31}e_3,\ A_F e_2 = a_{42}e_4$ have determinant $a_{31}a_{42}$; with inputs at $\{1,3\}$ the analogous determinant is $-a_{21}a_{43}$. Each is a single non-zero monomial, so *both* placements are not merely SC but SSC, at every realization. What is *not* free is the source: $\{2,3\}$ actuates neither node 1 (unreachable from the twins) nor the dilation it sits above, and fails. So the diamond admits exactly two minimum SC driver sets, $\{1,2\}$ and $\{1,3\}$ — the same count, mirror-image locations, the twin symmetry resurfacing one last time. The minimum constrains how many; symmetry, not the minimum, narrows the where.

Example 9.11 (The star: placement changes the count)**.** The twin star (Examples 5.5 and 3.8) shows this most sharply. Driven at its *center* — the symmetric, obvious choice — it is uncontrollable: the end nodes $\{2,3\}$ form a dilation fed through one node, and $\det \mathcal{C}_F \equiv 0$ (Example 5.5). A first repair keeps the center input and adds a second at an end node; with $B = [\,e_1,\ e_2\,]$ the matrix $\mathcal{C}$ has rank three, so *two* inputs restore controllability. It is tempting to stop there and declare that the star needs two inputs. It does not. A maximum matching of the star has size two and leaves exactly *one* node unmatched, so Theorem 9.1 promises a minimum of one — provided it is placed correctly. Drive a single *end node* instead of the center: with $B = e_2$,

$$\det \mathcal{C}_F \;=\; -\,a_{12}^2\,a_{31},$$

a single monomial, non-zero at every realization. One end-node input makes the star not merely SC but SSC. The central placement is what turns $\{2,3\}$ into a dilation and needs a second input; the matching count gives the correct answer. Here the “where” does not merely tune the cost — it changes the minimum count from an apparent two to an actual one. It shows as plainly as possible that a design chapter cannot read cardinality off the network without also solving for location.

So the minimum input problem returns a number and a family of placements that achieve it — one placement for the well-matched four-node network, two for the diamond, a specifically-located one for the star. Cardinality is settled; among the placements of minimum size — and the deliberately larger ones — none is yet singled out. Yet placements of equal size can differ enormously in *how much control effort* they demand — energy, speed, robustness — and that is the metric the final chapter introduces to break the tie (Chapter 10).

9.5 Minimum Inputs, Two Ways

The chapter’s two answers fit the book’s running two-column format, now read as a design summary rather than an analysis grid:

	SC (almost all)	**SSC** (all)
minimum count	*inputs*: unmatched nodes, $\lvert\mathcal{V}^S\rvert - \nu$	*driver nodes*: zero forcing number $Z(\mathcal{G})$
attained by	maximum matching (Theorem 9.1)	minimum ZFS (Theorem 9.6)
cost	polynomial [2]	NP-hard; polynomial on special classes (Remark 9.8)

The two columns count different objects: Theorem 9.1 counts *inputs*, one of which may attach to several nodes, while Theorem 9.6 counts *driver nodes*, one dedicated column each; a minimum driver set on the SC side can therefore be strictly larger than the input count (Remark 9.2). Both columns end at the same point: a count fixed, a placement free. Choosing among the free placements — by control energy, by robustness, by cost — is the optimal placement problem of Chapter 10, where the decision returns to the algebra of Gramians.

Sources. The matching solution of the SC minimum is Liu, Slotine, and Barabási [2], resting on Lin [13]; the survey [3] sets it in the wider MIP taxonomy. The zero-forcing-number identification of the SSC minimum and its constrained-matching form are Trefois and Delvenne [23], on the criterion of [22]; NP-hardness of the SSC minimum is due to [24, 23]; the polynomial special classes (paths, trees, cycles, pactus; diffusively-coupled networks) are the author's merging-rules paper [20], with the tree/path-cover case going back to [31] and the general (non-structural) minimal-controllability hardness to [30].

Chapter 10

Optimal Input Placement and Outlook

Chapter 9 closed on a deliberate limitation: the minimum input count constrains how many *drivers a network needs, but barely* where *they go, and placements of equal size can differ wildly in control effort. This closing chapter supplies the missing metric. The controllability Gramian of Chapter 3 measures a placement by three numbers — average ease, worst-direction energy, reachable volume — each an exact statement about the minimum-energy formula, and a computed comparison on the diamond shows the metrics disagreeing in an instructive way. Optimal placement is then an optimization over driver sets: combinatorial in general, greedy in practice, with guarantees worth knowing at statement level. A short duality section transfers everything to output placement, and four open problems — each scoped as a course project — leave the reader at the open questions of the field.*

10.1 Comparing Placements

Recall the promise made in Chapter 3: the Gramian

$$W(T) = \int_0^T e^{A\tau} BB^\top e^{A^\top \tau}\, d\tau$$

refines *whether* a direction is reachable into *how hard*, through the minimum-energy formula of Theorem 3.4 — steering the state from the origin to x_f in time T costs, at best,

$$E(x_f) \;=\; x_f^\top W(T)^{-1} x_f. \tag{10.1}$$

Every question about the *quality* of an input placement is a question about Eq. (10.1), hence about the spectrum of $W(T)$. Three functionals of that spectrum dominate the literature, and each has an exact reading — not an analogy — in terms of Eq. (10.1):

- *Worst direction:* over unit targets, the most expensive one costs

$$\max_{\|x_f\|=1} E(x_f) \;=\; \frac{1}{\lambda_{\min}(W)},$$

so maximizing $\lambda_{\min}(W)$ minimizes worst-case energy. Unlike the trace, this metric *guarantees* controllability (as does $\log\det W$, which is $-\infty$ exactly when W is singular): $\lambda_{\min}(W) > 0$ iff (A, B) is controllable, and of the two metrics that do, it is the one that returns a usable positive margin.

- *Average ease:* for a target drawn uniformly from the unit sphere, the expected energy is $\operatorname{tr}(W^{-1})/n$; minimizing $\operatorname{tr}(W^{-1})$ minimizes average energy. Its low-cost and popular proxy reverses the inverse: $\operatorname{tr}(W)$ is n times the spherical average of $x_f^\top W x_f$, equivalently the total impulse-response energy $\int_0^T \|e^{A\tau}B\|_F^2\, d\tau$ — a measure of how strongly the inputs excite the state on average. The proxy fails in one important way, exposed below.
- *Reachable volume:* the set of states reachable with unit energy, $\{x : x^\top W^{-1} x \le 1\}$, is an ellipsoid with semi-axes $\sqrt{\lambda_i(W)}$ along the eigenvectors of W; its volume is proportional to $\sqrt{\det W}$. Maximizing $\log\det W$ maximizes the log-volume reachable at unit energy, spreading effort over *all* directions rather than the worst or the average one.

Remark 10.1 (Which Gramian? A horizon caveat)*.* For Hurwitz A one may take $T = \infty$ and solve the Lyapunov equation of Chapter 3. The pattern used below, the diamond of Fig. 6.1, is a DAG: its A is *nilpotent*, every eigenvalue is zero, no infinite-horizon Gramian exists ($W(T)$ grows polynomially in T). Two standard repairs: fix a finite horizon T — our choice below, with $T = 1$; the integral is then exact, since $e^{A\tau}$ is a polynomial in τ — or shift $A \mapsto A - \alpha I$ with α large enough to make the matrix Hurwitz, which discounts late response and restores the Lyapunov argument (the brain-network literature stabilizes similarly, by normalizing the adjacency matrix [32]). Either way the metric *values* depend on the horizon or the shift; the zero/non-zero outcomes do not, because $\operatorname{Im} W(T) = \operatorname{Im} \mathcal{C}$ for every $T > 0$ (Theorem 3.4).

Example 10.2 (Two placements on the diamond, compared)**.** Take the diamond of Fig. 6.1 with all weights 1,

$$A = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \end{bmatrix},$$

and compare two placements of the same size two: drivers $\mathcal{D} = \{1, 2\}$, *e.g.* $B = [\,e_1\ e_2\,]$, against $\mathcal{D} = \{1, 4\}$. Since $A^3 = 0$, the matrix exponential terminates, $e^{A\tau} = I + A\tau + A^2\tau^2/2$, and the columns of $e^{A\tau}B$ are read off the graph as walk sums (Example 3.6):

$$e^{A\tau}e_1 = \begin{bmatrix} 1 \\ \tau \\ \tau \\ \tau^2 \end{bmatrix}, \quad e^{A\tau}e_2 = \begin{bmatrix} 0 \\ 1 \\ 0 \\ \tau \end{bmatrix}, \quad e^{A\tau}e_4 = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}.$$

Integrating the outer products over $\tau \in [0, 1]$ gives, exactly,

$$W_{\{1,2\}} = \begin{bmatrix} 1 & \frac{1}{2} & \frac{1}{2} & \frac{1}{3} \\ \frac{1}{2} & \frac{4}{3} & \frac{1}{3} & \frac{3}{4} \\ \frac{1}{2} & \frac{1}{3} & \frac{1}{3} & \frac{1}{4} \\ \frac{1}{3} & \frac{3}{4} & \frac{1}{4} & \frac{8}{15} \end{bmatrix}, \qquad W_{\{1,4\}} = \begin{bmatrix} 1 & \frac{1}{2} & \frac{1}{2} & \frac{1}{3} \\ \frac{1}{2} & \frac{1}{3} & \frac{1}{3} & \frac{1}{4} \\ \frac{1}{2} & \frac{1}{3} & \frac{1}{3} & \frac{1}{4} \\ \frac{1}{3} & \frac{1}{4} & \frac{1}{4} & \frac{6}{5} \end{bmatrix}.$$

The second and third rows of $W_{\{1,4\}}$ are *identical* — the twins again, visible in the matrix itself — so $e_2 - e_3$ lies in its kernel. The metric values:

metric at $T=1$	$\mathcal{D}=\{1,2\}$	$\mathcal{D}=\{1,4\}$
$\mathrm{tr}(W)$	$16/5=3.20$	$43/15\approx 2.87$
$\lambda_{\min}(W)$	≈ 0.043	0
$\det W$	$1/135$	0
$\log\det W$	≈ -4.91	$-\infty$
energy to reach e_2-e_3	16	∞

By trace the two placements look nearly interchangeable (3.20 versus 2.87); by the other two metrics they are not even in the same class. The structural chapters explain why. With drivers $\{1,2\}$ the maximal minor built from $[\,e_1,\ e_2,\ Ae_1,\ Ae_2\,]$ equals $a_{31}a_{42}$ — a single monomial, non-zero for every realization — so $\{1,2\}$ is not just controllable here but SSC (sufficiency by the single-monomial argument of Chapter 5; recall from Chapter 7 that in the multi-input case a never-vanishing minor is sufficient, not necessary). With drivers $\{1,4\}$ the controllable directions are spanned by e_1, e_4, and the *single* combination $a_{21}e_2+a_{31}e_3$: rank three for every realization — this is exactly the augmented pair of Example 8.8, where node 4 passed the FSC test, meaning a driver at 4 does not increase the generic dimension. The budget is spent on a node the placement $\{1\}$ already covered; the effective input sites (Section 8.6) are the non-fixed twins. The Gramian turns that structural conclusion into energy: splitting the twins costs 16 under $\{1,2\}$ and cannot be achieved at any finite energy under $\{1,4\}$.

The example generalizes into the working rules of the field: $\mathrm{tr}(W)$ has the lowest computational cost and is the best behaved for optimization (next section), but it guarantees nothing — an uncontrollable placement can score near the top, because an average is dominated by the easy directions. $\lambda_{\min}(W)$ guarantees everything but is the hardest to optimize, and in large networks with few drivers it is typically minuscule: worst-case control energy grows very rapidly — exponentially, under broad assumptions — as the driven fraction of nodes shrinks [33], and for targeted control it grows with the graph distance between drivers and targets [34]. $\log\det W$ sits usefully in between: sensitive to every direction, finite only for controllable placements, and blessed with the combinatorial structure the next section exploits.

10.2 Optimal Placement Problem

The design problem can now be stated. It is the point of the book where the two halves — numerical Part I, structural Parts II–III — finally recombine: the *pattern* decides which placements are feasible at all, the *numbers* decide which feasible placement is best.

Problem 10.1 (Optimal input placement problem)**.** Given a pattern A on the graph $\mathcal{G}$, a budget k, a horizon T, and a metric $f\in\{\mathrm{tr},\lambda_{\min},\log\det\}$, choose a driver set $\mathcal{D}\subseteq\mathcal{V}^S$ with $|\mathcal{D}|\le k$ — and, in the gain-design variant, the non-zero entries of a B supported on $\mathcal{D}$ — maximizing $f(W_{\mathcal{D}}(T))$ subject to the family $\mathcal{Q}(A)$ with $B_{\mathcal{D}}$ being SC (respectively SSC).

Three comments on the formulation. First, the feasibility constraint is the whole of Chapters 7 and 9 re-read as a constraint set: SC feasibility is a matching condition (Theorem 9.1), polynomially checkable, while SSC feasibility is a zero-forcing condition (Theorem 7.14, under its free-self-loop hypothesis) — polynomial to check for a given $\mathcal{D}$, though minimizing over $\mathcal{D}$ is NP-hard (Remark 9.7). Second, the metric uses a *realization*: in a structured network one evaluates f at a nominal realization, or averages it, or takes the worst case over the family $\mathcal{Q}(A)$ — the last choice being the true structural analogue and essentially unexplored (see

Section 10.4). Third, cost need not be cardinality: attaching an input to node i may carry a cost c_i, and the budget $|\mathcal{D}| \le k$ becomes $\sum_{i\in\mathcal{D}} c_i \le c$; for purely *structural* objectives of that kind — feasibility at minimum cost, no Gramian — the selection problem is solvable in polynomial time by the framework of [35], which is worth knowing before reaching for heuristics.

With a Gramian in the objective, however, the problem is genuinely combinatorial: even with all weights known exactly, selecting a minimal set of standard-basis input columns rendering (A, B) controllable is NP-hard [30], and metric optimization inherits the obstruction. The practical theory is built on one algebraic property. Because the placement matrix satisfies $B_\mathcal{D} B_\mathcal{D}^\top = \sum_{i\in\mathcal{D}} e_i e_i^\top$, the Gramian is *additive over drivers*:

$$W_\mathcal{D}(T) \;=\; \sum_{i\in\mathcal{D}} W_{\{i\}}(T), \tag{10.2}$$

one fixed positive-semidefinite increment per node, independent of the rest of $\mathcal{D}$. Consequences, at statement level, with proofs in the cited literature:

- *Trace is modular:* by Eq. (10.2), $\operatorname{tr} W_\mathcal{D} = \sum_{i\in\mathcal{D}} \operatorname{tr} W_{\{i\}}$, so the trace-optimal budget-k set is simply the k nodes with the largest individual scores — no search at all. This is the metric behind the "average controllability" rankings of network neuroscience [32].
- *Log-determinant is submodular:* $\mathcal{D} \mapsto \log\det(W_\mathcal{D} + \varepsilon I)$ (regularized so that infeasible sets score $\approx \log\varepsilon$ rather than $-\infty$) is monotone and submodular — adding a driver helps less the richer the set already is — so greedy selection, adding at each step the node with the largest marginal gain, is guaranteed a $(1 - 1/e)$-approximation of the optimum [36]. The ratio is understood for the normalized gain $g(\mathcal{D}) = \log\det(W_\mathcal{D} + \varepsilon I) - n\log\varepsilon$, which vanishes at $\mathcal{D} = \varnothing$.
- *The metric that guarantees controllability is the hardest to optimize:* $\lambda_{\min}(W_\mathcal{D})$ is neither modular nor submodular, and greedy can fail it badly; guarantees exist only in weakened or averaged forms [36, 33]. Structured heuristics — choose among the *feasible* minimum placements of Chapter 9 by a Gramian score — are the common compromise, and joint metrics mixing controllability with robustness are treated in the leader-selection literature [37].

Example 10.3 (Greedy on the diamond)**.** The single-driver traces at $T = 1$, by Eq. (10.2) and the columns computed in Example 10.2: nodes $1, 2, 3, 4$ score $\frac{28}{15}$, $\frac{4}{3}$, $\frac{4}{3}$, 1. Greedy (equivalently, top-k — trace is modular) picks node 1, then a twin: $\mathcal{D} = \{1, 2\}$, the placement that Example 10.2 showed to be SSC — here the low-cost metric and the structural conclusion happen to agree, and the additivity check $\frac{28}{15} + \frac{20}{15} = \frac{16}{5}$ confirms the table. The agreement is a coincidence, not a rule: reweight the diamond with $a_{42} = a_{43} = 3$ and the twins' scores rise to 4 each, so the trace-optimal pair becomes $\{2, 3\}$ — which leaves node 1 unreachable and is not even SC. Feasibility first, metric second: with that discipline the trace is a useful low-cost metric; without it, it selects infeasible placements.

10.3 Everything Dualizes

This book developed one body of results, and duality (Theorem 3.10) doubles it. Transpose the system and every object of Parts II–III maps to a sensing dual: drivers become output nodes, the input-walks of Chapter 4 become walks *into* those nodes — stems become branches — SC and SSC become their observability counterparts, the subspaces of Chapter 6 become statements about the kernel of $\mathcal{O}$, the node-level theory of Chapter 8 classifies which *states can be reconstructed* in every realization [28, 14], and the minimum input problem of Chapter 9 becomes minimum output placement on the edge-reversed graph. The Gramian results transfer verbatim: the observability Gramian $M(T) = \int_0^T e^{A^\top \tau} C^\top C e^{A\tau}\, d\tau$ measures the output energy $x_0^\top M(T)\, x_0$ that an initial state deposits on the output nodes, its small eigenvalues flag the directions hardest to *see*, and Problem 10.1 with $(A, B) \mapsto (A^\top, C^\top)$ is the optimal output placement problem, greedy guarantees included. Every result of this book is, silently, two results; the reader who reproves any of them in terms of sensing will find nothing new and understand everything better.

10.4 Open Problems and Course Projects

The field closes no chapter cleanly, and this book should not pretend otherwise. Four problems, each open as of this writing [3, 14], each scoped so that a semester of work produces either partial results or an instructive negative answer.

Project 1: mixed weights. Part II's dichotomy — every weight either exactly zero or freely non-zero — is a modeling idealization. Real networks are *partially structured*: some entries are fixed numbers (a circuit's fixed resistor, a known line impedance), others free (Remark 2.11 already noted that dependent entries need care). Fixing some weights shrinks the family $\mathcal{Q}(A)$, so the answers can only improve: patterns that are SC but not SSC may become controllable for the whole reduced family. *Project*: redo the trichotomy of Chapter 5 for mixed patterns, starting with the flip family of Example 3.12 with a_{11} held at one numerical value — for which values does the family become uniformly controllable? — then attempt a mixed-weight version of one graph criterion from Chapter 7.

Project 2: the exact worst-case dimension beyond HDAGs. $\dim \mathcal{C}_\Gamma$ has a tight shortest-stem lower bound and an exact algorithm only on hierarchical DAGs (Section 6.4); the dissertation conjectures the same layer logic extends to general DAGs [14]. *Project*: enumerate all single-input DAG patterns on up to six nodes, compute $\dim \mathcal{C}_\Gamma$ symbolically (one computer-algebra run per graph class, in the style of Chapter 5), and map exactly where the shortest-stem bound loses tightness once walks of different lengths reach the same node — on the diamond of Fig. 6.1 with the extra edge $1 \to 4$, node 4 is reached in two steps and in three. A proof for any class strictly beyond HDAGs would be publishable.

Project 3: FSSC nodes on general graphs. The node-level worst-case theory of Chapter 8 is effective exactly where $\dim \mathcal{C}_\Gamma$ is, and even there only in one direction: the FSSC half of Theorem 8.6 rules a node out when the added input raises $\dim \mathcal{C}_\Gamma$, but decides nothing when it does not, and $\dim \mathcal{C}_\Gamma$ itself is computable only on HDAGs, open in general. The known necessary and sufficient condition for FSSC nodes on general graphs is state-space, not graph-theoretical [10, 3]; for SSC nodes the subset condition of Definition 8.14 is graph-theoretical but exhaustive, and a polynomial criterion is known only on polytrees (Theorem 8.15). *Project*: extend the shortest-path-graph condition of Theorem 8.15 to one family with cycles (start: a single cycle with one entry point), or build a counterexample showing why that condition fails once walk lengths mix; either outcome is a result.

Project 4: energy-aware placement under structural constraints. Part II's answers are binary; energy is continuous. A pattern can be SC while a realization sits arbitrarily close to a cancellation — on the diamond, let $a_{42}a_{21} + a_{43}a_{31} \to 0$ and the energy to reach e_4 diverges — and even an SSC chain becomes practically uncontrollable as a weight tends to zero: the dissertation calls the distance to the nearest failing realization a *control margin* [14]. The true structural metric is therefore worst-case over the family, $\inf_{A\in\mathcal{Q}(A)} \lambda_{\min}(W(T))$ under a normalization of the weights — a quantity about which almost nothing is known. *Project*: compute it numerically on the four running example graphs of this book as the weights range over a box, locate the minimizing realizations relative to the cancellation loci of Chapter 5, and formulate (or refute) the natural conjecture that margins are governed by the same multi-terms that governed rank [33, 34].

The book ends where it began, and that is the point. One object — the determinant of the controllability matrix, a polynomial whose monomials are disjoint sets of stems and cycles — carried the whole development: its non-vanishing was controllability (Chapter 3), its generic and worst-case behavior split SC from SSC (Chapter 5), its surviving minors measured subspaces (Chapter 6) and nodes (Chapter 8), its combinatorics became graph criteria (Chapter 7) and placement rules (Chapter 9), and its magnitude — through the Gramian — measured the placements this chapter compared. The open problems above are the places where that object still keeps its secrets. Take one.

Sources. The metric interpretations and their limitations follow [33]; submodularity and greedy guarantees are from [36], hardness from [30], the structural cost framework from [35], leader selection from [37], and the energy-scaling and application context from [34, 32]. The open problems are drawn from the survey's concluding section [3] and the dissertation's closing chapter [14], with the node-level state of the art in [10, 28].

Appendix A

Proofs: Structural Side

The non-trivial proofs behind the SC column of the book's grid, collected here so the main text can move without interruption. Each section restates a result of Part II by cross-reference, proves it in the book's notation, and closes with a source-and-notation note recording the origin and the notation translation performed. Trivial or two-line arguments stay where they were stated; only proofs that are genuinely long appear below.

A.1 Determinant Trichotomy and Genericity Principle

We prove Proposition 5.4: for a single-input pattern ($m = 1$) with square $\mathcal{C}_F$, exactly one of the three cases holds — (i) $\det \mathcal{C}_F \equiv 0$ (never controllable); (ii) $\det \mathcal{C}_F \not\equiv 0$ but vanishing for some realization (SC, with a non-empty measure-zero uncontrollable set); (iii) $\det \mathcal{C}_F$ nowhere zero (SSC). All the content is in case (ii); (i) and (iii) are immediate from the definitions. It rests on a classical fact recorded first as a lemma.

Lemma A.1 (Genericity)**.** *A polynomial π in r real variables that is not identically zero vanishes on a set of Lebesgue measure zero in $\mathbb{R}^r$.*

Proof. Induction on r. For $r = 1$ a non-zero univariate polynomial has finitely many roots, a set of measure zero. For the step, write

$$\pi(a_1, \ldots, a_r) = \sum_{j=0}^{d} c_j(a_1, \ldots, a_{r-1})\, a_r^j.$$

Since $\pi \not\equiv 0$, some coefficient $c_j \not\equiv 0$; by the inductive hypothesis its zero set $Z \subseteq \mathbb{R}^{r-1}$ has measure zero. For every fixed $a' = (a_1, \ldots, a_{r-1}) \notin Z$ the univariate polynomial $a_r \mapsto \pi(a', a_r)$ is non-zero, hence has finitely many roots. The zero set of π therefore meets each line $\{a'\} \times \mathbb{R}$ with $a' \notin Z$ in finitely many points; by Fubini's theorem its r-dimensional measure is zero. $\square$

Proof of Proposition 5.4. Let $m = 1$, so $\mathcal{C}_F$ is $n \times n$ and $\pi(a) := \det \mathcal{C}_F$ is a polynomial in the network parameters $a = (a_1, \ldots, a_r)$, the structurally non-zero entries of A_F. The realizations of the pattern (Definition 4.1) are exactly the points of the open set $D = \{a \in \mathbb{R}^r : a_i \neq 0\ \forall i\}$, whose complement (a finite union of coordinate hyperplanes) has measure zero.

The three cases are exhaustive and mutually exclusive: either $\pi \equiv 0$, or $\pi \not\equiv 0$ and then either π has no zero in D or it has one. It remains to identify each with the stated outcome.

Case (i). If $\pi \equiv 0$ then $\det \mathcal{C}_F = 0$ for every $a \in D$, so $\operatorname{rank} \mathcal{C}_F < n$ at every realization; by the Kalman rank condition (Theorem 3.2) no realization is controllable, and in particular the family is not SC.

Case (iii). If π has no zero in D then $\det \mathcal{C}_F \neq 0$ for *every* $a \in D$, so every realization is controllable: the family is SSC by Definition 5.2.

Case (ii). Suppose $\pi \not\equiv 0$ yet $\pi(a^\circ) = 0$ for some $a^\circ \in D$. By Lemma A.1 the zero set $Z(\pi)$ has measure zero, so $Z(\pi) \cap D$ has measure zero as well. On the full-measure remainder $D \setminus Z(\pi)$ every realization is controllable, so the family is SC (Definition 5.1); the uncontrollable realizations form the set $Z(\pi) \cap D$, non-empty because a° lies in it and of measure zero by the preceding sentence. Note the constructive reading: because $D \setminus Z(\pi)$ is non-empty, *one* controllable realization already establishes SC — exhibiting a single realization with $\det \mathcal{C}_F \neq 0$ suffices, and genericity does the rest. □

Source and notation. Adapted from the definitions of SC and SSC and the worked three-node example of [14]. The genericity principle — a non-trivial polynomial vanishes only on a measure-zero variety — is classical and stated for verification against [8, 16]. Notation translated as: the source's controllability matrix $\mathcal{C}_{\mathcal{F}}$ is the book's $\mathcal{C}_F$; the source's verbal "controllable for almost all / for all network parameters" are Definitions 5.1 and 5.2.

A.2 Generic Dimension: Hosoe's Cover Theorem

We prove Theorem 6.4: $\dim \mathcal{C}_\Lambda$ equals the maximum number of state nodes that can be covered by a node-disjoint set of stems and cycles on the reachable part of the pattern graph. Write g for that cover count, computed on the subgraph $\mathcal{G}_{\text{reach}}$ induced by the reachable state nodes (Definition 4.13) together with the input nodes; the claim is $\dim \mathcal{C}_\Lambda = g$.

Proof of Theorem 6.4. By definition $\dim \mathcal{C}_\Lambda = \max_{A_F \in \mathcal{Q}(A)} \operatorname{rank} \mathcal{C}_F$. Each minor of $\mathcal{C}_F$ is a polynomial in the parameters, so by Lemma A.1 it is either identically zero or non-zero on a full-measure set; there are finitely many minors, so on the common full-measure intersection every non-identically-zero minor is simultaneously non-zero. Hence

$$\dim \mathcal{C}_\Lambda \;=\; \rho \;:=\; \max\{\, r : \text{some } r \times r \text{ minor of } \mathcal{C}_F \text{ is } \not\equiv 0 \,\}.$$

We show $\rho = g$ by two inequalities.

Lower bound $\rho \geq g$ (construction). Let $\mathcal{H}$ be a node-disjoint set of stems and cycles covering g reachable state nodes. To each covered node l attach the walk w_l that reaches it along $\mathcal{H}$: if l lies on a stem, w_l is the stem prefix from its input up to l; if l lies on a cycle, w_l is a shortest feeding stem into that cycle (which exists, since l is reachable) continued around the cycle to l. Let j_l be the length of w_l, and form the $g \times g$ submatrix M of $\mathcal{C}_F$ with rows the covered nodes and columns $A_F^{j_l} b$ (one per l). By Proposition 4.24 the $(l, \text{column } l)$ entry of M contains the WP of w_l. Because the stems and cycles of $\mathcal{H}$ are node-disjoint, the monomial obtained by multiplying these g WPs uses each edge of $\bigcup_l w_l$ a determined number of times, and the only assignment of columns to rows that reproduces it is the diagonal one — any other bijection would need a walk through a node, or of a length, already committed elsewhere, altering the monomial. That monomial therefore survives in the Leibniz expansion of $\det M$ with no cancelling term — a single surviving monomial, as in Section 5.3 — so $\det M \not\equiv 0$ and $\rho \geq g$.

Upper bound $\rho \le g$ (linking). By Poljak's generic-rank theorem [16], the generic rank ρ of $\mathcal{C}_F$ equals the maximum size of a *linking* from the inputs to the states: a family of node-disjoint input-rooted walks, one endpoint per covered state node. The traces of a maximum linking are node-disjoint paths from inputs — stems — whose revisited portions close into node-disjoint cycles; that is a disjoint set of stems and cycles, and every node it covers lies on an input-rooted walk, hence is reachable. So a linking of size ρ furnishes a disjoint stem-and-cycle cover of ρ reachable nodes, giving $\rho \le g$.

Combining, $\dim \mathcal{C}_\Lambda = \rho = g$. The restriction to $\mathcal{G}_{\text{reach}}$ is essential in the lower bound: a covered cycle disconnected from every input has identically-zero rows in $\mathcal{C}_F$ (Proposition 4.24) and contributes no surviving monomial, exactly the case isolated in Example 7.4. □

Source and notation. The theorem is Hosoe's [17] (verified only, adapted into the book's notation); the generic-rank-equals-linking identity underpinning the upper bound is Poljak's [16]. Notation translated as: the source's $|\mathcal{C}_\Lambda|$, the dimension of SCS on $\mathcal{G}(T_c)$, is the book's $\dim \mathcal{C}_\Lambda$ on the pattern graph; the source's "influenceable" restriction is the book's input-connectedness (Definition 4.13), and Λ denotes the generic (maximum) case as in the book.

A.3 Lin's Theorem: Sufficiency and Dilation Equivalence

Theorem 7.2 states three equivalent conditions for a pattern graph with at least one input: (i) SC; (ii) input-connected and some disjoint set of stems and cycles covers all state nodes; (iii) input-connected and dilation-free. The main text (Chapter 7) already gives the two necessity arguments — an unreachable node and a dilation each force a rank drop in every realization. Two implications remain: the graph equivalence (ii)⇔(iii) and the sufficiency (ii)⇒(i).

Proof that (ii)⇔(iii). Both conditions include input-connectedness, so it suffices to prove, on any pattern, that a disjoint set of stems and cycles covers all state nodes if and only if the pattern is dilation-free. Form the bipartite graph H with the state nodes $\mathcal{V}^S$ on the right, all nodes (state or input) on the left, and an edge $p - s$ whenever p feeds s in the pattern graph, i.e. $p \in T(\{s\})$ (Definition 7.1). A matching of H saturating every right node assigns each state node a *distinct* in-neighbour. By Hall's marriage theorem [5] such a saturating matching exists if and only if $|T(S)| \ge |S|$ for every non-empty $S \subseteq \mathcal{V}^S$ — that is, if and only if no dilation exists.

It remains to identify saturating matchings with covers. Given a saturating matching, read its edges in the pattern graph: every state node is a head exactly once and every node is a tail at most once, so the matched edges form node-disjoint paths and cycles covering all state nodes; a path can begin only at a node that is never a head, and since every state node is matched as a head, each path begins at an input — a stem. Thus the matching is a disjoint stem-and-cycle cover of $\mathcal{V}^S$. Conversely, in any such cover each state node has a unique in-neighbour along its stem or cycle, and these in-edges form a matching saturating $\mathcal{V}^S$. The two notions coincide, completing the equivalence. □

Proof that (ii)⇒(i). Assume every state node reachable and fix a disjoint set of stems and cycles $\mathcal{H}$ covering all state nodes. First convert $\mathcal{H}$ into *spanning cacti*. Each cycle of $\mathcal{H}$ is reachable, so choose a shortest walk from an input to it and attach the cycle to the already-built stem structure through the single edge by which that walk first enters the cycle — the *bridge* *edge*. A cycle together with its bridge edge is a *bud*, and performing this for every cycle yields node-disjoint cacti — stems carrying buds — that together span $\mathcal{V}^S$, one cactus per input used.

By Proposition 5.4 it now suffices to exhibit *one* controllable realization, since a single controllable member of a single-input family guarantees SC. Give value 1 to every stem edge, cycle edge, and bridge edge of the cacti, and value $\epsilon \neq 0$ to every remaining structurally non-zero entry. As $\epsilon \to 0$ the pair block-decomposes over the node-disjoint cacti; within one cactus the columns $b, A_F b, A_F^2 b, \ldots$ reach the stem nodes in order (a lower-triangular full-rank block on the stem) and each bud contributes one further independent direction through its bridge edge, so $\det \mathcal{C}_F\big|_{\epsilon=0} \neq 0$. The determinant is a polynomial in ϵ, hence non-zero for all sufficiently small $\epsilon \neq 0$ — a realization with $\det \mathcal{C}_F \neq 0$. By Proposition 5.4 the family is SC. □

Source and notation. The equivalence (ii)⇔(iii) is Hall's theorem applied to the bipartite graph representation [5]; the sufficiency is Lin's cacti construction [13], in the multi-input form of [19] (both third-party, verified only and rewritten in the book's notation). Notation translated as: the survey's "input-connected" [3] is the book's term as well (Definition 4.13); the survey states only the cover form (ii), while the dilation form (iii) is the book/Poljak reading via $T(S)$ (Definition 7.1); SC is established through a single controllable realization by Proposition 5.4.

A.4 Fixed-Node Input-Addition Test: Max-Dimension Side

We prove the FSC half of Theorem 8.6: on an input-connected pattern, node k is an FSC node if and only if adding a dedicated input at k — replacing B by $B_k = [\,B,\ e_k\,]$ — does not increase $\dim \mathcal{C}_\Lambda$. (The SSC-side clause of that theorem, with $\dim \mathcal{C}_\Gamma$, is only necessary, not sufficient; it is proved in the companion appendix `appB_ssc`, where Lemma B.3 supplies the exact criterion.) Throughout, $g := \dim \mathcal{C}_\Lambda$ of the pair (A_F, B) and $g' := \dim \mathcal{C}_\Lambda$ of the augmented pair (A_F, B_k); write $\mathcal{C}_{F,k}$ for the controllability matrix of the augmented pair. Since $\mathcal{C}_{F,k}$ contains the columns of $\mathcal{C}_F$, always $g' \geq g$.

The mechanism is the A_F-invariance of the controllable subspace. For a fixed realization,

$$\operatorname{Im} \mathcal{C}_{F,k} \;=\; \operatorname{Im} \mathcal{C}_F \;+\; \langle A_F \mid e_k \rangle, \qquad \langle A_F \mid e_k \rangle := \operatorname{span}\{e_k, A_F e_k, \ldots, A_F^{n-1} e_k\}, \tag{A.1}$$

because $\operatorname{Im} \mathcal{C}_{F,k}$ is the smallest A_F-invariant subspace containing both $\operatorname{Im} B$ and e_k, and $\operatorname{Im} \mathcal{C}_F$ is already the smallest one containing $\operatorname{Im} B$.

Proof of Theorem 8.6, FSC part. Recall $\mathcal{C}_\Lambda^F = \bigcap\{\operatorname{Im} \mathcal{C}_F : \operatorname{rank} \mathcal{C}_F = g\}$, the intersection over generic (maximum-dimension) realizations, and that k is an FSC node iff $e_k \in \mathcal{C}_\Lambda^F$ (Definitions 8.2 and 8.5).

(k FSC ⇒ no increase.) By Lemma A.1 the parameters at which *both* $\operatorname{rank} \mathcal{C}_F = g$ and $\operatorname{rank} \mathcal{C}_{F,k} = g'$ are attained form a full-measure set; pick a realization $A_F^\star$ in it. It is a maximum-dimension realization of the original pair, so $e_k \in \operatorname{Im} \mathcal{C}_F^\star$ because $e_k \in \mathcal{C}_\Lambda^F$. As $\operatorname{Im} \mathcal{C}_F^\star$ is $A_F^\star$-invariant, ${A_F^\star}^j e_k \in \operatorname{Im} \mathcal{C}_F^\star$ for all j, whence $\langle A_F^\star \mid e_k \rangle \subseteq \operatorname{Im} \mathcal{C}_F^\star$ and, by (A.1), $\operatorname{Im} \mathcal{C}_{F,k}^\star = \operatorname{Im} \mathcal{C}_F^\star$. Taking dimensions, $g' = \operatorname{rank} \mathcal{C}_{F,k}^\star = \operatorname{rank} \mathcal{C}_F^\star = g$: the addition does not increase $\dim \mathcal{C}_\Lambda$.

(no increase ⇒ k FSC, contrapositive.) Suppose k is not an FSC node, i.e. $e_k \notin \mathcal{C}_\Lambda^F$. Then some maximum-dimension realization $A_F^\star$ (with $\operatorname{rank} \mathcal{C}_F^\star = g$) has $e_k \notin \operatorname{Im} \mathcal{C}_F^\star$. By (A.1),

$\operatorname{Im} \mathcal{C}^\star_{F,k}$ strictly contains $\operatorname{Im} \mathcal{C}^\star_F$, so $\operatorname{rank} \mathcal{C}^\star_{F,k} \geq g+1$; hence $g' = \max \operatorname{rank} \mathcal{C}_{F,k} \geq g+1 > g$: the addition increases $\dim \mathcal{C}_\Lambda$.

The two implications together give: k is an FSC node iff adding a dedicated input at k leaves $\dim \mathcal{C}_\Lambda$ unchanged. □

Source and notation. The fixed subspace and this max-dimension input-addition test originate with Commault, van der Woude and Boukhobza [26]; the argument above is the author's restatement [14], matching the survey's SC-node theorem [3]. Notation translated as: the source's "leader" is the book's driver node, the augmented input is $B_k = [\,B,\ e_k\,]$ with e_k the standard basis vector, and the source's "dimension of SCS" is the book's $\dim \mathcal{C}_\Lambda$; the equality (A.1) is the invariance argument sketched after Theorem 8.6.

A.5 Matched-Node Criterion on Hierarchical DAGs

We prove Theorem 8.9: on an HDAG (Definition 4.21) whose drivers are its source nodes, so that the recursive layers are the distance layers, a node $i \in \ell_k$ is an FSC node if and only if i is matched in *every* feasible maximum disjoint-stem set for ℓ_k. Since the pattern is a DAG, Theorem 6.4 reduces $\dim \mathcal{C}_\Lambda$ to the maximum number of state nodes covered by disjoint *stems* (no cycles), and Theorem 8.6 reduces FSC-node membership to the non-increase of that cover count under a new driver. One structural observation links the whole-graph cover to the per-layer sets.

Lemma A.2 (Layerwise decomposition)**.** *In an HDAG (Definition 4.21) whose drivers occupy ℓ_1, so that the layers are the distance layers, every stem meets each layer in at most one node, and a node-disjoint set of stems covers the maximum number of state nodes overall if and only if, for each layer ℓ_k, it covers a maximum number of nodes of ℓ_k attainable by disjoint stems.*

Proof. No edge skips a layer. On an HDAG every walk into a node has one common length, and ℓ_r collects the nodes reachable from the inputs in exactly r steps (Definition 4.21). So if an edge runs from a node of ℓ_r to a node v, then v is reachable in $r+1$ steps, and every walk into v has that length: $v \in \ell_{r+1}$. A stem rooted in ℓ_1 therefore meets $\ell_1, \ell_2, \ldots$ in exactly one node each until it stops, reaching ℓ_k at its k-th step, and meets no layer twice.

Write $c_k(\mathcal{S})$ for the number of nodes of ℓ_k that a node-disjoint set of stems $\mathcal{S}$ covers — equivalently, the number of its stems that reach ℓ_k — and put $M_k := \max_{\mathcal{S}} c_k(\mathcal{S})$. The layers partition the state nodes, so $\mathcal{S}$ covers $\sum_k c_k(\mathcal{S})$ nodes in all, with $c_k(\mathcal{S}) \leq M_k$ term by term.

($\Leftarrow$) If $c_k(\mathcal{S}) = M_k$ in every layer then $\mathcal{S}$ covers $\sum_k M_k$ nodes, a total no set can exceed; so it covers as many state nodes as possible. ($\Rightarrow$) follows once that total is known to be attained, since a set reaching it while obeying $c_k \leq M_k$ must have $c_k = M_k$ in every layer. Two steps give the attainment.

Exchange step. If $q \leq M_k$ node-disjoint stems reach ℓ_k and end at a set X of q nodes there, some maximum disjoint-stem set for ℓ_k matches every node of X. Delete the layers below ℓ_k, those of larger index; in what is left no edge leaves ℓ_k. Compare the q stems with a set of M_k disjoint stems reaching ℓ_k and apply the standard exchange argument for systems of node-disjoint paths: it raises the count to M_k one stem at a time, re-drawing existing stems as it goes. No node of X is released on the way, because a re-drawn stem can enter ℓ_k only at its last step — no edge lets it leave again — so a node of ℓ_k that already carries the end of a stem keeps one. The ends grow from X to a set of M_k nodes containing X.

Attainment. What a set covers below ℓ_k hangs on which nodes of ℓ_k it uses, not on how they were reached, so its stems above ℓ_k may be re-drawn freely as long as those nodes are kept. Descend from the last layer ℓ_p: start with M_p disjoint stems reaching it. Given M_{k+1} disjoint stems that reach ℓ_{k+1} and attain M_j in every layer ℓ_j with $j > k$, let $X \subseteq \ell_k$ be the nodes of ℓ_k they use; each of them meets ℓ_k, so $|X| = M_{k+1}$. The exchange step replaces their parts above ℓ_k by M_k disjoint stems whose ends include X, and re-attaching below every node of X the part that hung there leaves the deeper layers as they were. The new set attains M_j in every layer with $j \geq k$. At $k = 1$ the set covers $\sum_k M_k$ nodes in all. □

Remark A.3 (Layer maxima need not add up). Without the hierarchy the (⇒) direction fails. On $1 \to 3$, $1 \to 4$, $2 \to 3$, $2 \to 4$, $2 \to 6$, $3 \to 5$, $3 \to 6$ with drivers $1, 2$, recursion gives $\{1, 2\}, \{3, 4\}, \{5, 6\}$ and each layer admits two covered nodes, yet no disjoint set of stems covers more than five of the six: the edge $2 \to 6$ skips a layer, so the per-layer maxima $2 + 2 + 2$ are not attainable together.

For a layer ℓ_k let the feasible maximum disjoint-stem sets be $\mathcal{M}^{(1)}(\ell_k), \ldots, \mathcal{M}^{(n_k)}(\ell_k)$ and let $\widehat{\mathcal{M}}^{(t)}(\ell_k)$ denote the set of nodes of ℓ_k that $\mathcal{M}^{(t)}(\ell_k)$ matches (the node of ℓ_k lying on one of its stems — at most one per stem, by Lemma A.2). The criterion is $i \in \bigcap_{t=1}^{n_k} \widehat{\mathcal{M}}^{(t)}(\ell_k)$.

Proof of Theorem 8.9. Write M_j for the maximum number of nodes of ℓ_j covered by a node-disjoint set of stems, and M'_j for the same quantity in the graph with a driver added at $i \in \ell_k$. That addition creates no edge, so in the augmented graph every edge still runs from one layer to the next and every stem — rooted in ℓ_1 or at i — still meets each layer in at most one node, over a consecutive block of layers. One step of the proof of Lemma A.2 needs care, since its downward induction is phrased for stems rooted in ℓ_1: above ℓ_k the stem rooted at i contributes nothing, so that induction runs on the ℓ_1-rooted stems alone, and the i-rooted stem rides along from ℓ_k downwards, adding one to the count in every layer from ℓ_k on and nothing above it. With that reading the largest number of state nodes covered in the augmented graph is $\sum_j M'_j$ and in the original graph $\sum_j M_j$. Every set of the original graph survives in the augmented one, so $M'_j \geq M_j$ in every layer, and the cover count is unchanged exactly when $M'_j = M_j$ throughout — by Theorems 6.4 and 8.6, exactly when i is an FSC node.

(⇐) Let i be matched in every feasible maximum disjoint-stem set for ℓ_k, and fix a layer ℓ_j. A stem rooted at i lies in $\ell_k, \ell_{k+1}, \ldots$, so for $j < k$ nothing changes and $M'_j = M_j$. For $j \geq k$ take a set $\mathcal{S}'$ of the augmented graph with $c_j(\mathcal{S}') = M'_j$. If no stem of it is rooted at i it is a set of the original graph and $M'_j \leq M_j$. Otherwise split it into the stems $\mathcal{S}'_0$ rooted in ℓ_1 and the stem P rooted at i, and let $T \subseteq \ell_k$ collect the nodes of ℓ_k that $\mathcal{S}'_0$ uses; $i \notin T$, since P holds it. Truncated at ℓ_k, $\mathcal{S}'_0$ reaches T by $|T|$ disjoint stems, so the exchange step of Lemma A.2 places T inside the matched set of some maximum disjoint-stem set for ℓ_k — and by hypothesis that matched set contains i as well. Keep the stems ending at $T \cup \{i\}$, re-attach below each node of T what hung there in $\mathcal{S}'_0$ and below i the stem P; this is legitimate for the reason recorded in Lemma A.2, that the part below ℓ_k depends only on which nodes of ℓ_k are used. Since every stem rooted in ℓ_1 that reaches ℓ_j passes through ℓ_k, the result covers every node of ℓ_j that $\mathcal{S}'$ covered, and it is a set of the original graph: $M_j \geq M'_j$. Hence $M'_j = M_j$ in every layer, and i is an FSC node.

(⇒, contrapositive) Let some feasible maximum disjoint-stem set for ℓ_k fail to match i, and let T be its matched set, so $|T| = M_k$ and $i \notin T$. None of its stems passes through i — one that did would match i — so in the augmented graph those stems and the length-zero stem $\{i\}$ are together node-disjoint and cover $M_k + 1$ nodes of ℓ_k, giving $M'_k \geq M_k + 1$. With $M'_j \geq M_j$

in the remaining layers the number of covered nodes rises, $\dim \mathcal{C}_\Lambda$ increases (Theorem 6.4), and by Theorem 8.6 i is not an FSC node. □

The single-driver corollary (Corollary 8.10) is the case $|\ell_k| = 1$. With one driver ℓ_1 holds a single node, so no two stems are node-disjoint and $M_k = 1$ in every layer; every node of ℓ_k carries a stem that reaches it at its k-th step, so the feasible maximum disjoint-stem sets for ℓ_k are exactly the stems reaching ℓ_k, one matched node apiece, and their matched nodes run over all of ℓ_k. Node i is matched in every one of them precisely when $|\ell_k| = 1$, which is Theorem 8.9 read in this case. The hierarchy is inherited from that theorem and is needed there: with a skipping edge a node alone in its layer still lies on every longest stem, yet a driver placed on it can free the old stem to take a different path and cover more (Remark 8.11). Layer by layer the criterion is a disjoint-paths computation: joining a super-source to the drivers and a super-sink to ℓ_k turns it into the m-vertex-disjoint-paths problem, solvable in $O(|\mathcal{E}| + n)$ for a single driver and $O(|\mathcal{E}|\, n^{m-1})$ for $m \geq 2$ drivers on a DAG [27].

Source and notation. Adapted from the author's own [27] (its Theorem 3, the general multi-driver form); the single-driver special case is also in [14]. The source states both results for a general p-layered DAG, in which an edge may skip a layer; the hierarchy is added here because without it the criterion is false (Remark 8.11) and the layerwise decomposition fails (Remark A.3). Notation translated as: the source's "leader" is the book's driver node (occupying ℓ_1); the source's layers $\mathcal{V}_{l_k}$ are the book's ℓ_k; the source's maximum disjoint stems $\mathcal{M}(\mathcal{V}_{l_k})$ and matched set $\widehat{\mathcal{M}}(\mathcal{V}_{l_k})$ are the book's maximum disjoint-stem set for ℓ_k and its matched set; the source's "fixed node" is the book's FSC node. The proof invokes the source's cited Commault theorem, which is the book's Theorem 8.6, and Hosoe's theorem, the book's Theorem 6.4.

Appendix B

Proofs: Strong Structural Side

The non-trivial proofs behind the SSC column of the book's grid, collected here so the main text can move without interruption. As in the companion appendix Chapter A, each section restates a result of Part II by cross-reference, proves it in the book's notation, and closes with a source-and-notation note recording the origin and the notation translation performed. The SSC side asks for controllability for every realization, so the arguments below rest on Lemma 7.8 — "full row rank for all realizations" as a dedicated-node count — proved once in Chapter 7 and reused below without reproof.

B.1 Shortest-Stem Lower Bound on $\dim \mathcal{C}_\Gamma$

Section 6.4 records, in prose, that the worst-case dimension is bounded below by the length of a shortest stem; we promote that remark to a theorem and prove it. Recall (Definition 4.9) that a *stem* is a path from an input node. Call a stem $\sigma = (u, v_1, v_2, \ldots, v_q)$ — input u followed by q state nodes — a *shortest stem* if for every j the node v_j sits at input-distance j (Definition 4.8: the distance measured from the input nodes as a whole) and $(u, v_1, \ldots, v_j)$ is the only walk of that length from an input node to v_j, so that the corresponding entry of $\mathcal{C}_F$ is a single-term. (With one input this is exactly the standing hypothesis of Section 6.4: on the diamond the stem $u \to 1 \to 2$ qualifies, its entries 1 and a_{21} being single-terms. With several inputs it matters that the distance is taken over all of them — see Remark B.2.) Let $q^\star$ be the largest number of state nodes on any such stem.

Theorem B.1 (Shortest-stem lower bound)**.** *For a single-input pattern,* $\dim \mathcal{C}_\Gamma \geq q^\star$*. More generally, with p inputs, let $m^\star$ be the largest number of state nodes that can be covered by p node-disjoint shortest stems, and suppose that for every node l and every step count k there is at most one k-step stem to l from* any *input node (multiple stems to one node are allowed, provided their step counts differ). Then* $\dim \mathcal{C}_\Gamma \geq m^\star$*.*

Proof. Single input. Let the input be u with input column b, so $\mathcal{C}_F = [\, b,\ A_F b,\ \ldots,\ A_F^{n-1} b\,]$, and fix a shortest stem $\sigma = (u, v_1, \ldots, v_q)$ with $q = q^\star$ nodes. By Proposition 4.24 the entry of $A_F^{j-1} b$ at row v_j is the SWP of all length-$(j-1)$ walks from the driver node v_1 to v_j; because σ is a shortest stem, node v_j sits at input-distance j and its shortest walk is unique, so this entry is the single non-zero term $\prod_{i=1}^{j-1} [A_F]_{v_{i+1} v_i} \neq 0$.

Permute rows and columns so that rows $v_1, \ldots, v_q$ and columns $b, A_F b, \ldots, A_F^{q-1} b$ come first, and let M be the resulting $q \times q$ leading submatrix; its $(v_w, \text{column } j)$ entry is the row-v_w

coefficient of $A_F^{j-1}b$. For $w > j$ this entry would require a walk from the driver node v_1 to v_w of length $j-1 < w-1$, contradicting that v_w lies at distance w on the shortest stem; hence all such entries vanish and M is triangular. Its diagonal entries (at $w = j$) are the single non-zero terms above, so

$$\det M \;=\; \prod_{j=1}^{q} \prod_{i=1}^{j-1} [A_F]_{v_{i+1}v_i} \;\neq\; 0 \qquad \text{for every realization.}$$

Thus $\operatorname{rank} \mathcal{C}_F \geq q$ for *every* $A_F \in \mathcal{Q}(A)$, and taking the worst case, $\dim \mathcal{C}_\Gamma \geq q = q^\star$.

Multiple inputs. Let $\sigma_1, \ldots, \sigma_p$ be p node-disjoint shortest stems covering $m^\star$ state nodes, stem σ_r rooted at input u_r with q_r state nodes, input column b_r, and j-th state node v_j^r. Select the $m^\star = \sum_r q_r$ rows v_j^r — distinct, by disjointness — and the $m^\star$ columns $A_F^{j-1} b_r$ with $1 \leq j \leq q_r$, written (r, j); order rows and columns by the step count j, *not* by stem. By Proposition 4.24 the entry of column (r, j) at row $v_w^{r'}$ collects the length-$(j-1)$ walks from v_1^r to $v_w^{r'}$, that is, the j-step stems from u_r to that node. Two counts settle the submatrix.

For $j < w$ the entry vanishes: $v_w^{r'}$ sits at input-distance w, so no walk from any input node reaches it in j steps. For $j = w$ every such walk has the minimum length, hence is a w-step stem to $v_w^{r'}$; the definition of a shortest stem — and equally the step-count hypothesis — leaves exactly one of those, the prefix of $\sigma_{r'}$, rooted at $u_{r'}$. That entry is therefore zero unless $r = r'$, when it is the prefix's single non-zero weight product. So the submatrix is triangular by step count, each step-count group carrying one non-zero entry per stem, and its determinant is, up to sign, the product over r of the single-input monomial displayed above for σ_r — non-zero for every realization. Thus $\operatorname{rank} \mathcal{C}_F \geq m^\star$ for every $A_F \in \mathcal{Q}(A)$ and $\dim \mathcal{C}_\Gamma \geq m^\star$. (Entries with $j > w$ are unconstrained; they sit above the diagonal and do not enter the determinant.) $\square$

The bound is tight on the diamond: the shortest stem $u \to 1 \to 2$ gives $q^\star = 2$, and the cancellation of Example 6.2 shows $\dim \mathcal{C}_\Gamma = 2$ exactly.

Remark B.2 (Why the count is taken over all inputs). Inputs at $1, 2$ with edges $1 \to 3$, $1 \to 4$, $2 \to 3$, $2 \to 4$: counting $u_1 \to 1 \to 3$ and $u_2 \to 2 \to 4$ would give $m^\star = 4$, but

$$\mathcal{C}_F = [\, e_1,\ e_2,\ a_{31}e_3 + a_{41}e_4,\ a_{32}e_3 + a_{42}e_4 \,]$$

has rank 3 whenever $a_{31}a_{42} = a_{32}a_{41}$, because node 3 sits two steps from *both* input nodes. Measuring each stem against its own input only would let a stem pass a node that another input node reaches sooner, and fails for the same reason.

Source and notation. Adapted from the author's dissertation [14] (its single- and multiple-input shortest-stem theorems) with path-enumeration support from [10, 3]. Notation translated as: the source writes A_Λ for the generic realization inside this bound's proof — inessential there, since the argument holds for *every* realization — which is the book's A_F; the source's $|\mathcal{C}_\Gamma|$ is $\dim \mathcal{C}_\Gamma$ (Γ the worst case, as in the book). The row-swap to a triangular submatrix in the single-input proof is the source's, rewritten in the book's notation.

Two departures from the source are recorded here, both needed for the multi-input clause to be true. First, the source's standing Assumption is not a per-stem uniqueness but the statement the theorem now carries: for every node l and every step count k there is at most one k-step stem to l from *any* input node, several stems to one node being allowed when their step counts differ. The proof draws on it only at the input-distance, where the definition of a shortest stem already secures the uniqueness; the theorem states it in full because that is the form the source assumes and the form Section 6.4 quotes. Second, the source measures a shortest stem against its own input node, which for a single input is the input-distance of Definition 4.8 but for several is weaker; the definition above takes the distance over all input nodes. Without that, a stem may run past a node that a different input node reaches in fewer steps, and the two single-terms so produced cancel for some realization. The source's multi-input proof concludes instead, from disjoint full-rank diagonal blocks, that the ranks add; that does not follow for a submatrix with arbitrary off-diagonal entries, so the proof above replaces the step by the ordering on step count, under which the whole submatrix is triangular.

B.2 Mayeda's Theorem: Two Dedicated-Node Conditions

Theorem 7.9 characterizes SSC of a single-attachment pattern by two conditions on subsets $S \subseteq \mathcal{V}^S$: (C1) every non-empty S has a dedicated node, possibly inside S; (C2) every non-empty S in which each node has an out-neighbour inside S has a dedicated node outside S. We prove the equivalence with SSC in book notation, building on Lemma 7.8 (full row rank for all realizations, proved in Chapter 7, not reproved here) and the PBH test Theorem 3.7. Throughout, recall the edge convention (Definition 2.3): column v of $[A_F\ B]$ has its non-zero rows exactly at the nodes v drives, so "a column with exactly one non-zero in the rows S" is precisely a dedicated node of S (Definition 7.7), and the diagonal of A_F is structurally zero (no self-loops in this regime).

Proof of Theorem 7.9. The family is SSC iff every realization is controllable, iff (PBH, Theorem 3.7) for every realization and every $\lambda \in \mathbb{C}$ the matrix $[\,A_F - \lambda I \;\; B\,]$ has full row rank n. We split on λ.

The $\lambda = 0$ regime is (C1). At $\lambda = 0$ the test matrix is $[\,A_F \;\; B\,]$. By Lemma 7.8 its pattern has full row rank at every realization iff every non-empty row set $S \subseteq \mathcal{V}^S$ admits a column with exactly one non-zero in S — i.e. a dedicated node of S. Since the diagonal of A_F is zero, a member $j \in S$ can serve, through its single out-edge to another member; the dedicated node ranges over all nodes, state or input, exactly as (C1) states. Hence full row rank for all realizations at $\lambda = 0 \Leftrightarrow$ (C1).

The $\lambda \neq 0$ regime is (C2). Fix $\lambda \neq 0$. We show $[\,A_F - \lambda I \;\; B\,]$ has full row rank for all realizations iff (C2) holds.

($\Leftarrow$) Assume (C1) and (C2), and suppose for contradiction that some realization admits $w \neq 0$ with $w^\top[\,A_F - \lambda I \;\; B\,] = 0$. Let $S = \operatorname{supp}(w) \neq \emptyset$. If some node $p \in S$ has no out-neighbour inside S, then column p of $A_F - \lambda I$ meets the rows S only in its diagonal entry $[A_F - \lambda I]_{pp} = -\lambda$ (as $a_{pp} = 0$), so $0 = w^\top(A_F - \lambda I)e_p = -\lambda\, w_p$ forces $w_p = 0$, contradicting $p \in \operatorname{supp}(w)$. Hence every node of S has an out-neighbour inside S, and (C2) supplies a dedicated node v outside S with a single edge into S, say $v \to j_0$ with $j_0 \in S$. If v is a state node, column v meets the rows S only at row j_0 (value $[A_F]_{j_0 v} \neq 0$; $v \notin S$ contributes no diagonal term), so $0 = w^\top(A_F - \lambda I)e_v = w_{j_0}[A_F]_{j_0 v}$ gives $w_{j_0} = 0$; if v is an input, the same computation on its column of B, using $w^\top B = 0$, gives $w_{j_0} = 0$. Either way $j_0 \in S$ is contradicted. So no such w exists and the row rank is full.

($\Rightarrow$, contrapositive) Suppose (C2) fails: some non-empty S in which every node has an out-neighbour inside S has *no* dedicated node outside S — every node outside S with an edge into S has at least two. Build a realization and a null vector as follows. Set $\lambda = 1$ and $w = \mathbf{1}_S$ (the indicator of S). Column by column of $[\,A_F - I \;\; B\,]$:

- for $j \in S$: since j has an out-neighbour inside S, the free weights $[A_F]_{ij}$ on its out-edges into S can be chosen to sum to $1 = w_j$, making $w^\top(A_F - I)e_j = \sum_{i \in S}[A_F]_{ij} - w_j = 0$; these entries belong to column j alone, so the choices never conflict;
- for a state node $j \notin S$: if j has edges into S it has at least two free weights $[A_F]_{ij}$, chosen to sum to 0; if it has none the sum is empty; either way $w^\top(A_F - I)e_j = 0$;
- for an input column: a single-attachment input driving into S would be a dedicated node outside S, excluded by the failure of (C2); so every input drives a node outside S and $w^\top B = 0$ automatically.

This realization has $w^\top[\,A_F - I \;\; B\,] = 0$ with $w \neq 0$, so PBH fails at $\lambda = 1$ and the family is not SSC. Likewise if (C1) fails, some S has every column of $[A_F \; B]$ carrying none or ≥ 2 non-zeros in S, and Lemma 7.8 produces a realization with dependent rows at $\lambda = 0$: not SSC. Contrapositively, SSC forces both (C1) and (C2). □

The two regimes are genuinely independent: (C1) is the $\lambda = 0$ test and (C2) the $\lambda \neq 0$ test, and the four-node companion passes both (Example 7.10) while the diamond and the twin star already fail (C1).

Source and notation. The result is Mayeda's [15] in the modern algebraic form of [21], with the dedicated/sharing-node reading of [14, 20] and the survey [3]. Notation translated as: the source restricts the dedicated node of (C2) to $\mathcal{N}^{\text{in}}(\alpha) \setminus \alpha$ (in-neighbours of S lying outside S), while its (C1) carries no such difference and asks only for a node of $\mathcal{N}^{\text{in}}(\alpha)$. A node with exactly one edge into S is an in-neighbour of S in any case, so the book states (C1) over all nodes — a member of S may itself serve as the dedicated node, which the $\lambda = 0$ application of Lemma 7.8 yields directly — and (C2) over all nodes outside S, the form the $\lambda \neq 0$ construction produces. The substantive translation, flagged in the Chapter 7 scope comment, is that the dedicated node may be an *input* node; without that both conditions fail on the four-node pattern. The source's α is the book's S, its $\mathcal{C}_\Gamma$ the book's $\mathcal{C}_\Gamma$, and its input-attachment convention is the single-attachment standing convention of Section 4.1. The dissertation's own single-condition form (Lemma 7.8's $\lambda = 0$ test alone) is the *self-loop* regime treated next, not general Mayeda.

B.3 Zero-Forcing Criterion under Free Self-Loops

Theorem 7.14 states that when every diagonal entry of A_F is a free parameter — a self-loop of arbitrary weight, zero included, at every state node — the family is controllable for all realizations iff the driver set is a zero forcing set (Definition 7.13), decidable in polynomial time. We prove it by collapsing the two Mayeda conditions of Section B.2 to a single one and identifying that one with the color-change rule.

Proof of Theorem 7.14. Collapse to one condition. With a free diagonal, the shift $a_{ii} \mapsto a_{ii} - \lambda$ maps the free parameter onto itself, so ranging over all realizations already realizes every real shift (a non-real λ only makes $a_{ii} - \lambda$ harder to cancel, so it imposes nothing new); running the PBH test of Section B.2 with a_{ii} free absorbs λ into the diagonal. Concretely, re-examine the two regimes. Every state node j now carries a self-loop, hence an out-edge into any S it belongs to, so:

- a loop entry is free *including zero*, so it is never a guaranteed non-zero: only a structurally non-zero pattern edge, or an input column, can supply the single non-zero that Lemma 7.8 demands. A member $j \in S$ whose only entry in the rows S is its own loop therefore supplies no such non-zero;
- and if $j \in S$ does feed another member, its column carries that edge alongside the loop entry, which is free to be non-zero, so at some realization it has two non-zeros in S. Either way no member of S can serve as the guaranteed single non-zero that Lemma 7.8 demands, so the dedicated node must come from *outside* S.

By the argument of Lemma 7.8, adapted to a diagonal that is free *including zero* — such an entry can be set to zero, so it never supplies the required single non-zero, and can be set non-zero, so it never blocks a cancellation — full row rank of $[\, A_F \;\; B\,]$ for all realizations, a test that now subsumes the $\lambda \neq 0$ regime, is therefore equivalent to the single condition

$$\begin{aligned}(\mathrm{C}^\star)\text{: every non-empty } S \subseteq \mathcal{V}^S \text{ in which each node has}\\ \text{an out-neighbour inside } S \text{ has a dedicated node } \textit{outside } S,\end{aligned} \tag{B.1}$$

the two loop-free cases of Theorem 7.9 — an inside dedicated node, and a node with no out-neighbour inside S, which (C2) does not constrain — both having disappeared. This is the sense of the main text's compact phrase "every S has a dedicated node outside S".

Forcing decides (B.1). A black node with exactly one white out-neighbour is a node with exactly one edge into the current white set — a dedicated node, lying outside that set — so a forcing step exhibits exactly the outside dedicated node required by (B.1). Running the color-change rule from the drivers until it can no longer be applied computes the derived set; the driver set is a zero forcing set iff the derived set is all of $\mathcal{V}^S$, iff no non-empty white set survives without an outside dedicated node, iff (B.1) holds. This equivalence, and that the rule reaches that point in $O(n\,|\mathcal{E}|)$ time, are the content of [22, 25]. Combining, the free-diagonal family is SSC iff the driver set is a zero forcing set, in polynomial time. □

The self-loop hypothesis cannot be dropped. Without it the equivalence breaks in one direction only: forcing remains *sufficient* for SSC of a loop-free pattern (the free-diagonal family contains the zero-diagonal one), but not necessary, the four-node companion of Example 7.15 being SSC while the process stops with a derived set smaller than $\mathcal{V}^S$ — its remaining white set $\{3, 4\}$ has an *inside* dedicated node and a node with no out-neighbour inside it, the very cases that (B.1) discards. For loop-free patterns the exact criterion remains Theorem 7.9.

Source and notation. The equivalence is Monshizadeh, Zhang and Camlibel [22], with the dedicated-node collapse and the loop-free restriction from the author's [25] and the survey [3]. Notation translated as: the source's qualitative class with free self-loops is the free-diagonal family here; its "zero forcing set" and "color-change rule" are used verbatim (Definition 7.13); the collapse is expressed through Lemma 7.8 and the Mayeda proof of Section B.2 rather than re-derived, and the four-node counterexample is Example 7.15.

B.4 Fixed-Node Input-Addition Test: Min-Dimension Side

We prove the FSSC half of Theorem 8.6: if node k is an FSSC node — $e_k \in \mathcal{C}^F_\Gamma$ (Definitions 8.2 and 8.5) — then adding a dedicated input at k, replacing B by $B_k = [\,B,\;\; e_k\,]$, does not increase $\dim \mathcal{C}_\Gamma$. Only that direction holds, so on the SSC side the dimension comparison is a necessary condition and no more: a strict increase rules k out, a non-increase decides nothing (Remark 8.7). The exact criterion is the subspace equality of Lemma B.3 below; the SC side can make do with a comparison of dimensions because $\dim \mathcal{C}_\Lambda$ is a maximum, while $\dim \mathcal{C}_\Gamma$ is a minimum. (The FSC half, with $\dim \mathcal{C}_\Lambda$, is proved in the companion appendix, Section A.4, and stays an equivalence.) Write $\mathcal{C}_{F,k}$ for the controllability matrix of the augmented pair (A_F, B_k) and $\mathcal{C}_{\Gamma,k}$ for its strongly structurally controllable subspace. Since $\mathcal{C}_{F,k}$ contains the columns of $\mathcal{C}_F$, for every realization $\operatorname{rank} \mathcal{C}_{F,k} \geq \operatorname{rank} \mathcal{C}_F$; taking worst cases, $\dim \mathcal{C}_{\Gamma,k} \geq \dim \mathcal{C}_\Gamma$ always.

The argument, as on the SC side, rests on the A_F-invariance of the controllable subspace: for a fixed realization

$$\operatorname{Im} \mathcal{C}_{F,k} \;=\; \operatorname{Im} \mathcal{C}_F \;+\; \operatorname{span}\{e_k, A_F e_k, \ldots, A_F^{n-1} e_k\}, \tag{B.2}$$

because $\operatorname{Im} \mathcal{C}_{F,k}$ is the smallest A_F-invariant subspace containing both $\operatorname{Im} B$ and e_k (Definition 3.3).

Lemma B.3 (Augmented worst case)**.** *$e_k \in \mathcal{C}_\Gamma^F$ if and only if $\operatorname{Im} \mathcal{C}_{F,k} = \operatorname{Im} \mathcal{C}_F$ at every worst-case realization (every $A_F \in \mathcal{Q}(A)$ with $\operatorname{rank} \mathcal{C}_F = \dim \mathcal{C}_\Gamma$).*

Proof. ($\Rightarrow$) If $e_k \in \mathcal{C}_\Gamma^F = \bigcap\{\operatorname{Im} \mathcal{C}_F : \operatorname{rank} \mathcal{C}_F = \dim \mathcal{C}_\Gamma\}$, then at every worst-case realization $e_k \in \operatorname{Im} \mathcal{C}_F$; A_F-invariance gives $A_F^j e_k \in \operatorname{Im} \mathcal{C}_F$ for all j, so the span in (B.2) is already contained in $\operatorname{Im} \mathcal{C}_F$ and $\operatorname{Im} \mathcal{C}_{F,k} = \operatorname{Im} \mathcal{C}_F$ there. ($\Leftarrow$) If $\operatorname{Im} \mathcal{C}_{F,k} = \operatorname{Im} \mathcal{C}_F$ at every worst-case realization, then $e_k \in \operatorname{Im} \mathcal{C}_{F,k} = \operatorname{Im} \mathcal{C}_F$ at each, so e_k lies in their intersection, i.e. $e_k \in \mathcal{C}_\Gamma^F$. □

Proof of Theorem 8.6, FSSC part. Suppose $e_k \in \mathcal{C}_\Gamma^F$. By Lemma B.3, $\operatorname{Im} \mathcal{C}_{F,k} = \operatorname{Im} \mathcal{C}_F$ at every worst-case realization; such a realization then has $\operatorname{rank} \mathcal{C}_{F,k} = \operatorname{rank} \mathcal{C}_F = \dim \mathcal{C}_\Gamma$, and since $\dim \mathcal{C}_{\Gamma,k} \geq \dim \mathcal{C}_\Gamma$ always, the worst case of the augmented pair is attained at this value: $\dim \mathcal{C}_{\Gamma,k} = \dim \mathcal{C}_\Gamma$, no increase. □

The converse fails, and it is worth seeing where. From $\dim \mathcal{C}_{\Gamma,k} = \dim \mathcal{C}_\Gamma$ one does get, at each realization attaining the augmented minimum, both $\operatorname{rank} \mathcal{C}_F = \dim \mathcal{C}_\Gamma$ and $\operatorname{Im} \mathcal{C}_{F,k} = \operatorname{Im} \mathcal{C}_F$ — but only there. That is the inclusion "worst cases of $(A_F, B_k) \subseteq$ worst cases of (A_F, B)"; the equality across *all* worst-case realizations of (A_F, B), which is what Lemma B.3 asks for, runs the other way and is the conclusion one is trying to reach. The inclusion can be strict. Attach a new node 5 to the diamond by the edge $2 \to 5$ (Remark 8.7): here $\operatorname{rank} \mathcal{C}_F = 3$ at every realization, because the e_5 entry $a_{52}a_{21}$ of its third column is a single-term, so *every* realization is a worst case of (A_F, B); the augmented $\operatorname{rank} \mathcal{C}_{F,5}$ is 4 except on $a_{42}a_{21} + a_{43}a_{31} = 0$, so the augmented minimum is still 3 and the dimension test reports no increase. Yet $e_5 \notin \operatorname{Im} \mathcal{C}_F$ off that locus, so $e_5 \notin \mathcal{C}_\Gamma^F$. The exact criterion is therefore Lemma B.3, a subspace equality at every worst-case realization; the dimension comparison of Theorem 8.6 is on this side only a necessary test, useful for ruling nodes out.

Source and notation. Author's own result [10] (its augmented-subspace lemma and FSSC-node theorem). Notation translated as: the source's $\mathcal{C}_\Gamma$ is the book's $\mathcal{C}_\Gamma$, its $\mathcal{C}_\Gamma^{\mathcal{F}}$ the book's $\mathcal{C}_\Gamma^F$, its standard basis v_i the book's e_k, and its augmented input $B_{v_i} = [B, v_i]$ the book's $B_k = [\,B,\ e_k\,]$; "i becoming a leader with additional input" is the dedicated-input addition. One departure is recorded here. What the source establishes, and what the book keeps as Lemma B.3, is the *subspace* equality $\mathcal{C}_{\Gamma v_i} = \mathcal{C}_\Gamma$ "for all network parameters"; the source then restates it as a comparison of the dimensions of the two subspaces, and that restatement does not survive, not even on the hierarchical DAGs (Definition 4.21) the source works over — the diamond with the added node 5 above is one. The book therefore keeps only the necessary direction in Theorem 8.6 and leaves the criterion with Lemma B.3. The computability caveat — the test is effective exactly where $\dim \mathcal{C}_\Gamma$ is, on hierarchical DAGs — is main text prose (Section 8.4), not part of the proof.

B.5 Shortest-Path Criterion for SSC Nodes

Theorem 8.15 localizes the dedicated-node conditions to a single node on a polytree: for an input-connected polytree with a single input node u and sinks $t_1, \ldots, t_m$, node k is an SSC node if and only if k belongs to the intersection $\mathcal{V}^u_{\text{int}}$ of the shortest-path graphs (control paths) from u to $t_1, \ldots, t_m$. Recall Definition 8.14: a set $S \subseteq \mathcal{V}^S$ is a *controllable subset* if some node outside S sends exactly one edge into S, and k is an *SSC node* if every $S \subseteq \mathcal{V}^S$ containing k is a controllable subset.

One structural fact underlies the proof. An input-connected polytree with a single input u is a tree rooted at u with every edge oriented away from u: the undirected tree has a unique u–v path to each node, and reachability from u fixes that orientation, so every state node has exactly one in-edge, from its unique in-neighbour. Membership $k \in \mathcal{V}^u_{\text{int}}$ then says that k lies on the path from u to every sink; equivalently the path $u = w_0 \to w_1 \to \cdots \to w_s = k$ is *unbranched* — each w_i $(i < s)$ has w_{i+1} as its only out-neighbour — since any branch off that path would carry a sink not reachable from k.

Proof of Theorem 8.15. ($\Leftarrow$) Let $k \in \mathcal{V}^u_{\text{int}}$, so the path $w_0 \to \cdots \to w_s = k$ is unbranched, and take any $S \subseteq \mathcal{V}^S$ with $k \in S$. Since $k = w_s \in S$, some node of that path lies in S; let w_j be the one of least index; since S contains no input node, $j \geq 1$. Its in-neighbour w_{j-1} — the input u itself when $j = 1$ — lies outside S (by minimality when $j \geq 2$), and being on the unbranched path it has w_j as its *only* out-neighbour, hence exactly one edge into S. So w_{j-1} is a dedicated node of S outside S: S is a controllable subset. As S was arbitrary, k is an SSC node.

($\Rightarrow$, contrapositive) Let $k \notin \mathcal{V}^u_{\text{int}}$ and set $S = \mathcal{V}^S \setminus \mathcal{V}^u_{\text{int}}$, which contains k. Let z be the last node of the path $\mathcal{V}^u_{\text{int}}$ — the node at which the path branches. Every node strictly above z on that path has its single out-neighbour inside $\mathcal{V}^u_{\text{int}}$, hence no edge into S; the input u attaches inside $\mathcal{V}^u_{\text{int}}$ as well. So the only node *outside* S with edges into S is z itself — and beyond z the paths to different sinks diverge, so z has at least two out-neighbours, each opening a branch that lies in S. Thus z sends at least two edges into S and is a sharing node, leaving S with no dedicated node outside S: S is not a controllable subset. Since $k \in S$, k is not an SSC node. □

For several inputs, taking the union over inputs of the per-input intersections $\mathcal{V}^{u_j}_{\text{int}}$ still yields SSC nodes:

Corollary B.4 (Multiple inputs)**.** *On an input-connected polytree with inputs $u_1, \ldots, u_p$, every node in $\bigcup_j \mathcal{V}^{u_j}_{\text{int}}$ is an SSC node.*

Proof. If $k \in \mathcal{V}^{u_j}_{\text{int}}$ for some input u_j, the unbranched-path argument of the ($\Leftarrow$) direction, applied within the part reachable from u_j, furnishes a dedicated node outside each $S \ni k$ along the u_j-path; the presence of further inputs only adds more candidate dedicated nodes. Hence every such S is a controllable subset and k is an SSC node. □

Source and notation. Author's own [28] (its single-input path theorem and multi-input corollary), with the controllable-subset and SSC-node definitions of [20]. Notation translated as: the source's "acyclic once the edge directions are ignored" is the book's *polytree* and its *terminal nodes* are the book's sinks (Definition 4.20); the source's subsets α are written S here, as in Theorem 7.9; its shortest-path graphs $\mathcal{P}_i^u$ and their intersection $\mathcal{P}_{\text{int}}^u$ are the book's control paths and their intersection $\mathcal{V}_{\text{int}}^u$; the union graph $\mathcal{P}_{\text{int}}^c$ over inputs is $\bigcup_j \mathcal{V}_{\text{int}}^{u_j}$; and "SSC node" is used in the book's sense (every subset containing k controllable), which is the dissertation's, *not* the survey's min-dimension usage (Remark 8.18). The rooted-tree observation makes the source's "$\mathcal{P}_{\text{int}}^u$ is a path" step explicit in the book's notation.

B.6 SSC Nodes Are FSC Nodes

Proposition 8.16 states that on an input-connected polytree with a single input node every SSC node is an FSC node, and that with several inputs the same conclusion holds for every node of the union $\bigcup_j \mathcal{V}_{\text{int}}^{u_j}$ of the per-input intersections. The main text gives only a one-line sketch; here is the argument.

Proof of Proposition 8.16. Single input. Let k be an SSC node of an input-connected polytree with input u. By Theorem 8.15, $k \in \mathcal{V}_{\text{int}}^u$: the path $u \to \cdots \to k$ is unbranched, so *every* directed path from u passes through k (any path must descend it before the first branch point, which lies at or below k). With a single input the maximum disjoint stem-and-cycle cover of Theorem 6.4 is realized by a single stem — a longest directed path from u — and every such stem therefore contains k. Adding a dedicated input at k contributes only a stem whose single state node is k, which overlaps every such stem at k and claims no new node; the cover count, hence $\dim \mathcal{C}_\Lambda$, is unchanged. By the FSC half of Theorem 8.6 (proved in Section A.4), k is an FSC node.

Multiple inputs. Here the hypothesis is the one the several-input clause carries: $k \in \mathcal{V}_{\text{int}}^{u_j}$ for some input u_j, which by Corollary B.4 already makes k an SSC node. Within the part reachable from u_j the path $u_j \to \cdots \to k$ is unbranched and every u_j-rooted stem passes through k. In a maximum disjoint stem-and-cycle cover the stem rooted at u_j then contains k, so a dedicated input at k re-traces that stem and raises no cover count; $\dim \mathcal{C}_\Lambda$ is unchanged and k is an FSC node by the FSC half of Theorem 8.6. □

This is distinct from the inclusion of the next section: here the antecedent is an *SSC node* (the subset/path notion), not the FSSC-node membership $e_k \in \mathcal{C}_\Gamma^F$, and neither reading is claimed to imply the other (Remark 8.18) — the pruned diamond of Example 8.17 has node 4 FSSC but not an SSC node.

Source and notation. Author's own [28]: its single-input theorem, which reads "every SSC node is a fixed controllable node", and its multi-input theorem, which reads "every node of $\bigcup_j \mathcal{V}_{\text{int}}^{u_j}$ is a fixed controllable node". Notation translated as before: SSC node in the book/dissertation sense, control paths $\mathcal{V}_{\text{int}}^{u_j}$, and the FSC input-addition test = the source's cited Commault criterion = book Theorem 8.6. Each half proceeds through Theorem 8.15 exactly as the source does through its path theorem; with one input the two hypotheses coincide there, so that half covers every SSC node.

B.7 Fixed-Subspace Inclusion $\mathcal{C}_\Gamma^F \subseteq \mathcal{C}_\Lambda^F$

Section 8.2 defers to the appendix the inclusion $\mathcal{C}_\Gamma^F \subseteq \mathcal{C}_\Lambda^F$ and its node-level corollary, every FSSC node is an FSC node. The subtlety, flagged in the main text, is that the two fixed subspaces are intersections over *different* realization sets — $\mathcal{C}_\Gamma^F$ over the worst-case (minimum-dimension) realizations, $\mathcal{C}_\Lambda^F$ over the generic (maximum-dimension) ones — so the inclusion is not a matter of intersecting over a larger set. We prove the node-level statement in full and then lift it.

Proposition B.5 (Node-level inclusion)**.** *Every FSSC node is an FSC node: $e_k \in \mathcal{C}_\Gamma^F \Rightarrow e_k \in \mathcal{C}_\Lambda^F$.*

Proof. Let k be an FSSC node, so $e_k \in \mathcal{C}_\Gamma^F$: by Definition 8.2, e_k lies in $\operatorname{Im}\mathcal{C}_F$ at every worst-case realization. We claim a dedicated input at k does not increase $\dim\mathcal{C}_\Lambda$. The argument runs from that membership and not from the FSSC input-addition test, which on this side holds in one direction only (Section B.4).

The argument below assumes an input-connected HDAG (Definition 4.21) with a single driver node, the setting in which [10] computes the worst-case subspace; the general case is left to that source. There the worst-case subspace is computed through the subgraph reduction of [10]: its $\bar{\mathcal{G}}$ keeps every node of $\mathcal{G}$ and drops every edge meeting a node reached from the driver node in d or more steps, where d is the first step count at which the nodes reached in exactly d steps are all integrators or intermediators (Definition 4.30) whose multi-terms can be set to zero together; below d the edges are those of $\mathcal{G}$. The reduction carries the membership to $\bar{\mathcal{G}}$, so e_k lies in every worst-case realization of the controllable subspace there; by the shortest-stem account of Section B.1 this puts k on a stem of every maximum disjoint stem-and-cycle cover of $\bar{\mathcal{G}}$, and hence — since $\bar{\mathcal{G}}$ preserves the upstream edges — on every maximum disjoint stem-and-cycle cover of $\mathcal{G}$. A dedicated input at k therefore re-traces a stem of such a cover and adds no covered node, so $\dim\mathcal{C}_\Lambda$ is unchanged (Theorem 6.4). By the FSC test (Theorem 8.6, proved in Section A.4), k is an FSC node. ☐

Proof that $\mathcal{C}_\Gamma^F \subseteq \mathcal{C}_\Lambda^F$. Take $v \in \mathcal{C}_\Gamma^F$. The augmented-subspace argument of Lemma B.3 does not use that the added column is a standard basis vector — for any fixed vector v, $\operatorname{Im}[\,\mathcal{C}_F,\ A_F^j v\,]_j = \operatorname{Im}\mathcal{C}_F + \operatorname{span}\{v, A_F v, \dots\}$ — so the same proof gives the subspace form on both sides: $v \in \mathcal{C}_\Gamma^F$ exactly when augmenting B by v leaves the controllable subspace unchanged at every worst-case realization, and $v \in \mathcal{C}_\Lambda^F$ exactly when it leaves that subspace unchanged at every generic realization. On the SC side, where $\dim\mathcal{C}_\Lambda$ is a maximum, the second is in turn equivalent to $\dim\mathcal{C}_\Lambda$ being unchanged; on the SSC side no such dimension form is available (Section B.4), and none is needed here. Applying Proposition B.5's reduction to the direction v in place of e_k — $v \in \mathcal{C}_\Gamma^F$ means v lies in every worst-case subspace on $\bar{\mathcal{G}}$, hence in every maximum disjoint stem-and-cycle cover, hence in every generic subspace of $\mathcal{G}$ — shows that augmenting by v leaves $\dim\mathcal{C}_\Lambda$ unchanged as well. Therefore $v \in \mathcal{C}_\Lambda^F$, and $\mathcal{C}_\Gamma^F \subseteq \mathcal{C}_\Lambda^F$. The node corollary is the special case $v = e_k$ (Proposition B.5). ☐

Source and notation. The node-level inclusion is the author's [14] (every FSSC node is an FSC node, via the subgraph reduction of its Algorithm); the subspace-level inclusion is from [10], where the four-subspace framework and the containment of Fig. 6.2 originate. Notation translated as: the source's subgraph $\bar{\mathcal{G}}(T_c)$, written $\bar{\mathcal{G}}(\mathcal{V}, \bar{\mathcal{E}})$ in the published paper, is $\bar{\mathcal{G}}$; its $\mathcal{C}_\Gamma^{\mathcal{F}} \subseteq \mathcal{C}_\Lambda^{\mathcal{F}}$ is the book's $\mathcal{C}_\Gamma^F \subseteq \mathcal{C}_\Lambda^F$; "leader" is driver node; and the disjoint stem-and-cycle characterization of both tests is Hosoe's theorem, the book's Theorem 6.4. The proof compares intersections over the min- and max-dimension realization sets by comparing both on the same maximum disjoint stem-and-cycle covers of $\bar{\mathcal{G}}$, which is the reconciliation the main text flagged as the subtlety of the inclusion.

Bibliography

[1] C.-T. Chen, *Linear System Theory and Design*, 3rd ed. New York: Oxford University Press, 1999.

[2] Y.-Y. Liu, J.-J. Slotine, and A.-L. Barabási, “Controllability of complex networks,” *Nature*, vol. 473, no. 7346, pp. 167–173, 2011.

[3] N.-J. Park and H.-S. Ahn, “Network controllability of structured networks: A survey on graph-theoretical approaches,” 2026, manuscript.

[4] H. K. Khalil, *Nonlinear systems.* Prentice Hall, 2002.

[5] C. Godsil and G. F. Royle, *Algebraic graph theory.* Springer Science & Business Media, 2013.

[6] R. Olfati-Saber and R. M. Murray, “Consensus problems in networks of agents with switching topology and time-delays,” *IEEE Transactions on Automatic Control*, vol. 49, no. 9, pp. 1520–1533, 2004.

[7] W. Ren and R. W. Beard, “Consensus seeking in multiagent systems under dynamically changing interaction topologies,” *IEEE Transactions on Automatic Control*, vol. 50, no. 5, pp. 655–661, 2005.

[8] J.-M. Dion, C. Commault, and J. V. der Woude, “Generic properties and control of linear structured systems: a survey,” *Automatica*, vol. 39, no. 7, pp. 1125–1144, 2003.

[9] N.-J. Park, Y.-B. Bae, K. L. Moore, and H.-S. Ahn, “Controllability matrix analysis of structured networks: A tight lower bound on the dimension of controllable subspace,” in *2024 European Control Conference (ECC)*, 2024, pp. 1165–1170.

[10] N.-J. Park, Y.-U. Kim, K.-H. Oh, and H.-S. Ahn, “Controllable subspaces in structured networks of hierarchical directed acyclic graphs: Controllability of individual nodes,” *IEEE Transactions on Automatic Control*, vol. 71, no. 4, pp. 2630–2636, 2026.

[11] J. Gao, Y.-Y. Liu, R. M. D’souza, and A.-L. Barabási, “Target control of complex networks,” *Nature Communications*, vol. 5, no. 1, p. 5415, 2014.

[12] C. Commault, J. van der Woude, and P. Frasca, “Functional target controllability of networks: Structural properties and efficient algorithms,” *IEEE Transactions on Network Science and Engineering*, vol. 7, no. 3, pp. 1521–1530, 2019.

[13] C.-T. Lin, “Structural controllability,” *IEEE Transactions on Automatic Control*, vol. 19, no. 3, pp. 201–208, 1974.

[14] N.-J. Park, “Controllability and controllable subspaces of structured networks: A graph-theoretical approach,” Ph.D. dissertation, Gwangju Institute of Science and Technology, 2024.

[15] H. Mayeda and T. Yamada, “Strong structural controllability,” *SIAM Journal on Control and Optimization*, vol. 17, no. 1, pp. 123–138, 1979.

[16] S. Poljak, “On the generic dimension of controllable subspaces,” *IEEE Transactions on Automatic Control*, vol. 35, no. 3, pp. 367–369, 1990.

[17] S. Hosoe, “Determination of generic dimensions of controllable subspaces and its application,” *IEEE Transactions on Automatic Control*, vol. 25, no. 6, pp. 1192–1196, 1980.

[18] A. Y. Yazicioglu, M. Shabbir, W. Abbas, and X. Koutsoukos, “Strong structural controllability of diffusively coupled networks: Comparison of bounds based on distances and zero forcing,” in *Proceedings of the IEEE Conference on Decision and Control (CDC)*, 2020, pp. 566–571.

[19] R. W. Shields and J. B. Pearson, “Structural controllability of multiinput linear systems,” *IEEE Transactions on Automatic Control*, vol. 21, no. 2, pp. 203–212, 1976.

[20] N.-J. Park, S.-H. Kwon, Y.-B. Bae, B.-Y. Kim, K. L. Moore, and H.-S. Ahn, “Composition rules for strong structural controllability and minimum input problem in diffusively-coupled networks,” *arXiv preprint arXiv:2405.05557*, 2024.

[21] G. Reissig, C. Hartung, and F. Svaricek, “Strong structural controllability and observability of linear time-varying systems,” *IEEE Transactions on Automatic Control*, vol. 59, no. 11, pp. 3087–3092, 2014.

[22] N. Monshizadeh, S. Zhang, and M. K. Camlibel, “Zero forcing sets and controllability of dynamical systems defined on graphs,” *IEEE Transactions on Automatic Control*, vol. 59, no. 9, pp. 2562–2567, 2014.

[23] M. Trefois and J.-C. Delvenne, “Zero forcing number, constrained matchings and strong structural controllability,” *Linear Algebra and its Applications*, vol. 484, pp. 199–218, 2015.

[24] S. S. Mousavi, M. Haeri, and M. Mesbahi, “On the structural and strong structural controllability of undirected networks,” *IEEE Transactions on Automatic Control*, vol. 63, no. 7, pp. 2234–2241, 2017.

[25] N.-J. Park, Y.-U. Kim, and H.-S. Ahn, “Strong structural controllability of directed graphs via zero forcing sets,” *IFAC-PapersOnLine*, vol. 56, no. 2, pp. 3441–3446, 2023.

[26] C. Commault, J. V. der Woude, and T. Boukhobza, “On the fixed controllable subspace in linear structured systems,” *Systems & Control Letters*, vol. 102, pp. 42–47, 2017.

[27] N.-J. Park, Y.-U. Kim, and H.-S. Ahn, “Fixed node determination and analysis in directed acyclic graphs of structured networks,” *Journal of the Franklin Institute*, vol. 361, p. 106995, 2024.

[28] N.-J. Park, Y.-U. Kim, H.-S. Ahn, and K. L. Moore, “Strong structural controllability and observability of individual nodes: A graph-theoretical approach,” *IEEE Transactions on Automatic Control*, vol. 69, no. 4, pp. 2598–2604, 2024.

[29] C. Commault, J. V. der Woude, and T. Boukhobza, “A dynamic graph characterisation of the fixed part of the controllable subspace of a linear structured system,” *Systems & Control Letters*, vol. 129, pp. 17–25, 2019.

[30] A. Olshevsky, “Minimal controllability problems,” *IEEE Transactions on Control of Network Systems*, vol. 1, no. 3, pp. 249–258, 2014.

[31] F. Barioli, W. Barrett, S. M. Fallat, H. T. Hall, L. Hogben, B. Shader, P. V. den Driessche, and H. V. der Holst, “Zero forcing parameters and minimum rank problems,” *Linear Algebra and its Applications*, vol. 433, no. 2, pp. 401–411, 2010.

[32] S. Gu, F. Pasqualetti, M. Cieslak, Q. K. Telesford, A. B. Yu, A. E. Kahn, J. D. Medaglia, J. M. Vettel, M. B. Miller, and S. T. Grafton, “Controllability of structural brain networks,” *Nature Communications*, vol. 6, no. 1, p. 8414, 2015.

[33] F. Pasqualetti, S. Zampieri, and F. Bullo, “Controllability metrics, limitations and algorithms for complex networks,” *IEEE Transactions on Control of Network Systems*, vol. 1, no. 1, pp. 40–52, 2014.

[34] I. Klickstein, A. Shirin, and F. Sorrentino, “Energy scaling of targeted optimal control of complex networks,” *Nature Communications*, vol. 8, no. 1, p. 15145, 2017.

[35] S. Pequito, S. Kar, and A. P. Aguiar, “A framework for structural input/output and control configuration selection in large-scale systems,” *IEEE Transactions on Automatic Control*, vol. 61, no. 2, pp. 303–318, 2016.

[36] T. H. Summers, F. L. Cortesi, and J. Lygeros, “On submodularity and controllability in complex dynamical networks,” *IEEE Transactions on Control of Network Systems*, vol. 3, no. 1, pp. 91–101, 2016.

[37] K. Fitch and N. E. Leonard, “Optimal leader selection for controllability and robustness in multi-agent networks,” in *Proceedings of the European Control Conference (ECC)*, 2016, pp. 1550–1555.